

(BCS501) DBMS Unit-1st

1. OVERVIEW OF A DATABASE SYSTEM

WHAT IS DATA?

Data is raw facts or figures. For example:

- A name: “John”
- A number: “25”
- A date: “2025-06-27”

When this data is **organized in a meaningful way**, it becomes **information**.

WHAT IS A DATABASE?

A **database** is a **collection of related data** that is organized so it can be easily accessed, managed, and updated.

A **database** is like a **digital place** where we keep information in an organized way so we can find it later. Think of it **like a notebook or a file folder**, but on a computer.

- It allows storing large amounts of data efficiently.
- Data is structured in tables made up of rows (records) and columns (fields).

Example:

Imagine you have a list of your friends' names and phone numbers. That list is your **data**, and if you store it on your computer in an organized way, it becomes a **database**.

Example:

A **Student Database** might contain data like:

Student-ID	Name	Age	Class
1	John	15	10
2	Sara	14	9

WHAT IS A DBMS?

A **Database Management System (DBMS)** is software that **creates, manages, and operates databases**.

FUNCTIONS OF A DBMS:

- Store data efficiently
- Allow users to **insert, update, delete, and retrieve** data
- Ensure **data consistency** and **integrity**
- Provide **security** and **user access control**
- Handle **multiple users** accessing the database at the same time

Popular DBMS software includes:

- MySQL

- Oracle
- PostgreSQL
- Microsoft SQL Server
- SQLite

2. DATABASE SYSTEM vs FILE SYSTEM

Feature	File System	Database System (DBMS)
Definition	A method to store and organize files on a computer	Software system to store and manage data in a structured way
Structure	Data stored in separate files (.txt, .doc, etc.)	Data stored in tables (rows and columns)
Data Redundancy	High – same data might be repeated in multiple files	Low – data stored once and reused
Data Integrity	Hard to maintain	Maintained through constraints and relationships
Security	Basic file-level protection	Strong user-based access control
Searching	Manual or through limited tools	Powerful querying using SQL
Concurrency (Multi-user access)	Difficult	Built-in support for multiple users
Backup & Recovery	Manual	Automatic and managed by the DBMS
Scalability	Limited	Highly scalable

3. DATABASE SYSTEM CONCEPT

A **Database System** is made up of two main parts:

- **Database (DB):** This is the place where all the data is stored. For example, a collection of student records, or a list of books in a library.
- **Database Management System (DBMS):** This is the software that helps you create, manage, and use the database. It allows you to add, update, delete, and retrieve data safely and easily.

Together, the **Database + DBMS = Database System**.

4. WHY DO WE NEED A DATABASE SYSTEM?

- To **store a large amount of data** in an organized way.
- To **avoid data duplication** and inconsistency.
- To **share data** among many users.

- To **secure data** and control who can see or change it.
- To **make data access fast and efficient**.

5. ARCHITECTURE OF DATABASE SYSTEM

The architecture explains how different parts of the database system are organized and interact with each other. This system is called **Three-Level Architecture** and helps **hide complexity** and **make the database easier to use and maintain**.

It is usually shown in **three levels**:

a) Internal Level (Physical Level)

- This is the **lowest level**.
- It deals with how data is actually stored in the computer's memory (like files, indexes, and physical storage).
- It's about the **physical storage of data** on disks or drives.
- Users usually don't see this level.

b) Conceptual Level (Logical Level)

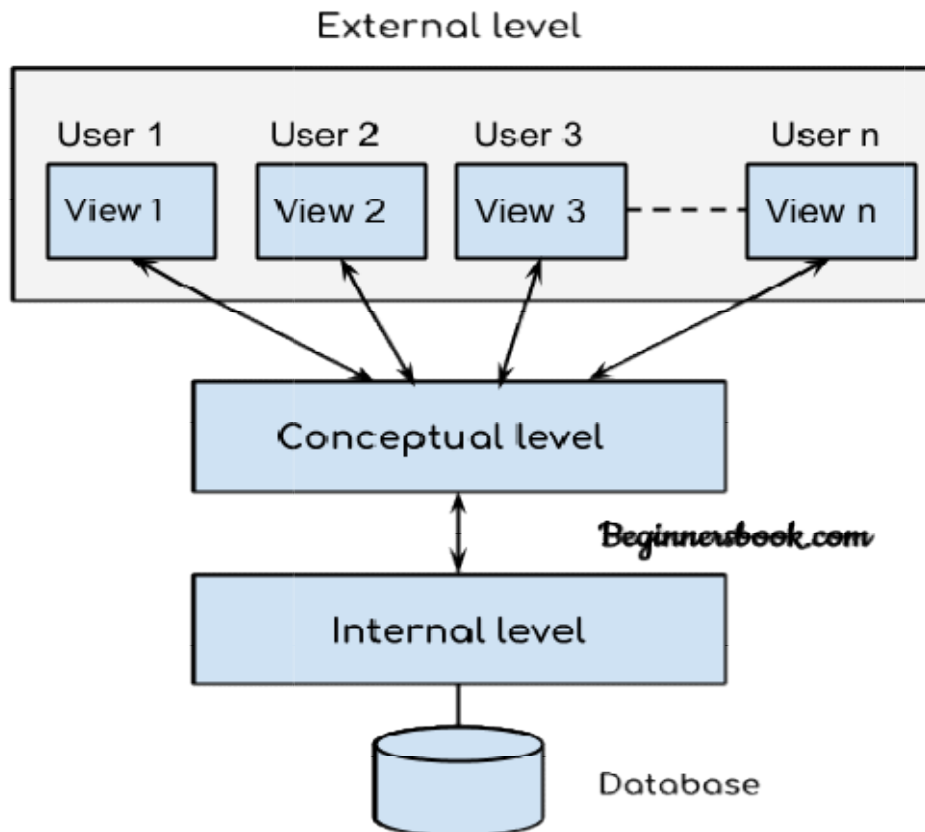
- This level describes **what data is stored** and the relationships between data.
- It shows the whole structure of the database — all entities, their attributes, and relationships.
- It hides the details of physical storage.
- This is what database designers focus on.

c) External Level (View Level)

- This is the **highest level**.
- It shows **different views of the data** to different users or applications.
- For example, a student might see only their own data, but an admin sees everything.
- It hides parts of the database that a user doesn't need to see.

How These Levels Work Together

- When you request data (like asking for a student's grade), your request goes from the **external level** (your view),
- To the **conceptual level** (database structure and rules),
- And finally to the **internal level** (how data is actually stored and retrieved).



DATA MODEL, SCHEMA, AND INSTANCES — DETAILED EXPLANATION

A. DATA MODEL:-

A **data model** is a formal way to describe how data is organized and how the relationships among data are structured. It acts like a **blueprint or a plan** that defines what kind of data will be stored, how it will be stored, and how different pieces of data relate to each other.

Think of a data model as a set of rules and guidelines that help designers and programmers decide how to build a database. It tells you:

- What types of data you will keep (like names, dates, numbers).
- How different data items connect with each other (for example, a student is linked to a class).
- What constraints or rules the data must follow (such as a student must have a unique ID).

There are different kinds of data models, each suited for different types of databases:

1. RELATIONAL MODEL:

The most common one, where data is stored in tables (called relations) consisting of rows and columns.

Table also called Relation

© guru99.com

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Primary Key (points to CustomerID)

Domain (points to CustomerName, Ex: NOT NULL)

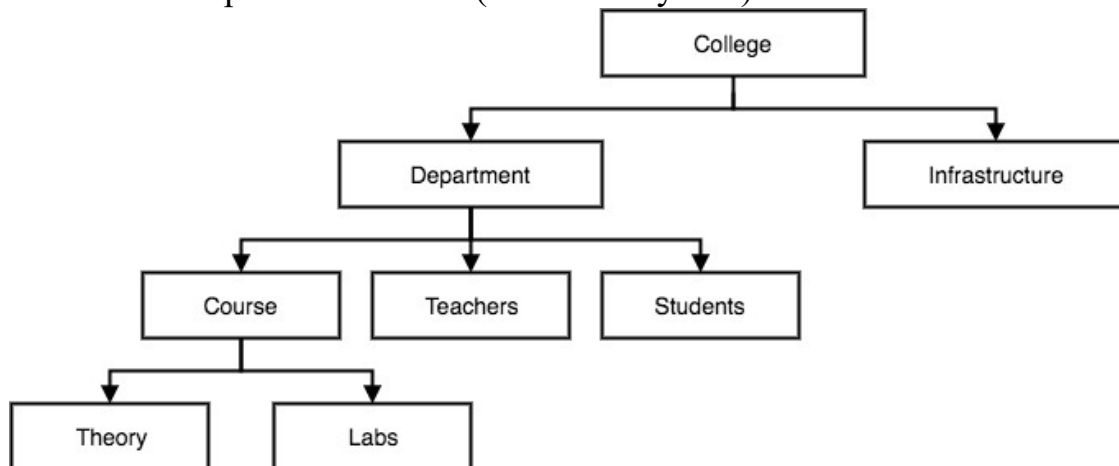
Tuple OR Row (points to the rows)

Total # of rows is Cardinality

Column OR Attributes (points to the columns)

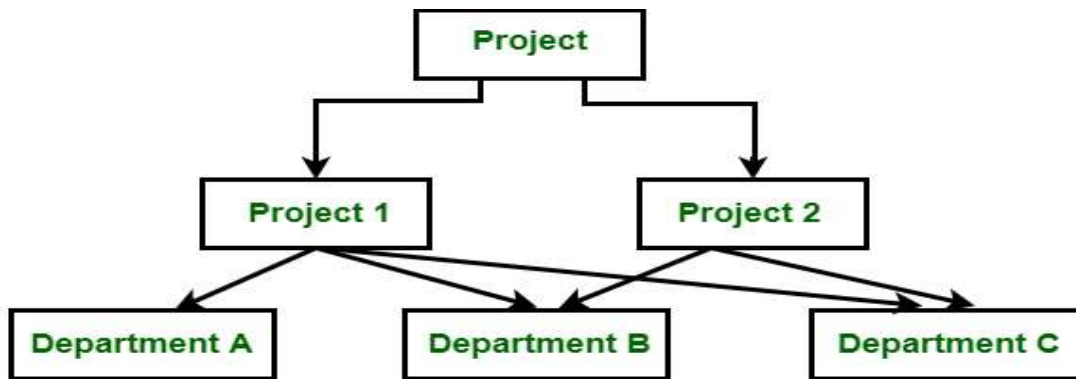
Total # of column is Degree

- 3. HIERARCHICAL MODEL:** Organizes data in a tree-like structure, where each record has a parent and child (like a family tree).



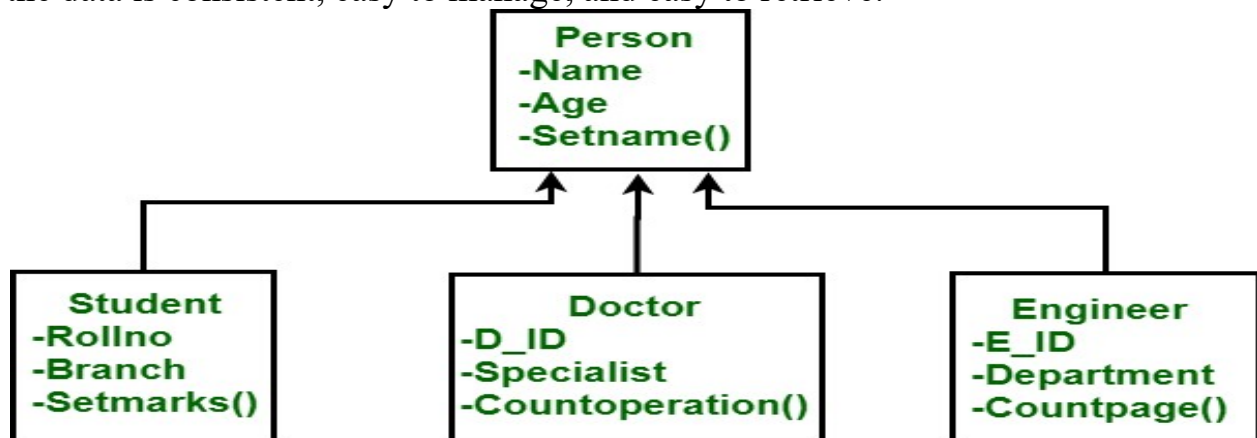
4. NETWORK MODEL:

More flexible than hierarchical, it organizes data as a graph where records can have multiple parent and child relationships.



5. OBJECT-ORIENTED MODEL: Uses the concept of objects (from programming) to store data along with the operations that can be performed on them.

The data model helps in designing the database properly before it is built so that the data is consistent, easy to manage, and easy to retrieve.



B. SCHEMA:-

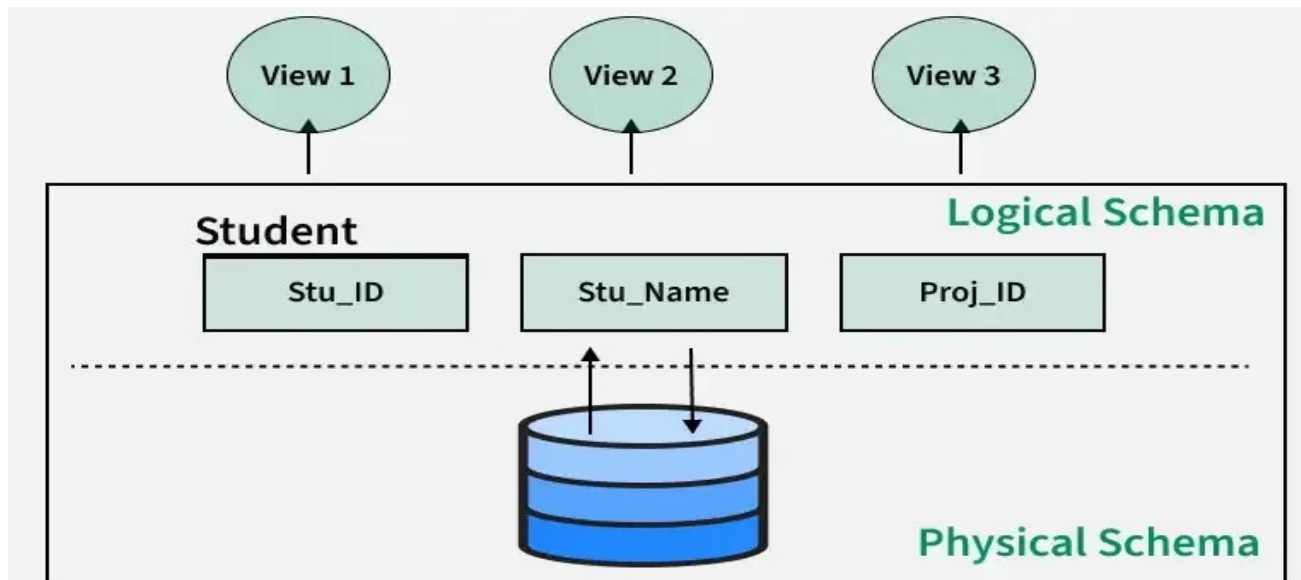
The **schema** is the actual **structure or design** of the database according to the data model you choose. It defines:

- What tables or entities exist in the database.
- What attributes (or columns) each table has.
- How tables or entities relate to one another.
- Rules and constraints on data (like which attributes are required, or unique).

You can think of the schema as the **blueprint or map** for the database that shows the organization of data but without the actual data itself.

For example, if you have a database for a school, the schema would define that there is a **Student** table with columns like StudentID, Name, Age, and Class. It would also define a **Teacher** table with TeacherID, Name, and Subject.

The schema is usually **fixed or stable** and changes only when you redesign the database structure, for example, by adding new tables or columns.



C. INSTANCE

The **instance** of a database is the **actual data stored in the database at a specific moment in time**. It is like a snapshot of the database content.

If the schema is the blueprint or plan, the instance is the real building constructed using that blueprint.

For example, the Student table defined by the schema might have these actual records stored in the database right now:

Student-ID	Name	Age	Class
101	Alice	12	6A
102	Bob	13	7B

This data is called the instance. It changes regularly as new data is added, updated, or deleted.

How They Relate to Each Other

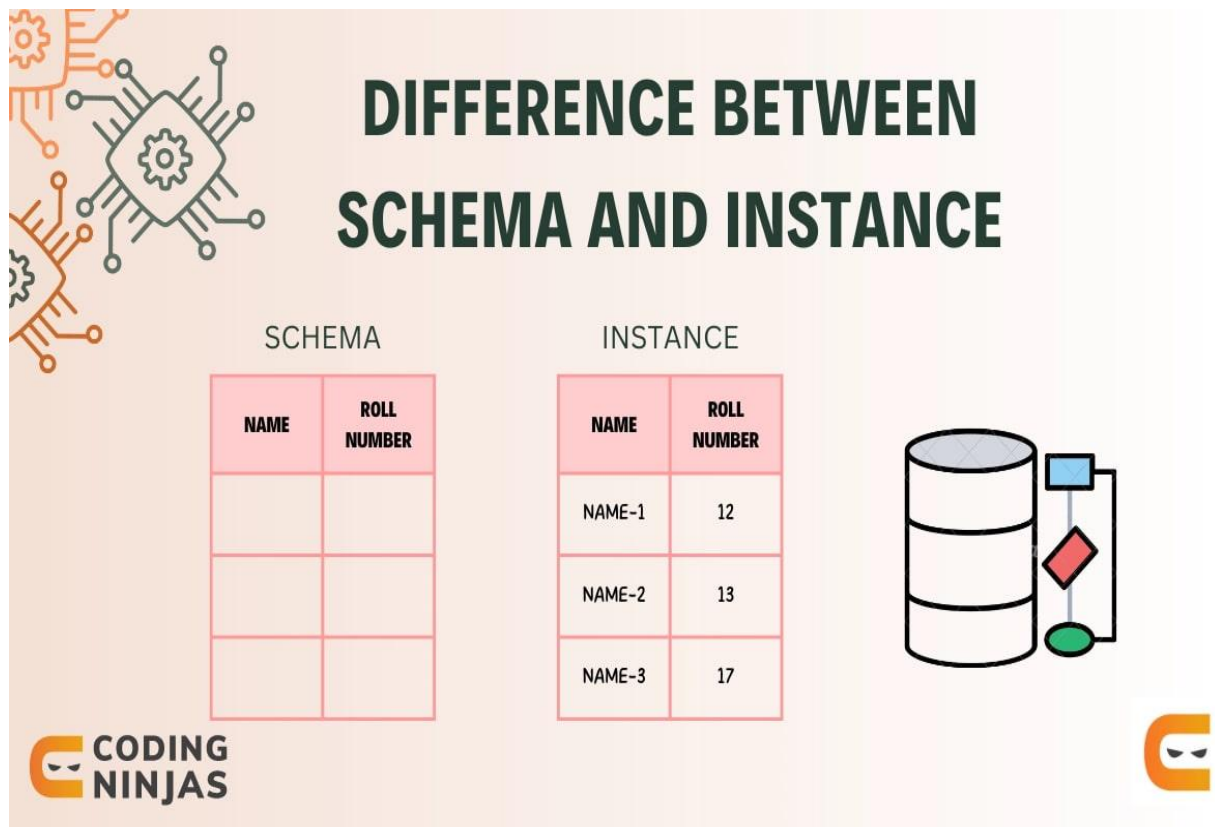
- The **data model** is the overall concept or methodology used to design the database.
- The **schema** is the actual design or structure created based on the data model.
- The **instance** is the data stored inside the database according to the schema, at a given point in time.

Why Is This Important?

Understanding these concepts is important because:

- The **data model** ensures you design a database that fits your needs.
- The **schema** guides the actual building of your database.

- The **instance** is what you work with when you add, update, or read data from the database.



▪ DATA INDEPENDENCE

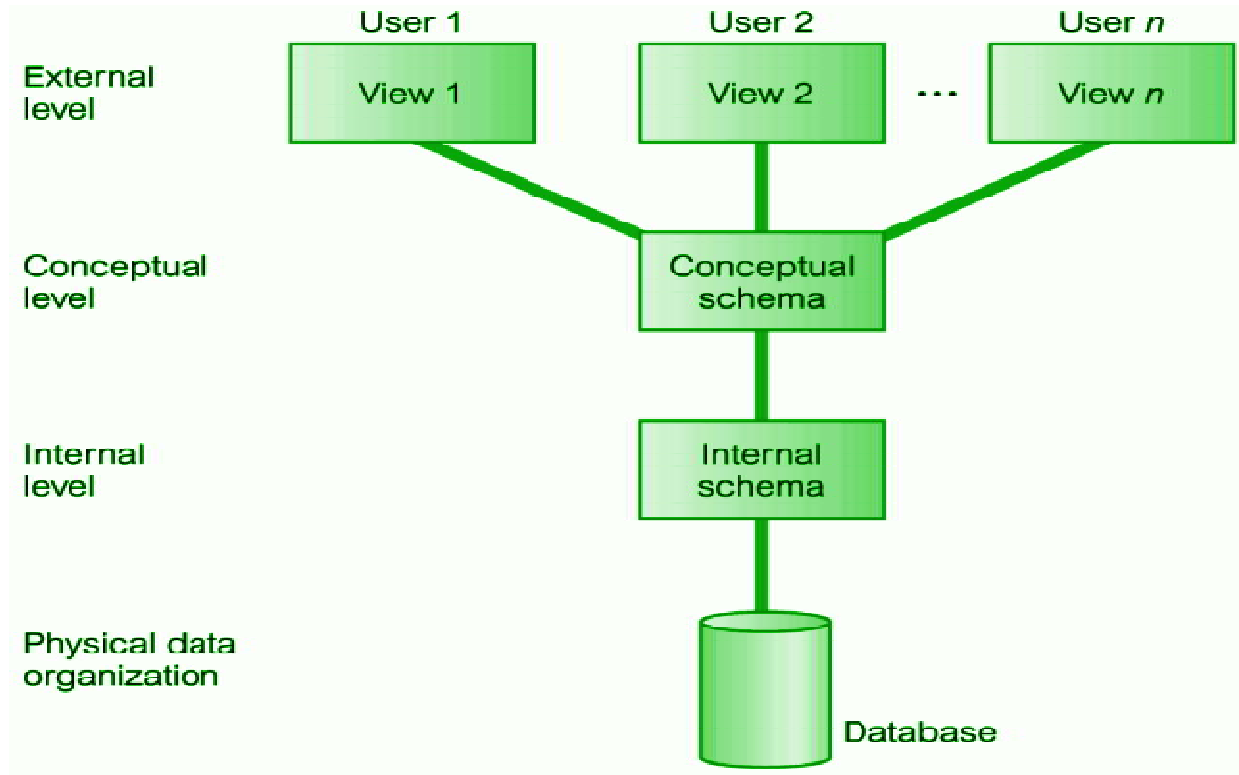
Data Independence means that changes in the way data is stored or organized do **not affect** how users access or use the data.

There are two types:

- **Physical Data Independence:** If the way data is stored on the computer (like changing from one file format to another) changes, users don't need to change their programs or queries.
- **Example:** You change how student data is stored on disk, but the reports you create still work fine without changes.
- **Logical Data Independence:** If you change the logical structure of the database (like adding new columns or tables), user programs don't need to change.

Example: You add a new column "Email" to the student table, but the old programs that read student names and IDs still work as before.

Why is it important? It helps keep the system flexible and easier to maintain.



- **ARCHITECTURE OF DBMS IN TERMS OF ITS COMPONENTS**

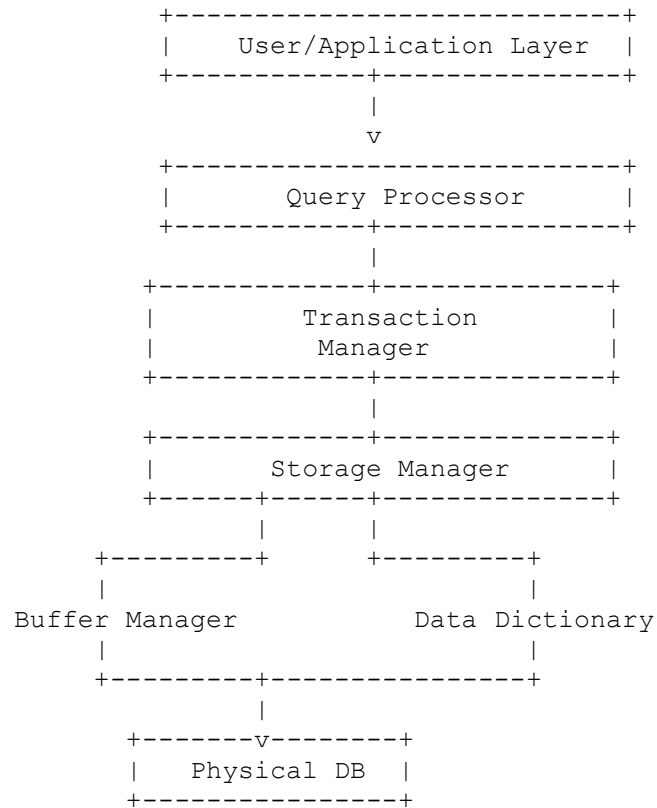
Definition:

The **architecture of a DBMS (Database Management System)** refers to the structure that describes how users interact with the system and how data is managed, stored, and retrieved. It includes various **components**, each responsible for a specific function.

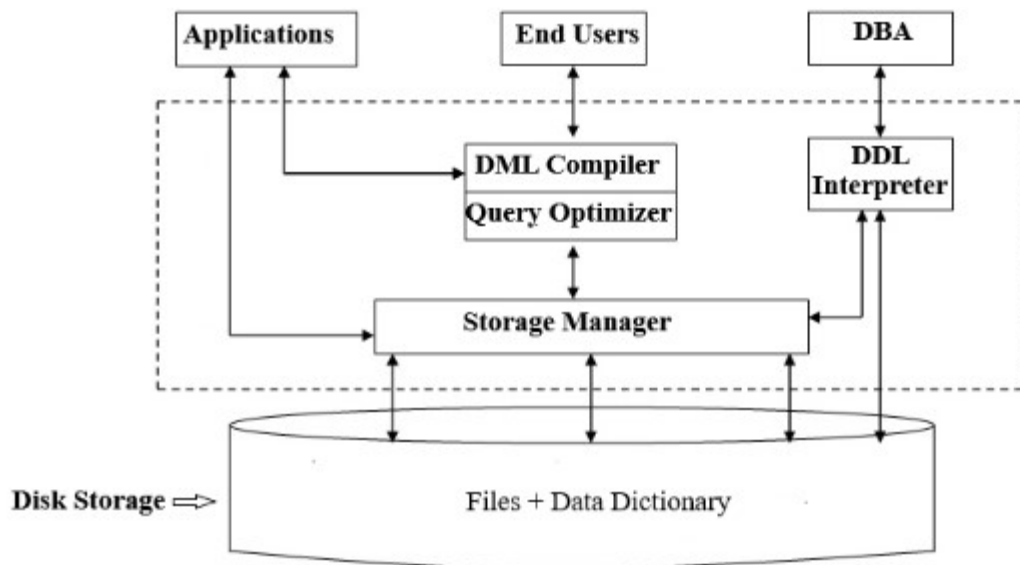
Major Components of DBMS Architecture:

1. **Query Processor**
2. **Storage Manager**
3. **Transaction Manager**
4. **Buffer Manager**
5. **Database Engine**
6. **Data Dictionary (Catalog)**
7. **User Interface / Application**

Diagram: DBMS Architecture



OR



Components Explained with Examples

1. Query Processor

- **Role:** Interprets and converts SQL queries into low-level instructions.
- **Includes:** Parser, Query Optimizer, Query Executor
- **Example:**

```
SELECT * FROM Students WHERE Class = '10th';
```

- The Query Processor parses this query, checks syntax, optimizes the execution path, and sends it for execution.

2. Transaction Manager

- **Role:** Manages transactions using ACID properties (Atomicity, Consistency, Isolation, Durability)
- **Ensures:** No conflict between simultaneous transactions
- **Example:**
If two users try to book the same train seat, the transaction manager ensures only one succeeds.

3. Storage Manager

- **Role:** Manages data storage on disk. Handles how data is stored and retrieved.
- **Includes:** File Manager, Authorization Manager
- **Example:**
Decides whether the `Student` table is stored in row-wise or column-wise format on disk.

4. Buffer Manager

- **Role:** Caches frequently used data in main memory to speed up processing
- **Example:**
When querying the `Students` table multiple times, it keeps data in memory to avoid disk access repeatedly.

5. Database Engine

- **Role:** Core component that performs read/write operations between queries and storage
- **Example:**
Reads rows from the `Employee` table when `SELECT` is executed and sends data to user.

6. Data Dictionary (Catalog)

- **Role:** Stores metadata (structure of tables, indexes, users, constraints, etc.)
- **Example:**
 - Student table → columns: RollNo (int), Name (varchar), Class (varchar)
 - This structure is stored in the data dictionary.

7. User Interface / Application Layer

- **Role:** Allows end-users or applications to interact with the DBMS
- **Example:**
GUI-based apps, web apps, or command-line tools (like MySQL Workbench or phpMyAdmin)

Example Scenario:

Imagine a school database system where a user submits the following query:

```
SELECT Name FROM Students WHERE Class = '10th';
```

- **Query Processor** checks and optimizes it.
- **Transaction Manager** ensures no interference with other queries.
- **Storage Manager** fetches the correct rows from disk.
- **Buffer Manager** keeps rows in memory for future queries.
- **Database Engine** performs the actual fetch.
- **Data Dictionary** provides table structure info.
- **User Interface** displays results to the user.

Summary

Component	Function	Example
Query Processor	Parses and optimizes queries	Interprets <code>SELECT</code> statement
Transaction Manager	Maintains ACID properties	Controls simultaneous access to a row
Storage Manager	Manages storage and retrieval	Organizes how data is stored
Buffer Manager	In-memory data handling	Speeds up repeated data access
Database Engine	Executes low-level operations	Reads rows from disk
Data Dictionary	Stores schema and metadata	Keeps info about tables and columns

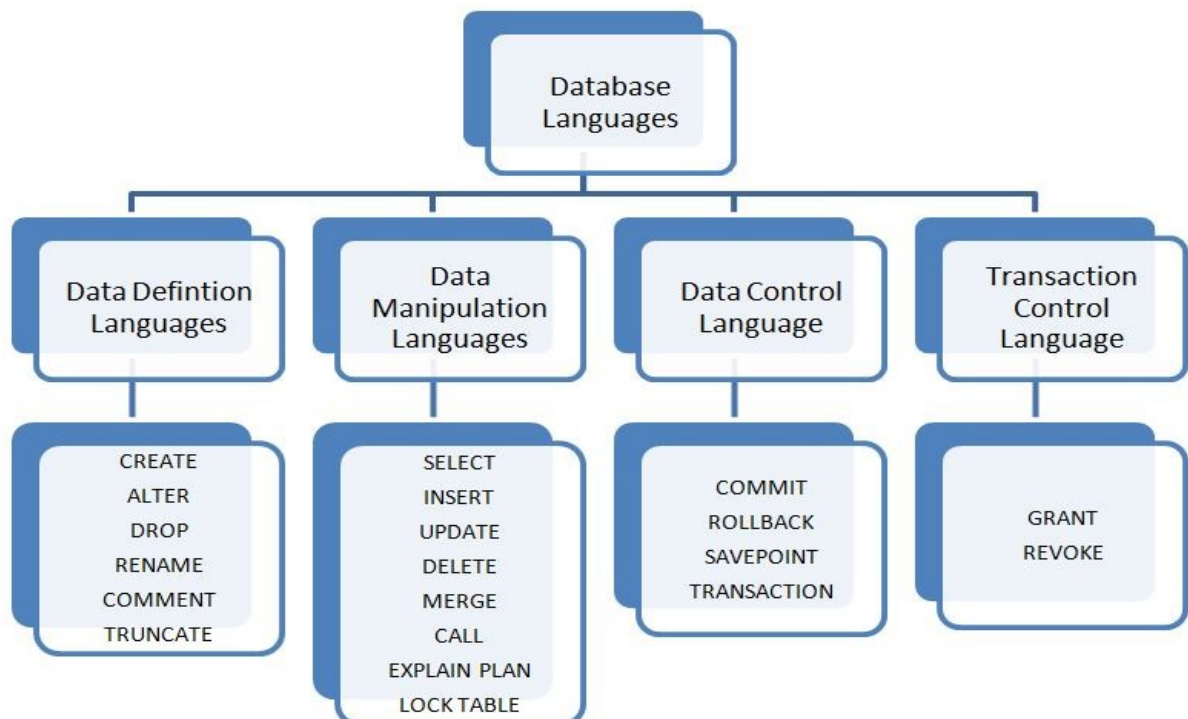
Component	Function	Example
User Interface	Front-end for users	Forms, CLI, SQL editor apps

- **DATABASE LANGUAGE AND INTERFACES**

To work with a database, we use special languages and interfaces.

a) DATABASE LANGUAGES

- **Data Definition Language (DDL):** Used to define the structure or schema of the database. It includes commands to create, modify, or delete tables and database structures.
Example commands: CREATE TABLE, ALTER TABLE, DROP TABLE.
- **Data Manipulation Language (DML):** Used to **insert, update, delete, and retrieve data** in the database. *Example commands:* INSERT, UPDATE, DELETE, SELECT.
- **Data Control Language (DCL):** Used to control access to the data (like giving permission to users).
Example commands: GRANT, REVOKE.



IN DETAILS

1. DATA DEFINITION LANGUAGE (DDL)

What is DDL?

DDL is a set of commands used to **define and manage the structure** of the database itself. This means you use DDL to create, change, or remove database objects such as tables, indexes, and schemas.

Main commands in DDL:

CREATE:

Used to **create** new database objects (like tables, indexes, or views).

Example:

```
CREATE TABLE Student (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

This command creates a table called "Student" with three columns.

ALTER: Used to **modify** the structure of an existing database object.

Example:

```
ALTER TABLE Student ADD Email VARCHAR(100);
```

This command adds a new column called "Email" to the Student table.

DROP:

Used to **delete** a database object permanently.

Example:

```
DROP TABLE Student;
```

This command deletes the entire Student table and all its data.

TRUNCATE:

Deletes all the data inside a table but keeps the table structure for future use.

2. DATA MANIPULATION LANGUAGE (DML)

What is DML?

DML commands are used to **work with the data inside the database tables**. This means adding new data, changing existing data, removing data, or retrieving data.

Main commands in DML:

INSERT:

Adds new rows (records) into a table.

Example:

```
INSERT INTO Student (StudentID, Name, Age) VALUES (101, 'Alice', 12);
```

This adds a new student record.

UPDATE:

Changes existing data in the table.

Example:

```
UPDATE Student SET Age = 13 WHERE StudentID = 101;
```

This updates Alice's age to 13.

DELETE:

Removes rows from a table.

Example:

```
DELETE FROM Student WHERE StudentID = 101;
```

This deletes the record for the student with ID 101.

SELECT:

Retrieves data from one or more tables.

Example:

```
SELECT * FROM Student;
```

This fetches all records from the Student table.

3. DATA CONTROL LANGUAGE (DCL)

What is DCL?

DCL commands are used to **control access to the database**. These commands help database administrators give or take away permissions from users to protect data security.

Main commands in DCL:**GRANT:**

Gives a user permission to perform specific actions (like reading or writing data).

Example:

```
GRANT SELECT, INSERT ON Student TO user1;
```

This allows user1 to read and add data to the Student table.

REVOKE:

Removes permissions previously granted to a user.

Example:

```
REVOKE INSERT ON Student FROM user1;
```

This removes user1's permission to add new data to the Student table.

Why Are These Important?

- **DDL** defines the database's **structure** and controls how the database is built.
- **DML** lets users **interact with the data** stored in the database.

- **DCL** helps **keep data safe** by managing who can do what.

Together, these languages form the core way people create, use, and secure databases.

If you want, I can provide sample scenarios or exercises to practice these commands!

b) Database Interfaces

These are the ways users or programs interact with the database:

- **Command-line Interface:** You type commands directly (like SQL queries).
- **Graphical User Interface (GUI):** You use buttons and forms (like in many apps) that send commands to the database behind the scenes.
- **Application Programming Interface (API):** Programs use APIs (like JDBC, ODBC) to communicate with the database.

3. DATA DEFINITION LANGUAGE (DDL)

DDL is used to **define or change the database structure**. **DDL** stands for **Data Definition Language**. It is a set of commands in a database system used specifically to **define, create, modify, and delete the structure** of database objects like tables, indexes, views, and schemas.

Common DDL Commands

1. CREATE

The **CREATE** command is used to **make new database objects** like tables or indexes.

- **Create a Table:** You tell the database the table name and the columns it should have, along with the data types for each column.

Example:

```
CREATE TABLE Employee (
    EmployeeID INT,
    Name VARCHAR(50),
    Salary DECIMAL(10,2),
    JoiningDate DATE
);
```

This creates a table called "Employee" with four columns: an integer ID, a name, a salary, and a joining date.

- **Create an Index:** An index helps speed up searching inside a table.

Example:

```
CREATE INDEX idx_name ON Employee(Name);
```

2. ALTER

The **ALTER** command is used to **change the structure of an existing table** without deleting it. You can:

- Add new columns,

- Modify the type or size of columns,
- Delete columns,
- Rename columns or tables.

Examples:

- **Add a new column:**
ALTER TABLE Employee ADD Email VARCHAR(100);
This adds an Email column to the Employee table.
- **Modify a column's data type:**
ALTER TABLE Employee MODIFY Salary DECIMAL(12,2);
This changes the Salary column to hold larger numbers.
- **Drop (remove) a column:**
ALTER TABLE Employee DROP COLUMN Email;
This deletes the Email column.

3. DROP

The **DROP** command is used to **completely delete database objects** like tables or indexes.

- **Drop a Table:**
DROP TABLE Employee;
This removes the entire Employee table and all its data.
- **Drop an Index:**
DROP INDEX idx_name ON Employee;

4. TRUNCATE

The **TRUNCATE** command **removes all data inside a table but keeps the structure** so you can reuse it.

Example:

TRUNCATE TABLE Employee;

This deletes all rows from the Employee table but keeps the table itself for future use.

What Happens When You Use DDL?

- When you run a DDL command like **CREATE TABLE**, the database updates its **data dictionary or system catalog** to remember the new structure.
- DDL commands typically **execute immediately and permanently** (auto-commit), meaning changes are saved right away.
- You use DDL mostly during the **database design and setup phase**, or when you need to **change the structure** later.

Functions of DDL Commands

1. CREATE

- **Function:**
To create new database objects such as tables, indexes, views, or schemas.
- **What it does:** Defines the structure of the new object including columns, data types, and constraints.
- **Example:**
Creating a new table with columns for student details.

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

2. ALTER

- **Function:**
To modify the structure of an existing database object.
- **What it does:** Adds or deletes columns, changes data types, renames tables or columns, or modifies constraints.
- **Example:**
Adding an Email column to the Student table.

```
ALTER TABLE Student ADD Email VARCHAR(100);
```

3. DROP

- **Function:**
To delete an existing database object permanently.
- **What it does:** Removes the table or object and all data stored within it.
- **Example:**
Removing the Student table entirely.

```
DROP TABLE Student;
```

4. TRUNCATE

- **Function:**
To delete **all data** in a table but keep the table structure intact.
- **What it does:** Quickly removes all rows without logging individual row deletions, which is faster than DELETE.
- **Example:**
Removing all records from the Student table but keeping the table ready for new data.

```
TRUNCATE TABLE Student;
```

Summary Table

DDL Command	Function	What It Does
CREATE	Create new database objects	Defines new tables, indexes, or other structures
ALTER	Modify existing database structures	Adds/removes columns or changes table structure
DROP	Delete existing database objects	Permanently removes tables or other objects
TRUNCATE	Delete all data from a table	Removes all rows but keeps the table itself

4. DATA MANIPULATION LANGUAGE (DML)

DML is used to **work with the data inside the tables**. **DML commands are used to manage the actual data stored inside the database tables**. These commands allow you to **insert, update, delete, and retrieve** data.

Examples of DML commands:

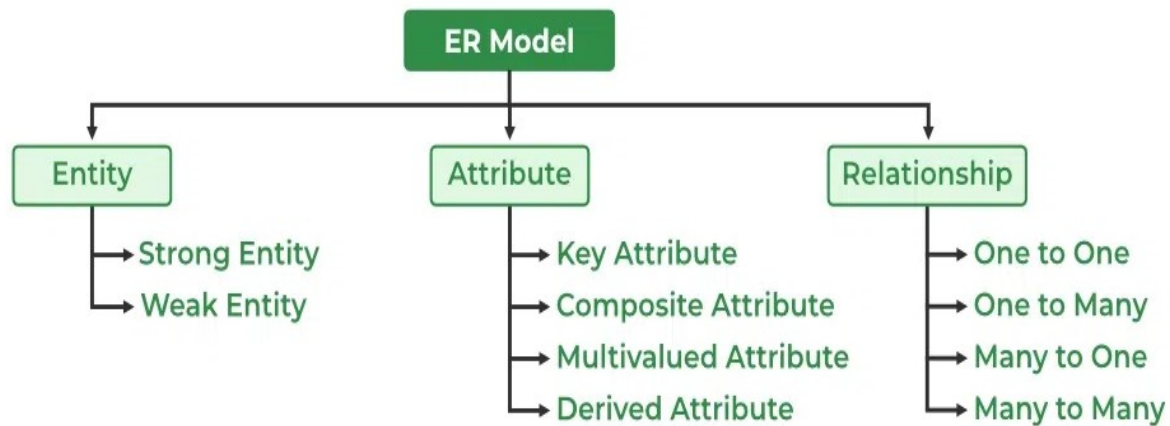
- **INSERT INTO Student VALUES (101, 'Alice', 12);** Adds a new student record.
- **UPDATE Student SET Age = 13 WHERE StudentID = 101;** Changes Alice's age to 13.
- **DELETE FROM Student WHERE StudentID = 101;** Deletes Alice's record.
- **SELECT * FROM Student;** Retrieves all student records.

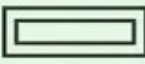
WHAT IS DATA MODELING?

Data modeling is the process of **creating a visual representation of the data** and how it relates in a database system. It helps in designing the database structure before actual implementation.

1. ER MODEL CONCEPTS

The **Entity-Relationship (ER) Model** is a popular way to represent data visually using entities, attributes, and relationships.



Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Key Components:

a) Entity

- Represents a **real-world object** or thing that can be distinguished from others.
- Can be a person, place, object, event, or concept.
- Each entity has **attributes** (properties) describing it.

Example:

Student, Course, Employee, Car.

b) Entity Set

- A collection of similar entities.
- Example: All students form the Student entity set.

c) Attribute

- A property or characteristic of an entity.
- Examples: Student's Name, Age, ID.

d) Key Attribute

- An attribute that **uniquely identifies** each entity in an entity set.
- Example: Student-ID uniquely identifies each student.

e) Relationship

- Represents an **association between two or more entities**.
- Example: A student **enrolls** in a course.

f) Relationship Set

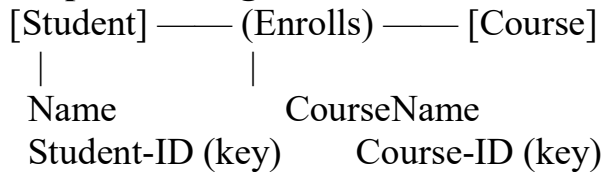
- A collection of similar relationships.
- Example: All enrollments form the Enrolls relationship set.

2. Notation for ER Diagrams

ER Diagrams are graphical representations using standard symbols:

Concept	Notation	Description
Entity	Rectangle	Named with a singular noun (e.g., Student)
Attribute	Oval (ellipse)	Connected to entity or relationship
Key Attribute	Oval with underline	Uniquely identifies entities
Relationship	Diamond	Connects entities to show association
Entity Set	Collection of entities (represented by rectangle)	Group of entities
Relationship Set	Collection of relationships (represented by diamond)	Group of relationships
Lines	Connect entities, attributes, and relationships	Show association or possession

Example ER Diagram Elements:



3. Mapping Constraints

Mapping constraints describe **how many entities participate in a relationship**.

a) Cardinality Ratios

Indicate the number of entities on each side of a relationship:

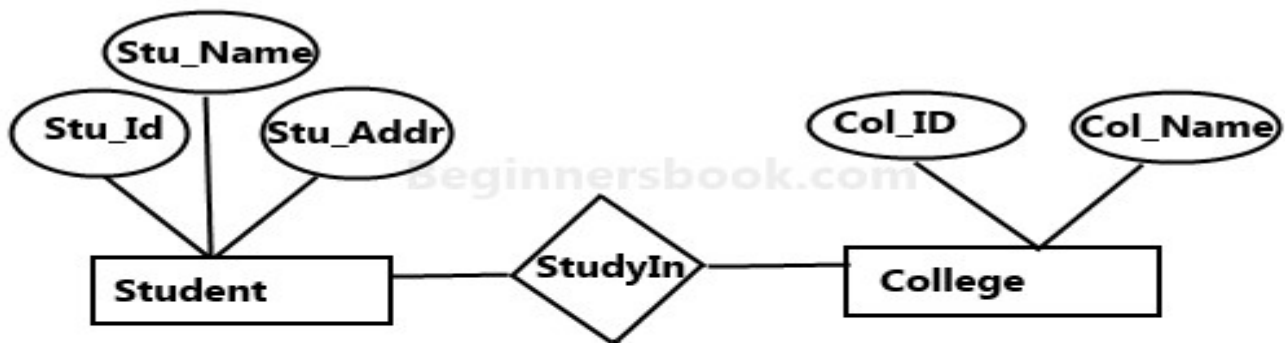
- **One-to-One (1:1):** One entity from A relates to one entity from B.
Example: One person has one passport.
- **One-to-Many (1:N):** One entity from A relates to many entities from B, but each entity from B relates to only one entity from A.
Example: One teacher teaches many students, but each student has one main teacher.
- **Many-to-One (N:1):** Many entities from A relate to one entity from B.
- **Many-to-Many (M:N):** Many entities from A relate to many entities from B.
Example: Students enroll in many courses, and courses have many students.

b) Participation Constraints

Describe whether entity participation in a relationship is optional or mandatory.

- **Total Participation:** Every entity in the set must participate in the relationship.
Represented by a **double line**. *Example:* Every student must enroll in at least one course.
- **Partial Participation:** Some entities may participate; others may not.
Represented by a **single line**. *Example:* Some employees may manage projects, others may not.

Example-1 E-R Diagram



Sample E-R Diagram

KEYS IN DATABASE

In databases, **keys** are used to uniquely identify records (rows) in a table. They help the database know which record is which and avoid confusion.

1. SUPER KEY

What is a Super Key?

- A **Super Key** is a set of one or more columns (attributes) in a table that **uniquely identifies each record**.
- It can be a single column or a combination of columns.
- **Every table has at least one Super Key.**
- A Super Key might have **extra attributes** that are not necessary for unique identification.

Example:

Consider a table **Student** with these columns:

StudentID	Name	Email	PhoneNumber
101	Alice	alice@example.com	123-456-7890
102	Bob	bob@example.com	987-654-3210

- Possible Super Keys:
 - {Student-ID} (because Student-ID alone is unique)
 - {Email} (assuming emails are unique)
 - {StudentID, Email} (also unique, but has an extra attribute, Email, which isn't necessary)
 - {StudentID, Name} (still unique, but Name is unnecessary here)

2. CANDIDATE KEY

What is a Candidate Key?

- A **Candidate Key** is a **minimal Super Key** — meaning it has **no unnecessary attributes**.
- It uniquely identifies records with the **smallest possible set of columns**.
- There can be **one or more Candidate Keys** in a table.
- From these Candidate Keys, one is selected to be the Primary Key.

Example:

From the Super Keys above:

- {Student-ID} is a Candidate Key (minimal and unique)
- {Email} can also be a Candidate Key if emails are unique
- {Student-ID, Email} is **not** a Candidate Key because it's not minimal (Email is unnecessary)

3. PRIMARY KEY

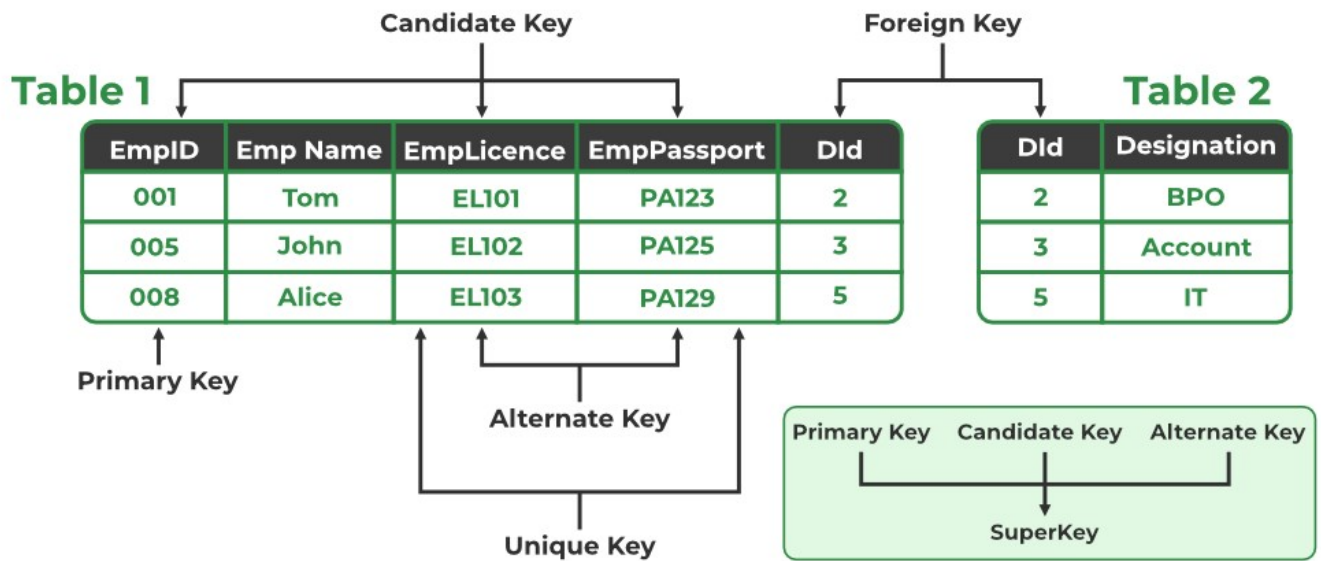
What is a Primary Key?

- The **Primary Key** is the **main Candidate Key chosen by the database designer** to uniquely identify records.
- It must be **unique** and **not null** (cannot have empty or duplicate values).
- Only **one Primary Key** can exist in a table.
- It is used to establish relationships with other tables (via foreign keys).

Example:

In the Student table:

- Suppose Student-ID is chosen as the Primary Key because it uniquely identifies students and is simple.
- Email might be unique, but it might change, so Student-ID is better.



Summary Table

Key Type	Description	Example in Student Table
Super Key	Any set of attributes that uniquely identify records	{StudentID}, {Email}, {StudentID, Email}
Candidate Key	Minimal Super Key (no unnecessary attributes)	{StudentID}, {Email}
Primary Key	Selected Candidate Key used to identify records	{StudentID}

Why Are Keys Important?

- **Prevent duplicate records:** Ensure each record is unique.
- **Enable fast searching:** Databases can quickly locate a record using keys.
- **Maintain data integrity:** Helps link tables correctly and keep data consistent.
- **Enforce uniqueness:** Prevents errors like two students having the same ID.

Simple Analogy

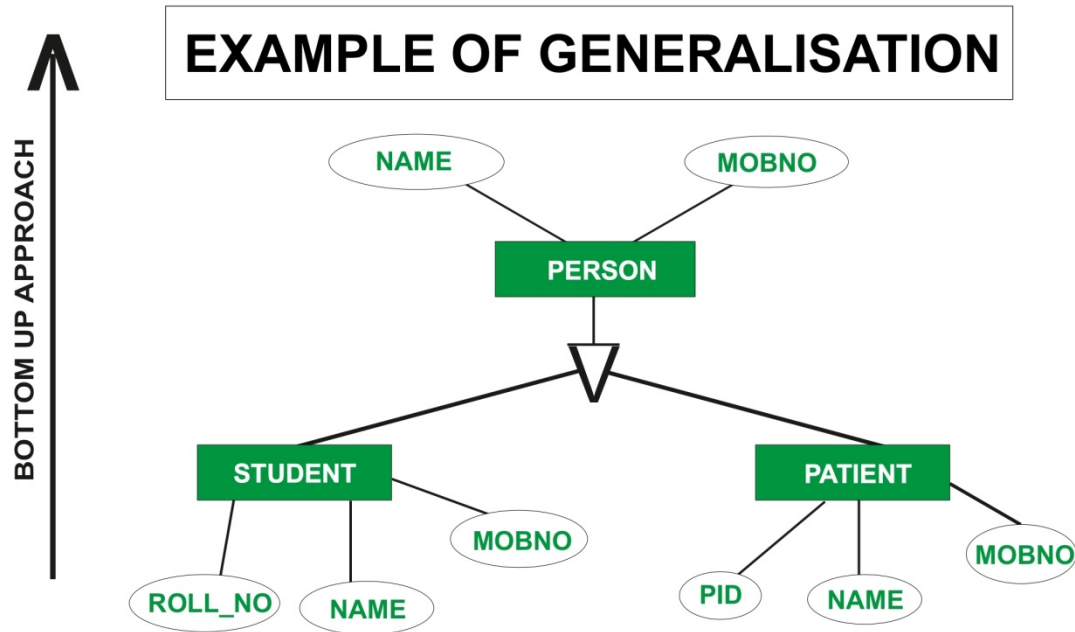
Think of a school where every student has:

- A **Student ID** card (Primary Key) — unique and mandatory.
- An **Email address** (Candidate Key) — unique but can change.
- Other info like name and phone number, which might not be unique.

The Student ID is the best way to identify a student uniquely and permanently.

GENERALIZATION

- Generalization means **grouping similar things together** into a bigger, general group.
- It is like saying: “Cars” and “Trucks” are both types of “Vehicles.”
- So instead of repeating the same information for Cars and Trucks, you create one general group called Vehicle.
- This helps keep things organized and avoids repetition.

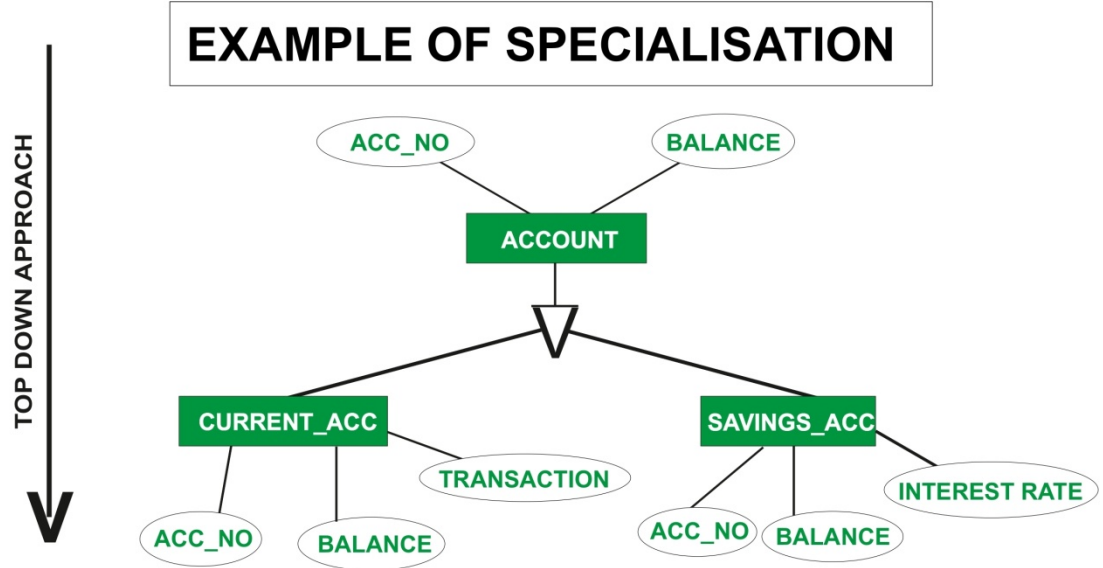


Example:

- Car and Truck both have “Make” and “Model.”
- So, you say: Vehicle (general group) has “Make” and “Model.”
- Car and Truck belong to Vehicle but also have their own special features.

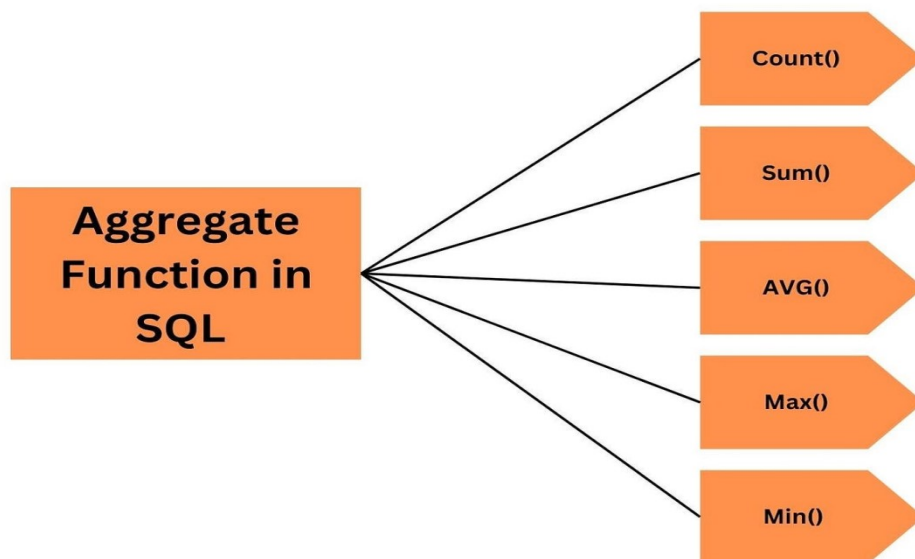
SPECIALIZATION

- **Specialization** means taking a **broad or general entity** and dividing it into **smaller, more specific entities** based on certain differences. For example, imagine you have an entity called "Employee." Not all employees do the same job — some are "Managers" while others are "Clerks" or "Technicians." So, we can use specialization to show that an Employee can be a Manager or a Technician, each with different attributes or roles. Specialization allows us to add new attributes that apply only to the specialized group.



AGGREGATION

- Aggregation means **treating a relationship like it is an object (thing)** so you can connect it to something else.
- Sometimes, you want to talk about how things are related, and that relationship itself has important details.
- So, you make that relationship into a thing, and then connect it to other things.



Example:

- A teacher teaches a course.
 - A student takes the course.
 - Now, you want to say: “Student got a grade in the course taught by the teacher.”
 - To say this clearly, you treat the “teacher teaches course” relationship as one thing.
 - Then you connect the student and the grade to that thing.
- **EXPLANATION OF HOW TO REDUCE AN ER DIAGRAM TO TABLES (CONVERT ER MODEL INTO A RELATIONAL DATABASE SCHEMA) IN BASIC LANGUAGE:**

REDUCTION OF ER DIAGRAMS TO TABLES

When we design a database using an ER Diagram, the next step is to **convert that design into tables** because databases store data in tables.

STEPS TO CONVERT ER DIAGRAMS TO TABLES**1. Convert Entity Sets to Tables**

- For each **entity** in the ER diagram, create a **table**.
- Each **attribute** of the entity becomes a **column** in the table.
- Choose the **primary key** (unique identifier) as the primary key of the table.

Example:

Entity: Student , Attributes: StudentID (key), Name, Age Table:

Student | StudentID (PK) | Name | Age |

2. Convert Relationship Sets to Tables

- For **one-to-one (1:1)** and **one-to-many (1:N)** relationships:
 - Add the **primary key** of the “one” side as a **foreign key** in the “many” side table.
- For **many-to-many (M:N)** relationships:
 - Create a **new table** for the relationship.
 - This table includes the primary keys of the related entities as foreign keys.
 - These foreign keys together form the **primary key** of the relationship table.
 - Add any attributes of the relationship as columns here.

Example (1:N):

Relationship: A teacher teaches many students

- Teacher table: TeacherID (PK), Name
- Student table: StudentID (PK), Name, TeacherID (FK)

Example (M:N):

- Relationship: Students enroll in many courses
- Student table: StudentID (PK), Name

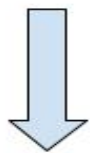
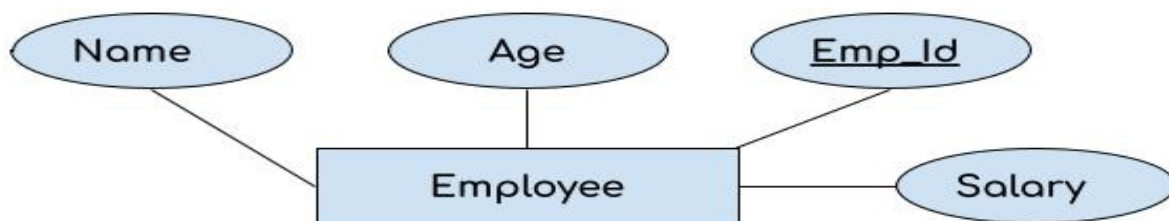
- Course table: CourseID (PK), Title
- Enrollment table (new): StudentID (FK), CourseID (FK), EnrollmentDate
- Primary Key of Enrollment is combination of StudentID + CourseID

3. Convert Weak Entities to Tables

- For weak entities (entities without a key of their own), create a table.
- Include their own attributes and the **primary key of the owner entity** as a foreign key.
- Together, these keys form the primary key of the weak entity table.

Summary Table

ER Concept	How to Convert to Tables
Entity	Create a table with attributes as columns
Relationship (1:1, 1:N)	Add foreign key in the “many” side table
Relationship (M:N)	Create a new table with foreign keys from both entities
Weak Entity	Create a table with foreign key from owner + its own attributes



<u>Emp_Id</u>	Name	Age	Salary
Beginnersbook.com			

Table Schema: (Emp_id, Name, Age, Salary)

EXTENDED ER MODEL (EER MODEL)

The **Extended Entity-Relationship (EER) Model** is an improved version of the basic ER model that we use to design databases. While the regular ER model allows us to show entities, their attributes, and their relationships, it may not be enough to describe more **complex and real-world** situations. That's why we use the EER model, which gives us **extra features** to represent data more clearly and completely.

In real life, entities can be related in different ways, or one entity can be a part of another. Sometimes, we need to show that one entity is a **type or version of another** entity. For example, if we have an entity called "Employee," then "Manager" and "Technician" could be two types of employees. This kind of classification is easier to model using the EER model.

Category (Union Type)

Sometimes, we need to create a special kind of entity that is a **subclass of more than one superclass**. This is called a **category** or **union type**. It means one entity comes from two or more parent entities. For example, suppose we have two entities: "SavingAccount" and "CurrentAccount." Now, we want to create an entity called "AccountHolder" that could be related to either a saving account or a current account. The "AccountHolder" is a category (or union) of both types. This is useful when an entity needs to inherit features from more than one source.

RELATIONSHIPS OF HIGHER DEGREE

In most ER diagrams, relationships connect **two entities**. These are called **binary relationships**. But in the real world, some situations involve **three or more entities at the same time**. These are called **relationships of higher degree**.

For example, think about this situation: A **doctor** treats a **patient** in a **hospital**. This is not just a relationship between two entities. It includes **three entities at once**: Doctor, Patient, and Hospital. So, we cannot represent this situation using only two entities at a time. This kind of relationship is called a **ternary relationship**, because it involves **three entities**.

In general, if a relationship involves **more than two entities**, we call it a **higher-degree relationship**. These relationships are used when all entities involved are connected **at the same time**, and we need to capture that connection clearly.

Final Words

The Extended ER model helps database designers build better, more flexible, and more real-world database systems. It gives us tools to group or split entities, connect complex relationships, and create meaningful structures. Higher-degree relationships allow us to handle situations where many entities work together in one action or event. When we use these features properly, the database becomes easier to understand, more accurate, and more efficient to use.

UNIT-2 (RELATIONAL DATA MODEL AND LANGUAGE)

RELATIONAL MODEL IN DBMS

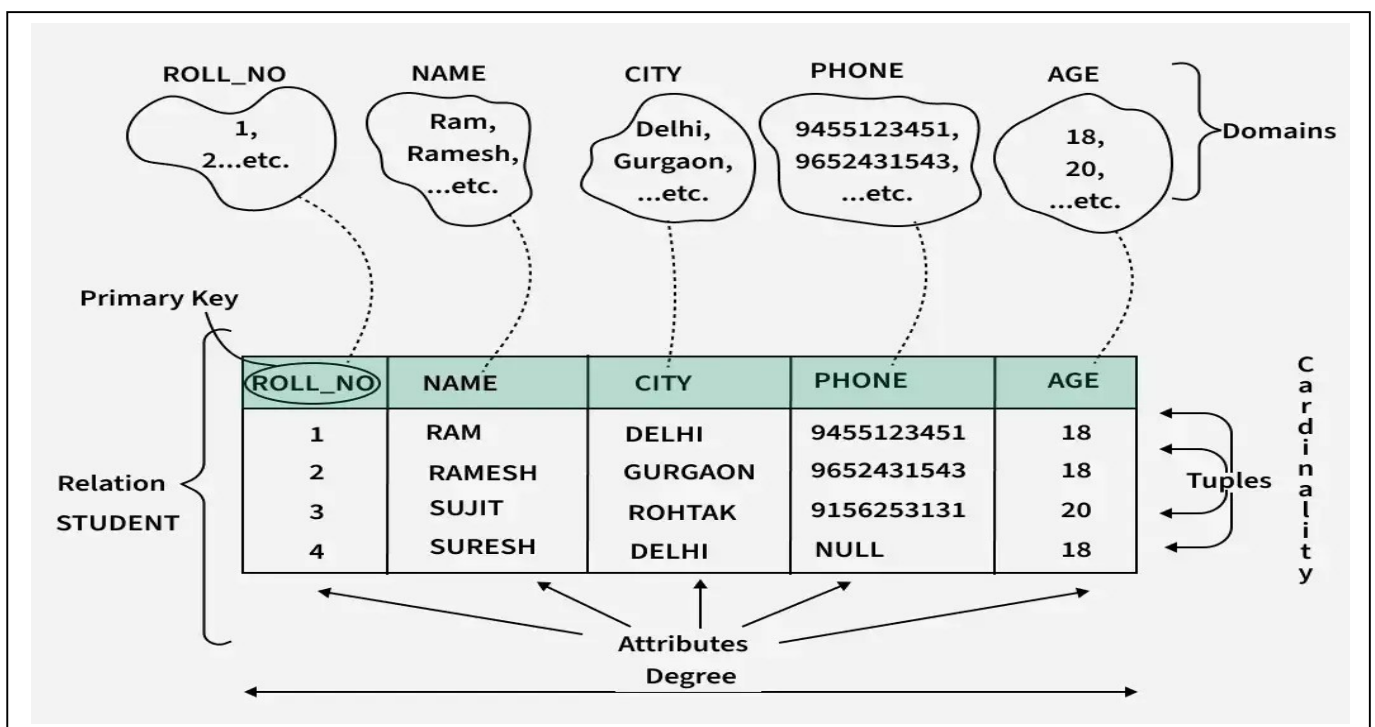
The Relational Model organizes data using tables (relations) consisting of rows and columns. Each column represents a specific attribute with a unique name, while each row holds data about a real-world entity or relationship. As a record-based model, it stores data in fixed-format records with defined attributes.

Introduced by E.F. Codd, the relational model transforms conceptual designs from ER diagrams into implementable structures. These structures are used in relational database systems like Oracle SQL and MySQL.

What is the Relational Model?

The relational model represents how data is stored and managed in Relational Databases. Data is organized into tables, each known as a relation, consisting of rows (tuples) and columns (attributes). Each row represents an entity or record, and each column represents a particular attribute of that entity. A relational database consists of a collection of tables each of which is assigned a unique name.

For example, consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE, and AGE shown in the table.



KEY TERMS IN THE RELATIONAL MODEL

1. Attribute: Attributes are the properties that define an entity.

Example: ROLL_NO, NAME, ADDRESS etc.

2. Relation Schema: A relation schema defines the structure of the relation and represents the name of the relation with its attributes.

Example: STUDENT (ROLL_NO, NAME, ADDRESS, PHONE, and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation it is called Relational Schema.

3. Tuple: A Tuple represents a row in a relation. Each tuple contains a set of attribute values that describe a particular entity.

Example: (1, RAM, DELHI, 9455123451, 18) is a tuple in the STUDENT table.

4. Relation Instance: The set of tuples of a relation at a particular instance of time is called a relation instance. It can change whenever there is an insertion, deletion or update in the database.

5. Degree: The number of attributes in the relation is known as the degree of the relation.

Example: The STUDENT relation has a degree of 5, as it has 5 attributes.

6. Cardinality: The number of tuples in a relation is known as cardinality.

Example: The STUDENT relation defined above has cardinality 4.

7. Column: The column represents the set of values for a particular attribute.

Example: The column ROLL_NO is extracted from the relation STUDENT.

8. NULL Values: The value which is not known or unavailable is called a NULL value. It is represented by NULL.

Example: PHONE of STUDENT having ROLL_NO 4 is NULL.

• WHAT IS DATA INTEGRITY?

Data integrity means keeping data **accurate**, **consistent**, and **reliable** over its entire life. It ensures that the data hasn't been changed accidentally or maliciously and that it stays correct and trustworthy.

Simple Example

Imagine a **student database** in a school.

Each student has:

- ID: 101
- Name: Alice

- Date of Birth: 2005-07-31

✓ With Data Integrity

- The ID is unique (no other student has ID 101).
- The name is stored correctly as "Alice".
- The date of birth is valid and in the right format.

✗ Without Data Integrity

- Two students have the same ID (101).
- The name is misspelled ("Allce").
- The date of birth is entered as "31/31/2005" (invalid date).

How to Maintain Data Integrity

- Use **primary keys** (e.g., unique student ID).
- Use **data validation** (e.g., date format checks).
- Use **constraints** (e.g., age must be > 5).
- Use **backups** to prevent loss or corruption.
- Control **access** (only authorized users can edit data).

• TYPES OF KEYS IN THE RELATIONAL MODEL

PRIMARY KEY

A Primary Key uniquely identifies each tuple in a relation. It must contain unique values and cannot have NULL values. Example: ROLL_NO in the STUDENT table is the primary key.

CANDIDATE KEY

A Candidate Key is a set of attributes that can uniquely identify a tuple in a relation. There can be multiple candidate keys, and one of them is chosen as the primary key.

SUPER KEY

A Super Key is a set of attributes that can uniquely identify a tuple. It may contain extra attributes that are not necessary for uniqueness.

FOREIGN KEY

A Foreign Key is an attribute in one relation that refers to the primary key of another relation. It establishes relationships between tables. Example: BRANCH_CODE in the STUDENT table is a foreign key that refers to the primary key BRANCH_CODE in the BRANCH table.

COMPOSITE KEY

A Composite Key is formed by combining two or more attributes to uniquely identify a tuple. Example: A combination of FIRST_NAME and LAST_NAME could be a composite key if no one in the database shares the same full name.

- **RELATIONAL MODEL NOTATION**

- Relation schema R of degree n is denoted by $R(A_1, A_2, \dots, A_n)$.
- Uppercase letters Q, R, S denote relation names.
- Lowercase letters q, r, s denote relation states.
- Letters t, u, v denote tuples.
- In general, the name of a relation schema such as STUDENT also indicates the current set of tuples in that relation.
- An attribute A can be qualified with the relation name R to which it belongs by using the dot notation R.A for example, STUDENT.Name or STUDENT.Age.
- An n-tuple t in a relation r(R) is represented as $t = \langle v_1, v_2, \dots, v_n \rangle$ where v_i is the value corresponding to the attribute A_i . The value v_i for attribute A_i in tuple t can be accessed using $t[A_i]$ or $t.A_i$.

CHARACTERISTICS OF THE RELATIONAL MODEL

1. **Data Representation:** Data is organized in tables (relations), with rows (tuples) representing records and columns (attributes) representing data fields.
2. **Atomic Values:** Each attribute in a table contains atomic values, meaning no multi-valued or nested data is allowed in a single cell.
3. **Unique Keys:** Every table has a primary key to uniquely identify each record, ensuring no duplicate rows.
4. **Attribute Domain:** Each attribute has a defined domain, specifying the valid data types and constraints for the values it can hold.
5. **Tuples as Rows:** Rows in a table, called tuples, represent individual records or instances of real-world entities or relationships.
6. **Relation Schema:** A table's structure is defined by its schema, which specifies the table name, attributes, and their domains.
7. **Data Independence:** The model ensures logical and physical data independence, allowing changes in the database schema without affecting the application layer.
8. **Integrity Constraints:** The model enforces rules like:
9. **Domain constraints:** Attribute values must match the specified domain.
10. **Entity integrity:** No primary key can have NULL values.
11. **Referential integrity:** Foreign keys must match primary keys in the referenced table or be NULL.
12. **Relational Operations:** Supports operations like selection, projection, join, union, and intersection, enabling powerful data retrieval manipulation.

13. **Data Consistency:** Ensures data consistency through constraints, reducing redundancy and anomalies.

14. **Set-Based Representation:** Tables in the relational model are treated as sets, and operations follow mathematical set theory principles.

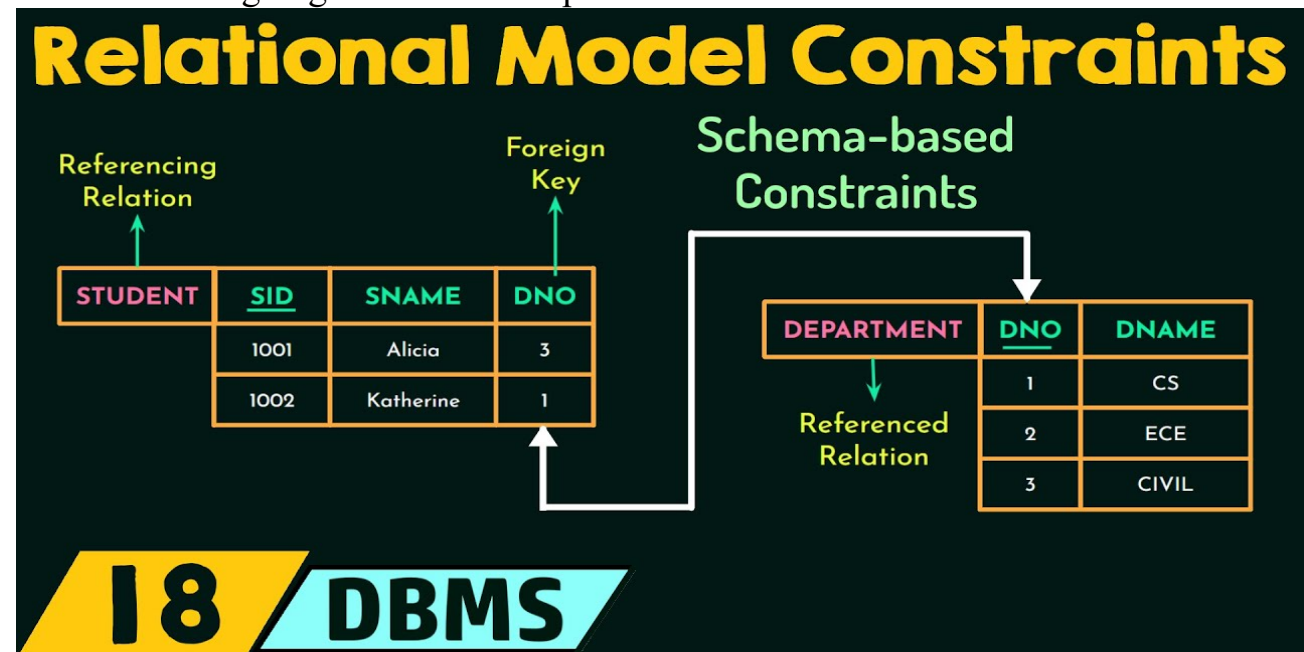
- **CONSTRAINTS ON RELATIONAL DATABASE MODEL**

In modeling the design of the relational database we can put some restrictions like what values are allowed to be inserted in the relation, and what kind of modifications and deletions are allowed in the relation. These are the restrictions we impose on the relational database.

In models like Entity-Relationship models, we did not have such features. Database Constraints can be categorized into 3 main categories:

1. Constraints that are applied in the data model are called Implicit Constraints.
2. Constraints that are directly applied in the schemas of the data model, by specifying them in the DDL(Data Definition Language). These are called Schema-Based Constraints or Explicit Constraints.
3. Constraints that cannot be directly applied in the schemas of the data model. We call these Application-based or Semantic Constraints.

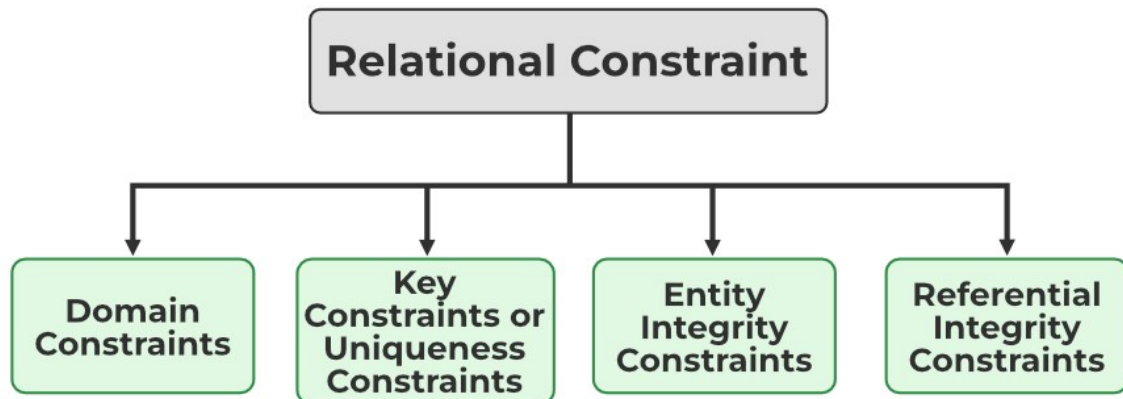
So here we are going to deal with Implicit constraints.



RELATIONAL CONSTRAINTS

These are the restrictions or sets of rules imposed on the database contents. It validates the quality of the database. It validates the various operations like data insertion, updation, and other processes that have to be performed without affecting the integrity of the data. It protects us against threats/damages to the database. Mainly Constraints on the relational database are of 4 types

- Domain constraints
- Key constraints or Uniqueness Constraints
- Entity Integrity constraints
- Referential integrity constraints



TYPES OF RELATIONAL CONSTRAINTS

Let's discuss each of the above constraints in detail.

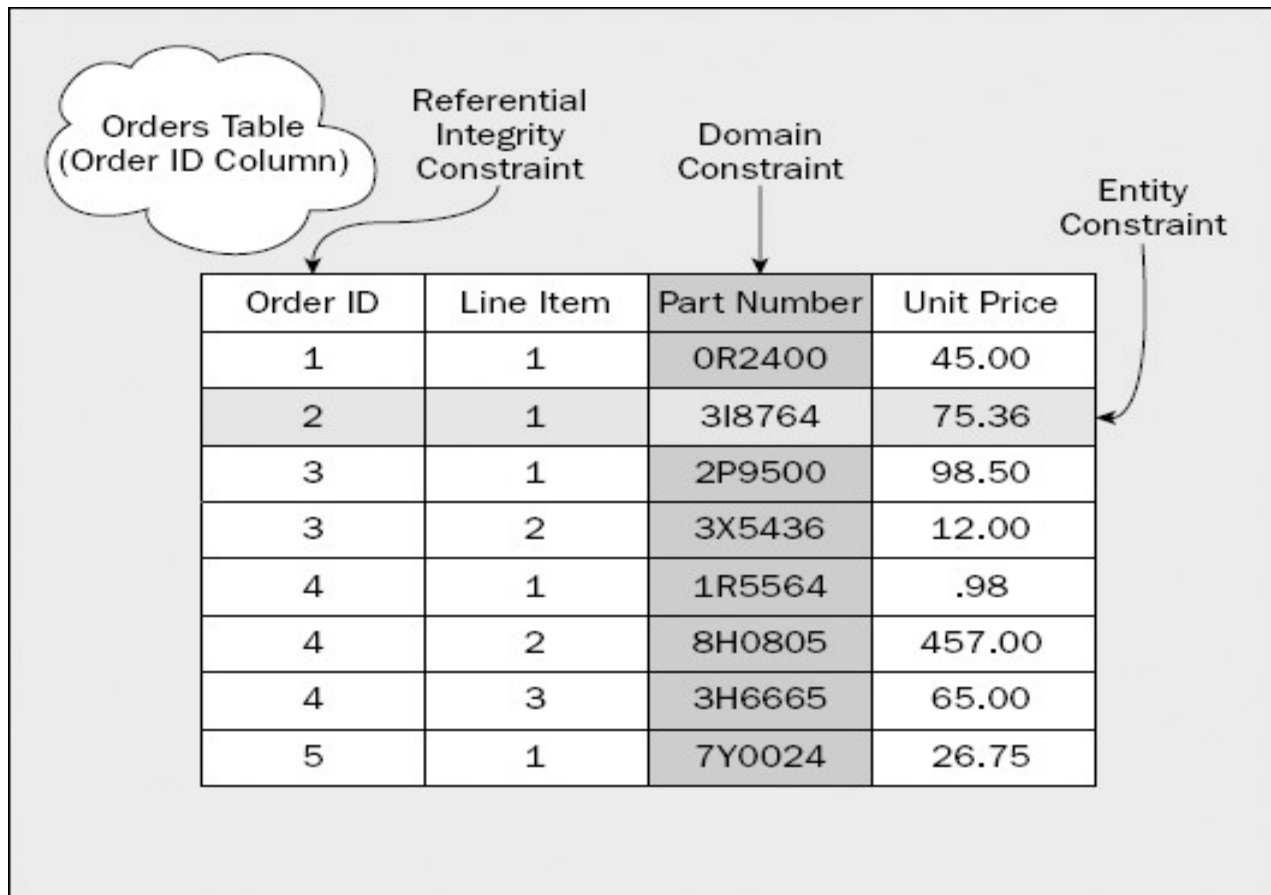
1. Domain Constraints

- Every domain must contain atomic values(smallest indivisible units) which means composite and multi-valued attributes are not allowed.
- We perform a data type check here, which means when we assign a data type to a column we limit the values that it can contain. Eg. If we assign the data type of attribute age as int, we can't give it values other than int data type.

Example:

EID	Name	Phone
01	Bikash Dutta	123456789 234456678

Explanation: In the above relation, Name is a composite attribute and Phone is a multi-values attribute, so it is violating domain constraint.



2. Key Constraints or Uniqueness Constraints

- These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
- A relation can have multiple keys or candidate keys(minimal superkey), out of which we choose one of the keys as the primary key, we don't have any restriction on choosing the primary key out of candidate keys, but it is suggested to go with the candidate key with less number of attributes.
- Null values are not allowed in the primary key, hence Not Null constraint is also part of the key constraint.

Example:

EID	Name	Phone
01	Bikash	6000000009
02	Paul	9000090009

EID	Name	Phone
01	Tuhin	9234567892

Explanation: In the above table, EID is the primary key, and the first and the last tuple have the same value in EID ie 01, so it is violating the key constraint.

3. Entity Integrity Constraints

- Entity Integrity constraints say that no primary key can take a NULL value, since using the primary key we identify each tuple uniquely in a relation.

Example:

EID	Name	Phone
01	Bikash	9000900099
02	Paul	600000009
NULL	Sony	9234567892

Explanation: In the above relation, EID is made the primary key, and the primary key can't take NULL values but in the third tuple, the primary key is null, so it is violating Entity Integrity constraints.

Student

Roll_no	Name	Class	Phone_no	Registration_no
1	Andrew	5	9854672256	895
2	Andrew	6	9955512456	564
3	Augusto	5		567

↑
Primary key

↑
Unique key

↑
Unique key

SCALED

4. Referential Integrity Constraints

When one attribute of a relation can only take values from another attribute of the same relation or any other relation, it is called referential integrity.

Let us suppose we have 2 relations

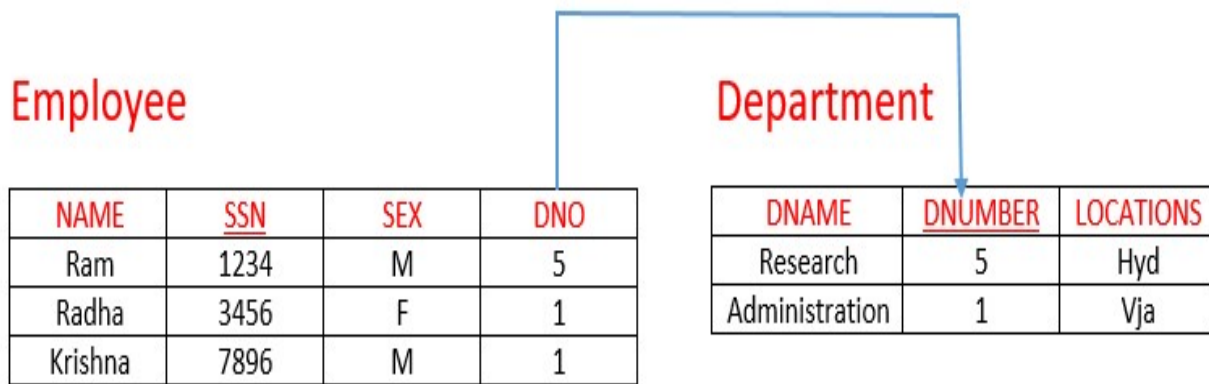
Table STUDENT

ROLL_NO	NAME	ADDRESS	PHONE	AGE	BRANCH_CODE
1	RAM	DELHI	9455123451	18	CS
2	RAMESH	GURGAON	9652431543	18	CS
3	SUJIT	ROHTAK	9156253131	20	ECE
4	SURESH	DELHI		18	IT

Table BRANCH

BRANCH_CODE	BRANCH_NAME
CS	COMPUTER SCIENCE
IT	INFORMATION TECHNOLOGY
ECE	ELECTRONICS AND COMMUNICATION ENGINEERING
CV	CIVIL ENGINEERING

Explanation: BRANCH_CODE of STUDENT can only take the values which are present in **BRANCH_CODE of BRANCH** which is called **referential integrity constraint**. The relation which is referencing another relation is called **REFERENCING RELATION** (STUDENT in this case) and the relation to which other relations refer is called **REFERENCED RELATION** (BRANCH in this case).



ADVANTAGES OF RELATIONAL DATABASE MODEL

- It is simpler than the hierarchical model and network model.
- It is easy and simple to understand.
- Its structure can be changed anytime upon requirement.
- **Data Integrity:** The relational database model enforces data integrity through various constraints such as primary keys, foreign keys, and unique constraints. This ensures that the data in the database is accurate, consistent, and valid.
- **Security:** The relational database model provides robust security features to protect the data in the database. These include user authentication, authorization, and encryption of sensitive data.
- **Data consistency:** The relational database model ensures that the data in the database is consistent across all tables. This means that if a change is made to one table, the corresponding changes will be made to all related tables.

DISADVANTAGES OF THE RELATIONAL MODEL

- Few database relations have certain limits which can't be expanded further.
- It can be complex and it becomes hard to use.
- **Complexity:** The relational model can be complex and difficult to understand, particularly for users who are not familiar with SQL and database design principles. This can make it challenging to set up and maintain a relational database.
- **Cost:** Relational databases can be expensive to license and maintain, particularly for large-scale deployments. Additionally, relational databases often require dedicated hardware and specialized software to run, which can add to the cost.
- **Data redundancy:** In some cases, the relational model can lead to data redundancy, where the same data is stored in multiple tables. This can lead to inefficiencies and can make it difficult to ensure data consistency across the database.

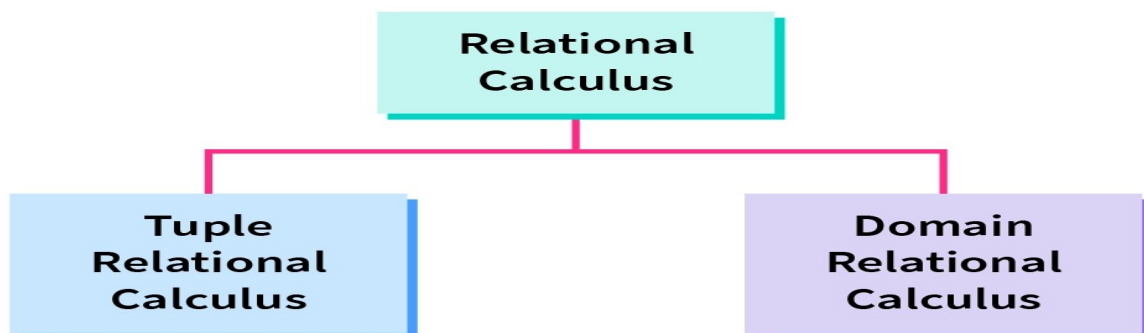
- **TUPLE RELATIONAL CALCULUS (TRC) IN DBMS**

What is Relational Calculus?

Before understanding Relational calculus in DBMS, we need to understand **Procedural Language** and **Declarative Language**.

1. **Procedural Language** - Those Languages which clearly define how to get the required results from the Database are called Procedural Language. **Relational algebra** is a Procedural Language.
2. **Declarative Language** - Those Language that only cares about What to get from the database without getting into how to get the results are called Declarative Language. **Relational Calculus** is a Declarative Language.

So Relational Calculus is a Declarative Language that uses Predicate Logic or First-Order Logic to determine the results from Database.



TYPES OF RELATIONAL CALCULUS IN DBMS

Relational Calculus is of Two Types:

1. Tuple Relational Calculus (TRC)
2. Domain Relational Calculus (DRC)

TUPLE RELATIONAL CALCULUS (TRC)

Tuple Relational Calculus in DBMS uses a tuple variable (t) that goes to each row of the table and checks if the predicate is true or false for the given row. Depending on the given predicate condition, it returns the row or part of the row.

The Tuple Relational Calculus expression Syntax

$\{t \mid P(t)\}$

Where t is the tuple variable that runs over every Row, and $P(t)$ is the predicate logic expression or condition.

Tuple Relational Calculus (TRC) is a non-procedural query language used in relational database management systems (RDBMS) to retrieve data from tables. TRC is based on the concept of tuples, which are ordered sets of attribute values that represent a single row or record in a database table.

Syntax: The basic syntax of TRC is as follows:

$\{t \mid P(t)\}$

where t is a **tuple variable** and $P(t)$ is a **logical formula** that describes the conditions that the tuples in the result must satisfy. The **curly braces** $\{\}$ are used to indicate that the expression is a set of tuples.

For example, let's say we have a table called "Employees" with the following attributes:

Tuple Relational Query

In Tuple Calculus, a query is expressed as

$\{t \mid P(t)\}$

where t = resulting tuples,

$P(t)$ = known as Predicate and these are the conditions that are used to fetch t . Thus, it generates a set of all tuples t , such that Predicate $P(t)$ is true for t .

$P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT ($\neg$).

It also uses quantifiers:

$\exists t \in r (Q(t))$ = "there exists" a tuple in t in relation r such that predicate $Q(t)$ is true.

$\forall t \in r (Q(t))$ = $Q(t)$ is true "for all" tuples in relation r .

DOMAIN RELATIONAL CALCULUS (DRC)

Domain Relational Calculus is similar to Tuple Relational Calculus, where it makes a list of the attributes that are to be chosen from the relations as per the conditions.

$$\{ \langle a_1, a_2, a_3, \dots, a_n \rangle \mid P(a_1, a_2, a_3, \dots, a_n) \}$$

where $a_1, a_2, \dots, a_n$ are the attributes of the relation and P is the condition.

Tuple Relational Calculus Examples

Employee ID

Name

Salary

Department ID

To retrieve the names of all employees who earn more than \$50,000 per year, we can use the following TRC query:

$$\{ t \mid Employees(t) \wedge t.Salary > 50000 \}$$

In this query, the " $Employees(t)$ " expression specifies that the tuple variable t represents a row in the "Employees" table. The " $\wedge$ " symbol is the logical AND operator, which is used to combine the condition " $t.Salary > 50000$ " with the table selection.

The result of this query will be a set of tuples, where each tuple contains the Name attribute of an employee who earns more than \$50,000 per year.

While TRC is a powerful query language, it can be more difficult to write and understand than other SQL-based query languages, such as Structured Query Language (SQL). However, it is useful in certain applications, such as in the formal verification of database schemas and in academic research.

Tuple Relational Calculus is a **non-procedural query language**, unlike relational algebra. Tuple Calculus provides only the description of the query but it does not provide the methods to solve it. Thus, it explains what to do but not how to do it.

Table Customer

Customer name	Street	City
Saurabh	A7	Patiala
Mehak	B6	Jalandhar
Sumiti	D9	Ludhiana
Ria	A5	Patiala

Table Branch

Branch name	Branch City
ABC	Patiala
DEF	Ludhiana
GHI	Jalandhar

Table Account

Account number	Branch name	Balance
1111	ABC	50000
1112	DEF	10000

Account number	Branch name	Balance
1113	GHI	9000
1114	ABC	7000

Table Loan

Loan number	Branch name	Amount
L33	ABC	10000
L35	DEF	15000
L49	GHI	9000
L98	DEF	65000

Table Borrower

Customer name	Loan number
Saurabh	L33
Mehak	L49
Ria	L98

Table Depositor

Customer name	Account number
Saurabh	1111
Mehak	1113
Suniti	1114

Example 1: Find the loan number, branch, and amount of loans greater than or equal to 10000 amount.

$\{t \mid t \in \text{loan} \wedge t[\text{amount}] \geq 10000\}$

Resulting relation:

Loan number	Branch name	Amount
L33	ABC	10000
L35	DEF	15000
L98	DEF	65000

In the above query, $t[\text{amount}]$ is known as a tuple variable.

Example 2: Find the loan number for each loan of an amount greater or equal to 10000.

$\{t \mid \exists s \in \text{loan} (t[\text{loan number}] = s[\text{loan number}] \wedge s[\text{amount}] \geq 10000)\}$

Resulting relation:

Loan number
L33
L35
L98

Example 3: Find the names of all customers who have a loan and an account at the bank.

$$\{t \mid \exists s \in \text{borrower}(t[\text{customer-name}] = s[\text{customer-name}]) \wedge \exists u \in \text{depositor}(t[\text{customer-name}] = u[\text{customer-name}])\}$$

Resulting relation:

Customer name
Saurabh
Mehak

Example 4: Find the names of all customers having a loan at the "ABC" branch.

$$\{t \mid \exists s \in \text{borrower}(t[\text{customer-name}] = s[\text{customer-name}] \wedge \exists u \in \text{loan}(u[\text{branch-name}] = \text{"ABC"} \wedge u[\text{loan-number}] = s[\text{loan-number}]))\}$$

Resulting relation:

Customer name
Saurabh

WHAT IS SQL?

SQL was invented in the 1970s by IBM and was first commercially distributed by Oracle. The original name was SEQUEL (Structured English Query Language), later shortened to SQL. It is a standardized programming language used to manage, manipulate and interact with relational databases.

It allow users to perform tasks such as querying data, creating and modifying database structures, and managing access permissions. SQL is widely used across various relational database management systems such as MySQL, PostgreSQL, Oracle, and SQL Server.

HOW SQL MANAGES AND INTERACTS WITH DATA

Data is the core of every application and SQL plays a key role in managing and interacting with this data. Whether you are working with a small user database or handling massive datasets like sales records, SQL allows you to efficiently query, update, and manage relational databases.

Data Retrieval: SQL constructs a query that tells the Database Management System (DBMS) what data is required.

DBMS Processes Query: The DBMS processes the SQL query, retrieves the data and returns it to the user or application.

Declarative Nature: SQL is declarative, meaning it focuses on what data should be retrieved, organized or modified, leaving the how to the DBMS.

SQL encompasses DDL (Data Definition Language) and DML (Data Manipulation Language) commands, which handle operations like creating, updating, and modifying database structures and data.

KEY COMPONENTS OF A SQL SYSTEM

A SQL system consists of several key components that work together to enable efficient data storage, retrieval, and manipulation.

Databases: Databases are structured collections of data organized into tables, rows, and columns. Databases serve as repositories for storing information efficiently and provide a way to manage and access data.

Tables: Tables are the fundamental building blocks of a database, consisting of rows (records) and columns (attributes or fields). Tables ensure data integrity and consistency by defining the structure and relationships of the stored information.

Queries: Queries are SQL commands used to interact with databases. They enable users to retrieve, update, insert, or delete data from tables, allowing for efficient data manipulation and retrieval.

CONSTRAINTS: Constraints are rules applied to tables to maintain data integrity. Constraints define conditions that data must meet to be stored in the database, ensuring accuracy and consistency.

Stored Procedures: Stored procedures are pre-compiled SQL statements stored in the database. Stored procedures can accept parameters, execute complex

operations, and return results, enhancing efficiency, reusability, and security in database management.

TRANSACTIONS: Transactions are groups of SQL statements that are executed as a single unit of work. Transactions ensure data consistency and integrity by allowing for the rollback of changes if any part of the transaction fails.

Other important components include Data Types, Indexes, Views, Security, Permissions, and Joins.

SQL INJECTION AND DATABASE SECURITY

SQL Injection is a type of cyber attack where malicious SQL queries are inserted into input fields to manipulate the database. This can lead to unauthorized data access, modification, or corruption. Preventing SQL injection involves using parameterized queries and input validation.

WHY IS SQL IMPORTANT?

User-Friendly and Accessible: SQL is designed for a broad range of users, including those with minimal programming experience, making it approachable for non-technical individuals.

Declarative Language: As a non-procedural language, SQL allows users to specify what data is needed rather than how to retrieve it, focusing on the desired results rather than the retrieval process.

Efficient Database Management: SQL enables the creation, modification, and management of databases efficiently, saving time and simplifying complex database operations.

Standardized Language: Based on ANSI (American National Standards Institute) and ISO (International Organization for Standardization) standards, SQL ensures consistency and stability across various database management systems (DBMS).

Command Structure: SQL does not require a continuation character for multi-line queries, allowing flexibility in writing commands across one or multiple lines.

Execution Mechanism: Queries are executed using a termination character (e.g., a semicolon ;), enabling immediate and accurate command processing.

Built-in Functionality: SQL includes a rich set of built-in functions for data manipulation, aggregation, and formatting, empowering users to handle diverse data-processing needs effectively.

HOW DOES SQL WORKS?

Structured Query Language (SQL) operates on a server machine, where it processes database queries and returns results efficiently. Below are the key steps involved in the SQL query execution.

Input: The process begins when a user submits an SQL query through a database interface or application. This query typically specifies the desired operation, such as data retrieval, insertion, updating, or deletion.

Parsing: The query is passed to the query processor, which breaks it into smaller units called tokens. These tokens represent keywords, table names, column names, and other elements of the query. The processor then validates the syntax against SQL standards and the database schema to ensure the query is well-formed and executable.

Optimization: After parsing, the query is handed to the optimizer, which evaluates multiple ways to execute the query. The optimizer considers factors like indexes, table statistics, and available resources to generate the most efficient execution plan. This step ensures that the query runs with minimal resource consumption and maximum performance.

Execution: The execution engine follows the plan provided by the optimizer. It interacts with the storage engine, which retrieves, manipulates, or updates the required data from the database tables. During this step, SQL statements like SELECT, INSERT, UPDATE, or DELETE are translated into actions performed on the underlying data.

Output: Once the execution engine processes the query, the result is formatted and returned to the user. Depending on the query type, the output could be a result set (for SELECT queries) or an acknowledgment of the operation (for INSERT, UPDATE, or DELETE queries).

Rules for Writing SQL Queries

There are certain rules for SQL which would ensure consistency and functionality across databases. By following these rules, queries will be well formed and well executed in any database.

1. Statement Termination: Every SQL statement ends with a semicolon (;), signaling the DBMS to execute the command.

2. Case Insensitivity: SQL keywords (e.g., SELECT, INSERT) are case-insensitive, but database names and column names may be case-sensitive depending on the DBMS.

3. Whitespace Flexibility: SQL statements can span multiple lines, but keywords and identifiers must be separated by at least one space.

3. Unique Identifiers: Reserved words (e.g., SELECT, FROM) cannot be used as table or column names unless enclosed in double quotes (") or backticks (`), depending on the DBMS.

5. Comments: Comments enhance readability:

Single-line comments: --

Multi-line comments: /* ... */

6. **Data Integrity:** Constraints like NOT NULL, UNIQUE, and PRIMARY KEY must be defined correctly to maintain data consistency.

7. **String Literals:** String values must be enclosed in single quotes (').

8. **Valid Identifiers:** Table and column names must:

Begin with an alphabetic character.

Contain up to 30 characters.

Avoid special characters except underscores (_).

WHAT ARE SQL COMMANDS?

Structured Query Language (SQL) commands are standardized instructions used by developers to interact with data stored in relational databases. These commands allow for the creation, manipulation, retrieval and control of data, as well as database structures. SQL commands are categorized based on their specific functionalities:

1. DATA DEFINITION LANGUAGE

These commands are used to define the structure of database objects by creating, altering, and dropping the database objects. Based on the needs of the business, database engineers create and modify database objects using DDL. The CREATE command, for instance, is used by the database engineer to create database objects like tables, views, and indexes.

Command Description:

CREATE

Creates a new table, a view on a table, or some other object in the database.

Example: CREATE TABLE

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT,  
    Course VARCHAR(50)  
);
```

Output:

Query OK, 0 rows affected

A table named `Students` is created with 4 columns.

INSERT DATA INTO TABLE

```
INSERT INTO Students (StudentID, Name, Age, Course)
VALUES
(1, 'Ravi Kumar', 20, 'BCA'),
(2, 'Anita Sharma', 21, 'BBA'),
(3, 'Vikram Singh', 22, 'B.Sc');
```

Output:

Query OK, 3 rows affected

SELECT DATA FROM TABLE (to show output)

```
SELECT * FROM Students;
```

Output Table:

StudentID	Name	Age	Course
1	Ravi Kumar	20	BCA
2	Anita Sharma	21	BBA
3	Vikram Singh	22	B.Sc

ALTER

Modifies an existing database object, such as a table

DROP

Deletes an entire table, a view of a table, or other objects in the database

Example :

SQL ALTER TABLE – Use on Students Table with Output

We'll perform the following actions:

1. Add a new column Gender
2. Modify the data type of Age
3. Rename the column Course to Program
4. Drop the column Age

1. ADD a Column

```
ALTER TABLE Students  
ADD Gender CHAR(1);
```

Output:

```
Query OK, 0 rows affected  
Records: 0 Duplicates: 0 Warnings: 0
```

STRUCTURE AFTER ADDING:

StudentID	Name	Age	Course	Gender
1	Ravi Kumar	20	BCA	NULL
2	Anita Sharma	21	BBA	NULL
3	Vikram Singh	22	B.Sc	NULL

2. MODIFY THE COLUMN AGE TO SMALLINT

```
ALTER TABLE Students  
MODIFY Age SMALLINT;
```

Output:

```
Query OK, 0 rows affected  
Records: 3 Duplicates: 0 Warnings: 0
```

This changes the datatype of Age from INT to SMALLINT.

3. RENAME Column Course to Program

```
ALTER TABLE Students  
RENAME COLUMN Course TO Program;
```

Output:

```
Query OK, 0 rows affected  
Records: 0  Duplicates: 0  Warnings: 0
```

TABLE STRUCTURE NOW:

StudentID	Name	Age	Program	Gender
1	Ravi Kumar	20	BCA	NULL
2	Anita Sharma	21	BBA	NULL
3	Vikram Singh	22	B.Sc	NULL

4. DROP Column Age

```
ALTER TABLE Students  
DROP COLUMN Age;
```

Output:

```
Query OK, 0 rows affected  
Records: 3  Duplicates: 0  Warnings: 0
```

FINAL TABLE STRUCTURE:

StudentID	Name	Program	Gender
1	Ravi Kumar	BCA	NULL
2	Anita Sharma	BBA	NULL
3	Vikram Singh	B.Sc	NULL

2. DATA MANIPULATION LANGUAGE

A relational database can be updated with new data using data manipulation language (DML) statements. The INSERT command, for instance, is used by an application to add a new record to the database.

Command Description:

INSERT

To Creates a record.

UPDATE

To Modifies records.

DELETE

To Deletes records.\

Example: **INSERT**, **UPDATE**, and **DELETE** SQL commands using the `Students` table we've created earlier. For each command, I'll show:

- SQL query
- Result/Output
- Table before and after the operation (for clarity)

Table Before Any Operation

Student-ID	Name	Program	Gender
1	Ravi Kumar	BCA	NULL
2	Anita Sharma	BBA	NULL
3	Vikram Singh	B.Sc	NULL

1. INSERT – Adds a New Record

Query:

```
INSERT INTO Students (StudentID, Name, Program, Gender)
VALUES (4, 'Pooja Verma', 'BCA', 'F');
```

Output:

Query OK, 1 row affected

Table After INSERT:

Student-ID	Name	Program	Gender
1	Ravi Kumar	BCA	NULL
2	Anita Sharma	BBA	NULL
3	Vikram Singh	B.Sc	NULL
4	Pooja Verma	BCA	F

2. UPDATE – Modifies Existing Record(s)

Query:

```
UPDATE Students  
SET Gender = 'M'  
WHERE StudentID = 1;
```

Output:

Query OK, 1 row affected

Table After UPDATE:

StudentID	Name	Program	Gender
1	Ravi Kumar	BCA	M
2	Anita Sharma	BBA	NULL
3	Vikram Singh	B.Sc	NULL
4	Pooja Verma	BCA	F

3. DELETE – Removes Record(s)

Query:

```
DELETE FROM Students  
WHERE StudentID = 2;
```

Output:

Query OK, 1 row affected

Table After DELETE:

Student-ID	Name	Program	Gender
1	Ravi Kumar	BCA	M
3	Vikram Singh	B.Sc	NULL
4	Pooja Verma	BCA	F

SUMMARY TABLE OF COMMANDS

Command	Purpose	Example Usage
INSERT	Add new row	INSERT INTO Students (...) VALUES (...);
UPDATE	Modify existing row	UPDATE Students SET Gender = 'M' WHERE StudentID=1;
DELETE	Remove row	DELETE FROM Students WHERE StudentID = 2;

3. DATA QUERY LANGUAGE

Data retrieval instructions are written in the data query language (DQL), which is used to access relational databases. The SELECT command is used by software programs to filter and return particular results from a SQL table.

4. DATA CONTROL LANGUAGE

DCL commands manage user access to the database by granting or revoking permissions. Database administrators use DCL to enforce security and control access to database objects.

(a). GRANT – Giving Permissions to the user

Syntax:

```
GRANT privilege_name ON object_name TO user_name;
```

Example:

Let's say we have a table called employees, and we want to give user john permission to select (read) data from this table:

```
GRANT SELECT ON employees TO john;
```

This allows user john to run:

```
SELECT * FROM employees;
```

(b). REVOKE – Removing Permissions from the user

Syntax:

```
REVOKE privilege_name ON object_name FROM user_name;
```

Example:

Now suppose we want to remove john's permission to read data from the employees table:

```
REVOKE SELECT ON employees FROM john;
```

After this command, john will no longer be able to run:

```
SELECT * FROM employees;
```

Common Privileges Used in DCL:

- SELECT – Read data.
- INSERT – Add new data.
- UPDATE – Modify existing data.
- DELETE – Remove data.
- ALL PRIVILEGES – Grants all of the above.

5. Transaction Control Language

TCL commands manage transactions in relational databases, ensuring data integrity and consistency. These commands are used to commit changes or roll back operations in case of errors.

Command

Description

COMMIT

Saves all changes made during the current transaction on a permanent basis. Some databases provide an auto-commit feature, which can be configured using settings.

ROLLBACK

Cancels all changes made in the current transaction. Reverts changes made during the current transaction, ensuring no unwanted changes are saved.

SAVEPOINT

Sets a point within a transaction to which you can later roll back or Sets a point within a transaction to which changes can be rolled back, allowing partial rollbacks.

BENEFITS OF SQL:

Efficiency: SQL is designed to handle complex queries and large datasets with optimal performance, making data retrieval and manipulation seamless.

Standardization: As an ANSI and ISO standard language, SQL provides a universal method to interact with relational databases across platforms.

Scalability: SQL supports databases ranging from small-scale applications to enterprise-level systems, ensuring smooth operations regardless of size.

Flexibility: SQL can be extended with procedural programming (e.g., PL/SQL, T-SQL) to build complex business logic and custom functions.

Limitations of SQL

Complexity in Advanced Operations: Advanced functionalities such as indexing, query optimization and performance tuning require in-depth technical knowledge.

Scalability Concerns: SQL performs best with structured data; handling unstructured data or massive distributed systems can pose challenges.

Platform-Specific Variations: While SQL is standardized, many databases implement unique extensions, leading to portability and compatibility issues.

SQL USE CASES

E-Commerce: Manage customer orders, product catalogs, and inventory.

Healthcare: Maintain patient records and appointment schedules.

Banking: Analyze transaction histories and generate financial reports.

Web Development: Power dynamic websites with user-specific content.

Machine Learning and Data Science: Combine SQL databases with tools like Python, R, and Tensor Flow to streamline machine learning workflows.

SQL(Structured Query Language) is a programming language designed for managing and manipulating data stored in relational databases. It is used for interacting with DBMS like MySQL, SQL Server, Oracle, and PostgreSQL. In this article we have covered about SQL and understood its characteristics, rules to write SQL queries, commands and many important concepts. We also covered the SQL injection and how it can be harmful for database security.

1. SQL DATA TYPES AND LITERALS

SQL Data Types

SQL data types define the type of data a column can hold in a database table.

Common SQL Data Types:

Category	Data Type	Description
Numeric	INT, FLOAT, DECIMAL	Stores whole numbers and decimal values.
Character	CHAR(n), VARCHAR(n)	Fixed or variable-length string.
Date/Time	DATE, TIME, DATETIME	Stores date and time values.
Boolean	BOOLEAN	Stores TRUE or FALSE.
Binary	BLOB	Stores binary data (e.g., images, files).

SQL Literals

Literals are fixed values in SQL queries.

Types of Literals:

- **String Literal:** 'Hello', 'SQL'
- **Numeric Literal:** 100, 99.99
- **Date Literal:** '2025-07-05'
- **Boolean Literal:** TRUE, FALSE

2. TYPES OF SQL COMMANDS

SQL commands are divided into categories based on their function.

Main Types:

Command Type	Description	Example
DDL (Data Definition Language)	Defines database schema	CREATE, ALTER, DROP
DML (Data Manipulation Language)	Manipulates data	SELECT, INSERT, UPDATE, DELETE
DCL (Data Control Language)	Controls access to data	GRANT, REVOKE
TCL (Transaction Control Language)	Manages transactions	COMMIT, ROLLBACK, SAVEPOINT

3. SQL Operators and Their Procedure

SQL operators are used in queries to perform operations on data.

TYPES OF SQL OPERATORS:

1. Arithmetic Operators:

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

SELECT 100 + 20 AS Total;

2. Comparison Operators:

Operator	Description
=	Equal to
!= or <>	Not equal to

Operator	Description
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

SELECT * FROM employees WHERE salary > 50000;

3. Logical Operators:

Operator	Description
AND	Returns true if both conditions are true
OR	Returns true if any condition is true
NOT	Reverses the condition result

Example:

SELECT * FROM students WHERE age > 18 AND grade = 'A';

4. Special Operators:

Operator	Description
BETWEEN	Checks a range
IN	Matches any value in a list
LIKE	Pattern matching
IS NULL	Checks for NULL values

Examples:

SELECT * FROM books WHERE price BETWEEN 100 AND 500;
SELECT * FROM users WHERE name LIKE 'A%';

1. TABLES, VIEWS, AND INDEXES

1.1. TABLE

Definition:

A **table** is a collection of related data entries organized in **rows and columns**. Each column represents a field (attribute), and each row represents a record.

Example:

```
CREATE TABLE Students (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT,  
    Course VARCHAR(30)  
);
```

This creates a table called Students with four columns: ID, Name, Age, and Course.

1.2. VIEW

Definition:

A **view** is a virtual table that is based on a **SELECT query**. It does not store data physically but displays data from one or more tables.

Uses:

- Simplify complex queries
- Provide security by hiding certain columns

Example:

```
CREATE VIEW TeenStudents AS  
SELECT Name, Age FROM Students WHERE Age BETWEEN 13 AND 19;
```

To use the view:

```
SELECT * FROM TeenStudents;
```

1.3. INDEX

Definition:

An **index** is a database object that improves the speed of **data retrieval** operations on a table. It works like an index in a book.

Example:

```
CREATE INDEX idx_student_name ON Students(Name);
```

This creates an index on the Name column of the Students table to make searches faster.

2. QUERIES AND SUBQUERIES

2.1. Query

Definition:

A **query** is a command used to retrieve data from one or more tables.

Example:

```
SELECT Name, Age FROM Students WHERE Age > 18;
```

2.2. Subquery

Definition:

A **subquery** is a query nested inside another query. It can be used in SELECT, WHERE, or FROM clauses.

Example:

```
SELECT Name FROM Students  
WHERE Age = (SELECT MAX(Age) FROM Students);
```

This finds the student(s) with the maximum age.

3. AGGREGATE FUNCTIONS

Definition:

Aggregate functions perform calculations on a group of rows and return a single value.

Function	Description	Example
COUNT()	Counts total number of rows	SELECT COUNT(*) FROM Students;
SUM()	Returns sum of a column	SELECT SUM(Age) FROM Students;
AVG()	Returns average value	SELECT AVG(Age) FROM Students;
MAX()	Returns highest value	SELECT MAX(Age) FROM Students;
MIN()	Returns lowest value	SELECT MIN(Age) FROM Students;

4. INSERT, UPDATE, AND DELETE OPERATIONS

4.1. INSERT

Definition:

Used to add new records to a table.

Example:

```
INSERT INTO Students (ID, Name, Age, Course)
VALUES (1, 'Amit', 20, 'BCA');
```

4.2. UPDATE

Definition:

Used to modify existing data in a table.

Example:

```
UPDATE Students SET Age = 21 WHERE ID = 1;
```

4.3. DELETE

Definition:

Used to delete records from a table.

Example:

```
DELETE FROM Students WHERE ID = 1;
```

5. JOINS

JOINS in DBMS: Theory with Detailed Examples

In **Relational Databases**, data is stored in **multiple tables**. A **JOIN** in SQL helps us to **fetch data from two or more tables** based on a **related column** (commonly a foreign key).

SQL joins are fundamental tools for combining data from multiple tables in relational databases. Joins allow efficient data retrieval, which is essential for generating meaningful observations and solving complex business queries.

Why Use Joins?

Imagine a college database:

- One table stores student details.
- Another stores their course details.

Now, if we want to display both student and course names together, we need to **JOIN** those two tables using a common field (like `CourseID`).

Types of Joins

1. INNER JOIN (Most Common)

Definition:

Fetches only those records that have **matching values** in both tables. The **INNER JOIN** keyword selects all rows from both the tables as long as the condition is satisfied.

Example:

Table 1: Students

StudentID	Name	CourseID
1	Rohan	101
2	Priya	102
3	Aakash	103
4	Neha	104

Table 2: Courses

CourseID	CourseName
101	BCA
102	B.Tech
103	BBA

Query:

```
SELECT Students.Name, Courses.CourseName
FROM Students
INNER JOIN Courses ON Students.CourseID =
Courses.CourseID;
```

Output:

Name	CourseName
Rohan	BCA
Priya	B.Tech
Aakash	BBA

□ **Note:** Neha's CourseID (104) is not in the Courses table, so her record is excluded.

2. LEFT JOIN

Definition:

Returns **all records** from the **left table (Students)**, and matching records from the right table. If no match, it shows NULL. A LEFT JOIN returns all rows from the left table, along with matching rows from the right table. If there is no match, NULL values are returned for columns from the right table. LEFT JOIN is also known as LEFT OUTER JOIN.

Query:

```
SELECT Students.Name, Courses.CourseName
FROM Students
LEFT JOIN Courses ON Students.CourseID =
Courses.CourseID;
```

Output:

Name	CourseName
Rohan	BCA
Priya	B.Tech
Aakash	BBA
Neha	NULL

□ **Note:** Neha is shown even though her course is missing. Her course name is NULL.

3. RIGHT JOIN

Definition:

Returns **all records from the right table** (Courses), and matching records from the left (Students). If no match, student data is shown as NULL.

RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join. It is very similar to LEFT JOIN for the rows for which there is no matching row on the left side, the result-set will contain null. RIGHT JOIN is also known as RIGHT OUTER JOIN.

Query:

```
SELECT Students.Name, Courses.CourseName
FROM Students
RIGHT JOIN Courses ON Students.CourseID =
Courses.CourseID;
```

Output:

Name	CourseName
Rohan	BCA
Priya	B.Tech
Aakash	BBA
NULL	B.Com

(Assume Courses table has one more row: 104 - B.Com)

4. FULL OUTER JOIN

Definition:

Returns **all records** when there is a match in **either** table. Non-matching rows are filled with NULL.

FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values.

⚠ Note: MySQL doesn't support FULL OUTER JOIN directly. Use UNION of LEFT and RIGHT joins.

Query:

```
SELECT Students.Name, Courses.CourseName
FROM Students
LEFT JOIN Courses ON Students.CourseID =
Courses.CourseID
UNION
```

```
SELECT Students.Name, Courses.CourseName
FROM Students
RIGHT JOIN Courses ON Students.CourseID =
Courses.CourseID;
```

Output:

Name	CourseName
Rohan	BCA
Priya	B.Tech
Aakash	BBA
Neha	NULL
NULL	B.Com

5. CROSS JOIN

Definition:

Returns the **cartesian product** of two tables. Each row of first table is combined with **all rows** of second.

Query:

```
SELECT Students.Name, Courses.CourseName
FROM Students
CROSS JOIN Courses;
```

Output (partial, 4x4 = 16 rows):

Name	CourseName
Rohan	BCA
Rohan	B.Tech
Rohan	BBA
Rohan	B.Com
Priya	BCA
...	...

- ❑ Use with caution! May produce huge results.

6. SELF JOIN

Definition:

Used when a table needs to be joined with **itself**.

Example: Employees table

EmpID	EmpName	ManagerID
1	Rakesh	NULL
2	Suman	1
3	Arjun	1
4	Meena	2

Query:

```
SELECT E1.EmpName AS Employee, E2.EmpName AS Manager
FROM Employees E1
LEFT JOIN Employees E2 ON E1.ManagerID = E2.EmpID;
```

Output:

Employee	Manager
Rakesh	NULL
Suman	Rakesh
Arjun	Rakesh
Meena	Suman

6. What is a NATURAL JOIN?

A **NATURAL JOIN** in SQL joins two tables **automatically using the common column(s)** — **without writing the ON condition**. It matches **columns with the same name and same type** from both tables.

Example:

Table: Students

student_id	name	age
1	Rahul Verma	20
2	Anjali Singh	21
3	Mohit Gupta	22

Table: Marks

student_id	subject	marks
1	Math	85
1	English	90
2	Science	88

student_id	subject	marks
3	Computer	91

Common column in both tables: student_id

SQL Query using NATURAL JOIN

```
SELECT *
FROM Students
NATURAL JOIN Marks;
```

Output:

student_id	name	age	subject	marks
1	Rahul Verma	20	Math	85
1	Rahul Verma	20	English	90
2	Anjali Singh	21	Science	88
3	Mohit Gupta	22	Computer	91

Explanation:

- SQL automatically joins the two tables using the **student_id** column.
- Only **rows with matching student_id** are included.
- You don't need to write `ON Students.student_id = Marks.student_id`, because **NATURAL JOIN** does it **automatically**.

SUMMARY TABLE OF JOINS

Type	Description
INNER JOIN	Only matched rows from both tables
LEFT JOIN	All rows from left table + matched from right
RIGHT JOIN	All rows from right table + matched from left
FULL OUTER	All rows from both tables (matched + unmatched)
CROSS JOIN	Cartesian product (every row with every row)
SELF JOIN	Join a table with itself

Real-World Applications of Joins:

- Showing students and their enrolled courses.
- Displaying employee-manager relationships.

- Joining customer orders with payment history.
- Building dashboards from multiple datasets.

• 6.1 RELATIONAL ALGEBRA IN DBMS

Relational Algebra is a **procedural query language** used in **relational databases**. It provides a set of operations to retrieve data from **one or more relations (tables)** based on specific conditions.

It tells **HOW** to get the result, not just **WHAT** result is needed.

Why Relational Algebra?

- It forms the **theoretical foundation of SQL**.
- Used in **query processing, optimization, and compiler design**.
- Enables **manipulation and retrieval** of data using mathematical logic.

Basic Terminology

Term	Meaning
Relation	Table
Tuple	Row in a table
Attribute	Column in a table
Degree	No. of columns
Cardinality	No. of rows

Types of Relational Algebra Operations

1. SELECTION (σ)

Use:

Used to **select rows (tuples)** that satisfy a given condition.

□ Syntax:

$\sigma<\text{condition}>(\text{Relation})$

Example:

Table: Students

RollNo	Name	City
1	Rahul	Delhi
2	Priya	Mumbai
3	Meena	Delhi

RollNo	Name	City
4	Arjun	Kolkata

Query:

σ City = 'Delhi' (Students)

Output:

RollNo	Name	City
1	Rahul	Delhi
3	Meena	Delhi

SQL Equivalent:

SELECT * FROM Students WHERE City = 'Delhi';

□ **2. PROJECTION (π)**

Use:

Used to **select specific columns** (attributes) from a table.

Syntax:

π <columns>(Relation)

Query:

π Name, City (Students)

Output:

Name	City
Rahul	Delhi
Priya	Mumbai
Meena	Delhi
Arjun	Kolkata

SQL Equivalent:

SELECT Name, City FROM Students;

□ **3. UNION ($\cup$)**

Use:

Combines all **unique rows** from two relations.

Both tables must have **same number of columns** and **same data types**.

Table A: Students_2023

RollNo	Name
1	Rahul
2	Priya

Table B: Students_2024

RollNo	Name
2	Priya
3	Meena

Query:

Students_2023 U Students_2024

Output:

RollNo	Name
1	Rahul
2	Priya
3	Meena

☐ **SQL Equivalent:**

```
SELECT * FROM Students_2023
```

```
UNION
```

```
SELECT * FROM Students_2024;
```

☐ **4. SET DIFFERENCE OR MINUS (-)**

Use:

Finds rows that are in the **first relation** but **not in the second**.

Query:

Students_2023 - Students_2024

Output:

RollNo	Name
1	Rahul

☐ **SQL Equivalent:**

```
SELECT * FROM Students_2023
```

```
WHERE (RollNo, Name) NOT IN (  
    SELECT RollNo, Name FROM Students_2024);
```

□ 5. INTERSECTION ($\cap$)

Use:

Finds rows that are **common** in both tables.

Query:

```
Students_2023  $\cap$  Students_2024
```

Output:

RollNo	Name
2	Priya

□ SQL Equivalent:

```
SELECT * FROM Students_2023  
INTERSECT  
SELECT * FROM Students_2024;
```

□ 6. CARTESIAN PRODUCT ($\times$)

Use:

Combines **each row** of one table with **each row** of another. Often used before JOIN with filtering.

Table A: Students

RollNo	Name
1	Rahul
2	Priya

Table B: Courses

CID	CourseName
C1	BCA
C2	BBA

Query:

```
Students  $\times$  Courses
```

Output:

RollNo	Name	CID	CourseName
1	Rahul	C1	BCA
1	Rahul	C2	BBA
2	Priya	C1	BCA
2	Priya	C2	BBA

❑ **SQL Equivalent:**

```
SELECT * FROM Students, Courses;
```

❑ **7. RENAME (ρ)**

Use:

Renames a table or its columns.

❑ **Syntax:**

ρ (NewRelationName, OldRelation)

Example:

ρ (StudentDetails, Students)

❑ **SQL Equivalent:**

```
SELECT * FROM Students AS StudentDetails;
```

🔗 **JOIN OPERATIONS (Derived from Cartesian + Selection)**

8. NATURAL JOIN ($\bowtie$)

Use:

Joins two tables by **common attribute** (e.g., matching CourseID).

Students Table:

RollNo	Name	CourseID
1	Rahul	C1
2	Priya	C2

Courses Table:

CourseID	CourseName
C1	BCA
C2	BBA

Query:

```
Students  $\bowtie$  Courses
```

Output:

RollNo	Name	CourseID	CourseName
1	Rahul	C1	BCA
2	Priya	C2	BBA

SQL Equivalent:

```
SELECT * FROM Students  
JOIN Courses ON Students.CourseID = Courses.CourseID;
```

9. THETA JOIN (θ Join)

Use:

Join two tables with **any condition** (not just equality).

Query:

Students $\bowtie$ Students.CourseID $\neq$ Courses.CourseID

10. OUTER JOINS

Not directly available in Relational Algebra, but important in SQL.

- **Left Outer Join**
- **Right Outer Join**
- **Full Outer Join**

These are simulated in relational algebra using **union**, **difference**, and **null handling**.

Summary Table

Operation	Symbol	Description
Selection	σ	Select rows (tuples)
Projection	π	Select columns (attributes)
Union	$\cup$	Combine two relations
Difference	$-$	Tuples in A not in B
Intersection	$\cap$	Tuples common to A and B
Cartesian Prod.	$\times$	Every tuple with every other
Rename	ρ	Rename a table or attribute
Join	$\bowtie$	Combines related rows from tables

Conclusion

- Relational Algebra is the **theoretical foundation** of all relational queries.
- Helps in **query optimization and analysis**.
- Every SQL query can be broken down into a **series of relational algebra operations**.

6. Union, Intersection, Minus

6.1. UNION

Definition:

Combines results of two SELECT queries, removing duplicates.

SELECT Name FROM Students

UNION

SELECT Name FROM Teachers;

6.2. INTERSECT

Definition:

Returns only the common records.

```
SELECT Name FROM Students
INTERSECT
SELECT Name FROM Teachers;
```

6.3. MINUS (or EXCEPT)

Definition:

Returns records from the first query that are not in the second.

```
SELECT Name FROM Students
MINUS
SELECT Name FROM Teachers;
```

7. Cursors (PL/SQL)

Definition:

A **cursor** is used in PL/SQL to process rows returned by a query one by one.

Example:

```
DECLARE
  CURSOR stu_cursor IS SELECT Name FROM Students;
  stu_name Students.Name%TYPE;
BEGIN
  OPEN stu_cursor;
  LOOP
    FETCH stu_cursor INTO stu_name;
    EXIT WHEN stu_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(stu_name);
  END LOOP;
  CLOSE stu_cursor;
END;
```

7. TRIGGERS

Trigger: A procedure that runs **automatically** in response to events (INSERT, DELETE, UPDATE).

A **trigger** is a PL/SQL procedure that runs **automatically** when an event (INSERT, UPDATE, DELETE) occurs.

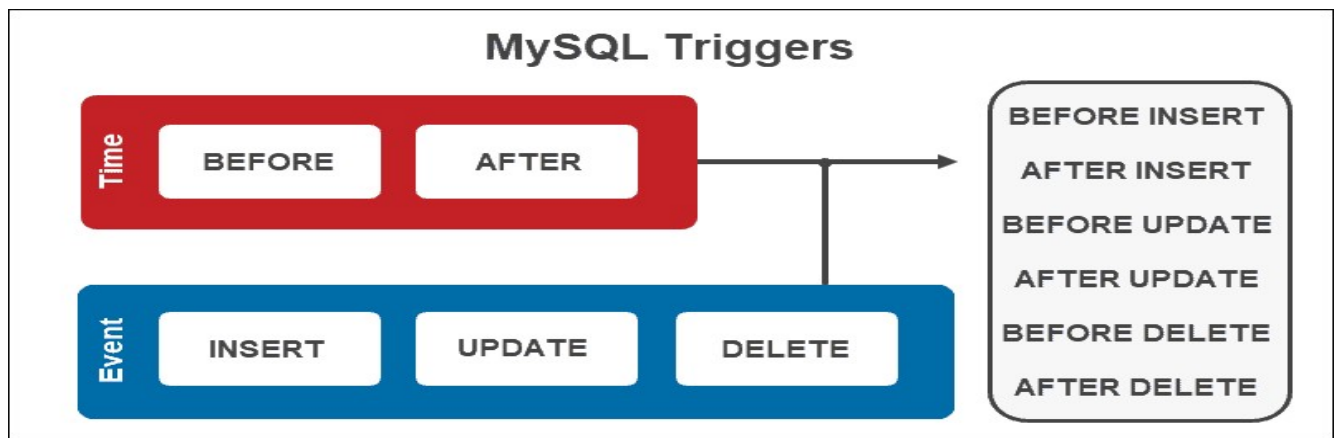
Trigger is a statement that a system executes automatically when there is any modification to the database. In a trigger, we first specify when the trigger is to be executed and then the action to be performed when the trigger executes. Triggers are used to specify certain integrity constraints and referential constraints that cannot be specified using the constraint mechanism of SQL.

Example - Suppose, we are adding a tuple to the 'Donors' table that is some person has donated blood. So, we can design a trigger that will automatically add the value of donated blood to the 'Blood_record' table.

TYPES OF TRIGGERS –

We can define 6 types of triggers for each table:

1. **AFTER INSERT** activated after data is inserted into the table.
2. **AFTER UPDATE**: activated after data in the table is modified.
3. **AFTER DELETE**: activated after data is deleted/removed from the table.
4. **BEFORE INSERT**: activated before data is inserted into the table.
5. **BEFORE UPDATE**: activated before data in the table is modified.
6. **BEFORE DELETE**: activated before data is deleted/removed from the table.



Example (AFTER INSERT Trigger in MySQL):

```
CREATE TRIGGER logInsert AFTER INSERT ON Student
FOR EACH ROW
INSERT INTO log_table (action) VALUES ('New student added');
```

Example:

```
CREATE OR REPLACE TRIGGER trg_student_update
AFTER UPDATE ON Students
FOR EACH ROW
BEGIN
  INSERT INTO log_table VALUES (USER, SYSDATE, 'Student Updated');
END;
```

9. PROCEDURES (PL/SQL)

Definition:

A **procedure** is a named block of code that performs a task and can be called with parameters. In PL/SQL, a procedure is a named block of code that performs a specific action. It can accept input parameters, return output parameters, and execute SQL statements and other PL/SQL logic. Procedures are a fundamental component of modular programming in Oracle databases, allowing for code reusability and organization.

Key characteristics of PL/SQL Procedures:

Definition:

Procedures are created using the CREATE PROCEDURE statement, which includes the procedure's name, its parameters (with their modes: IN, OUT, IN OUT), and the PL/SQL code within the BEGIN...END block.

Parameters:

IN parameters: Pass values into the procedure. They act like constants within the procedure and cannot be assigned new values.

OUT parameters: Return values from the procedure to the caller. They act like uninitialized variables within the procedure and must be assigned a value before the procedure completes.

IN OUT parameters: Pass initial values into the procedure and return updated values to the caller. They act like initialized variables within the procedure and can be used in expressions and assigned new values.

Execution:

Procedures are invoked using the CALL statement or by simply referencing their name with appropriate parameters in other PL/SQL blocks or applications.

No Return Value (Directly):

Unlike functions, procedures do not directly return a value using a RETURN statement. Instead, they return values through their OUT or IN OUT parameters.

Error Handling:

Procedures can incorporate exception handling blocks (EXCEPTION...WHEN...THEN) to manage runtime errors gracefully.

Modularization:

Procedures promote code modularity, allowing complex tasks to be broken down into smaller, manageable units. This enhances readability, maintainability, and reusability of code.

Packages:

Procedures are often grouped within PL/SQL packages, which provide a way to encapsulate related procedures, functions, variables, and types, offering further organization and control over scope and access.

UNIT-3 (DATA BASE DESIGN & NORMALIZATION)

FUNCTIONAL DEPENDENCIES IN DBMS – Explained in Detail

Functional Dependency (FD) is a fundamental concept in the field of **Database Management Systems (DBMS)**, especially in **relational database design and normalization**.

DBMS (Complete syllabus)
Functional Dependency
Properties
1. Reflexivity
2. Transitivity
3. Augmentation
4. Pseudotransitivity
5. Composition
6. Decomposition
7. Union
With real life examples

Types
1. Trivial FD
2. Non-trivial FD
3. Multi-valued FD
4. Transitive FD
@Constant Learners

DEFINITION OF FUNCTIONAL DEPENDENCY

A **functional dependency (FD)** is a relationship between two sets of attributes in a relational database. It occurs when the **value of one attribute (or a group of attributes) uniquely determines the value of another attribute**.

Notation:

If attribute Y is functionally dependent on attribute X, it is denoted as:

$X \rightarrow Y$

This means: "**X determines Y**", or "**Y is functionally dependent on X**".

Example:

Consider a table of students:

Roll_No	Name	Department
101	Ravi	CS
102	Seema	IT
103	Arjun	CS

Here:

- $\text{Roll_No} \rightarrow \text{Name}$ means: Knowing the Roll_No, you can uniquely determine the student's name.
- $\text{Roll_No} \rightarrow \text{Department}$ means: Knowing the Roll_No, you can also determine the Department.

So, we can write:

$\text{Roll_No} \rightarrow \text{Name, Department}$

Functional Dependency
It is a constraint between two sets of attributes in a relation from a Database.

Example : $R(A,B,C,D)$

{
 $A \rightarrow B$
 $B \rightarrow C$
 $C \rightarrow D$
 $D \rightarrow A$
}

Determinant
Dependent

www.go-gate-iit.com - It's all about GATEing

TYPES OF FUNCTIONAL DEPENDENCIES

1. Trivial Functional Dependency

If the dependent attribute is a subset of the determinant, it's trivial.

Example:

$\{\text{Roll_No, Name}\} \rightarrow \text{Roll_No}$

This is trivial because Roll_No is part of the left side.

2. Non-Trivial Functional Dependency

If the dependent attribute is **not a subset** of the determinant.

Example:

$\text{Roll_No} \rightarrow \text{Name}$

Here, Name is not part of the determinant Roll_No.

3. Full Functional Dependency

Y is **fully functionally dependent** on X if removing any attribute from X makes the dependency invalid.

Example:

$\{\text{Roll_No}, \text{Subject_Code}\} \rightarrow \text{Marks}$

This is full dependency if:

- Marks depend on both Roll_No and Subject_Code,
- Not on Roll_No or Subject_Code alone.

4. Partial Dependency

Y is partially dependent on X if **some attribute** in X can be removed and the dependency still holds.

Example:

$\{\text{Roll_No}, \text{Subject_Code}\} \rightarrow \text{Name}$

If Name depends only on Roll_No, not on Subject_Code, then it is a partial dependency.

5. Transitive Dependency

If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$ is a **transitive dependency**.

Example:

- $\text{Roll_No} \rightarrow \text{Department_ID}$
- $\text{Department_ID} \rightarrow \text{Department_Name}$ So, $\text{Roll_No} \rightarrow \text{Department_Name}$ (transitive)

6. Multivalued Dependency (used in 4NF)

Occurs when one attribute in a table uniquely determines multiple values of another attribute.

Importance of Functional Dependencies

1. **Normalization** – Helps break down tables to eliminate redundancy and anomalies.
2. **Design Integrity** – Ensures consistency in the relational schema.
3. **Detect Anomalies** – Helps avoid insertion, update, and deletion anomalies.
4. **Candidate Keys Identification** – Helps find primary keys based on dependency.

Example of Functional Dependency in Normalization

Given Relation: STUDENT (Roll_No, Name, Course, Fee)

Assume:

- $\text{Roll_No} \rightarrow \text{Name}$
- $\text{Course} \rightarrow \text{Fee}$

FUNCTIONAL DEPENDENCIES:

- Roll_No determines Name (each student has a unique roll number).
- Course determines Fee (each course has a fixed fee).

This helps decompose the relation into two:

1. STUDENT_INFO (Roll_No, Name, Course)
2. COURSE_FEE (Course, Fee)

Closure of Functional Dependency (F+)

The **closure** of a set of FDs is the set of all FDs that can be inferred from it using Armstrong's Axioms.

Armstrong's Axioms (Rules to infer new FDs)

1. **Reflexivity:** If $Y \subseteq X$, then $X \rightarrow Y$
2. **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$
3. **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

R1(stu_id, stu_mobile)

stu_id	stu_mobile
101	89832
101	98302
102	45698
103	92342
103	98364

R2 (stu_id, stu_course)

stu_id	stu_course
101	Java
101	Python
102	PHP
102	Python
103	Java

R (stu_id, stu_mobile, stu_course)

stu_id	stu_mobile	stu_course
101	89832	Java
101	98302	Python
102	45698	PHP
102	45698	Python
103	92342	Java
103	98364	Java

- **Multi valued dependency exist:**

- {stu_id $\twoheadrightarrow$ stu_mobile, stu_id $\twoheadrightarrow$ stu_course
- stu_mobile and stu_course are independent

- **Data Redundancy:**

- For every new course, mobile needs to be repeated.
- For every new mobile, course needs to be repeated.

Summary Table

Type	Example	Description
Trivial FD	$A, B \rightarrow A$	RHS is a subset of LHS
Non-Trivial FD	$A \rightarrow B$	B not in A
Full FD	$A, B \rightarrow C$	C depends on both A and B
Partial FD	$A, B \rightarrow C$, but $A \rightarrow C$	C depends only on part of composite key
Transitive FD	$A \rightarrow B, B \rightarrow C \Rightarrow A \rightarrow C$	Indirect dependency
Multi-valued FD	$A \twoheadrightarrow B$	A determines multiple B values

➤ INTRODUCTION OF DATABASE NORMALIZATION

NORMALIZATION is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.

- **Database normalization** is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).
- Normalization generally involves splitting a table into multiple ones which must be linked each time a query is made requiring data from the split tables.

1. DATA CONSISTENCY

Definition:

When data is the **same and accurate** in all places where it is stored, we call it **consistent**.

Example:

Imagine a student named **Ravi Kumar** has the following data in two different tables:

- In `Students` table:
 - Name: Ravi Kumar
 - Course: BCA
- In `Library` table:
 - Name: Ravi Kumar
 - Course: BCA

Since the course is **same** in both places, the data is **consistent**.

2. DATA INCONSISTENCY

Definition:

When the **same data** is stored in **multiple places but with different values**, it is called **inconsistent**.

Example:

- In `Students` table:
 - Name: Ravi Kumar
 - Course: BCA
- In `Library` table:

- Name: Ravi Kumar
- Course: BBA

Here, Ravi's course is **different** in two places — this is **data inconsistency**. This creates **confusion** and **errors** in data reports and systems.

3. DATA REDUNDANCY

Definition:

When the **same data is repeated** in different tables or columns, it is called **redundancy**.

Example:

Suppose a database has this:

- Students table:
 - StudentID: 1
 - Name: Ravi Kumar
 - Course: BCA
- Library table:
 - StudentID: 1
 - Name: Ravi Kumar
 - Course: BCA

The **same student name and course** is repeated in both tables. This is **data redundancy**.

Problem:

- Takes more **storage space**
- Increases chance of **inconsistency**
- **Hard to update** (if Ravi's course changes, we must update in many places)

How to Solve These Problems?

- Use **Normalization** in databases.
- Store common data **once** and use **relations (foreign keys)** to connect.
- Follow **database integrity rules**.

Why do we need Normalization?

The primary objective for normalizing the relations is to eliminate the below anomalies. Failure to reduce anomalies results in data redundancy, which may threaten data integrity and cause additional issues as the database increases. Normalization consists of a set of procedures that assist you in developing an effective database structure.

- **Insertion Anomalies:** Insertion anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. **For example**, if a database requires that every record has a primary key, but no value is provided for a particular record, it cannot be inserted into the database.
- **Deletion anomalies:** Deletion anomalies occur when deleting a record from a database and can result in the unintentional loss of data. **For example**, if a database contains information about customers and orders, deleting a customer record may also delete all the orders associated with that customer.
- **Updation anomalies:** Updation anomalies occur when modifying data in a database and can result in inconsistencies or errors. **For example**, if a database contains information about employees and their salaries, updating an employee's salary in one record but not in all related records could lead to incorrect calculations and reporting.

Before Normalization: The table is prone to redundancy and anomalies (insertion, update, and deletion).

After Normalization: The data is divided into logical tables to ensure consistency, avoid redundancy and remove anomalies making the database efficient and reliable.

Before Normalization

Employee_Department

Emp_ID	Emp_Name	Department	Dept_Location	Emp_Skills
101	Nick Wise	HR	London	Recruitment, Payroll
102	John Cader	Finance	Australia	Budgeting
103	Lily Case	HR	London	Recruitment
104	Ford Dawid	IT	Chicago	Programming, Testing

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721, 9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 1

Conversion to first normal form

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721	HARYANA	INDIA
1	RAM	9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 2

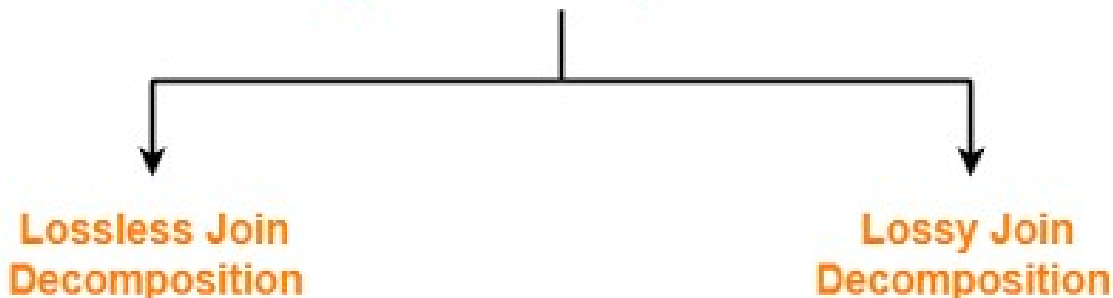
Prerequisites for Understanding Database Normalization

In database normalization, we mainly put only tightly related information together. To find the closeness, we need to find which attributes are dependent on each other. To understand dependencies, we need to learn the below concepts.

Keys are like unique identifiers in a table. For example, in a table of students, the student ID is a key because it uniquely identifies each student. Without keys, it would be hard to tell one record apart from another, especially if some information (like names) is the same. Keys ensure that data is not duplicated and that every record can be uniquely accessed.

DEPENDENCY PRESERVING DECOMPOSITION

Types of Decomposition



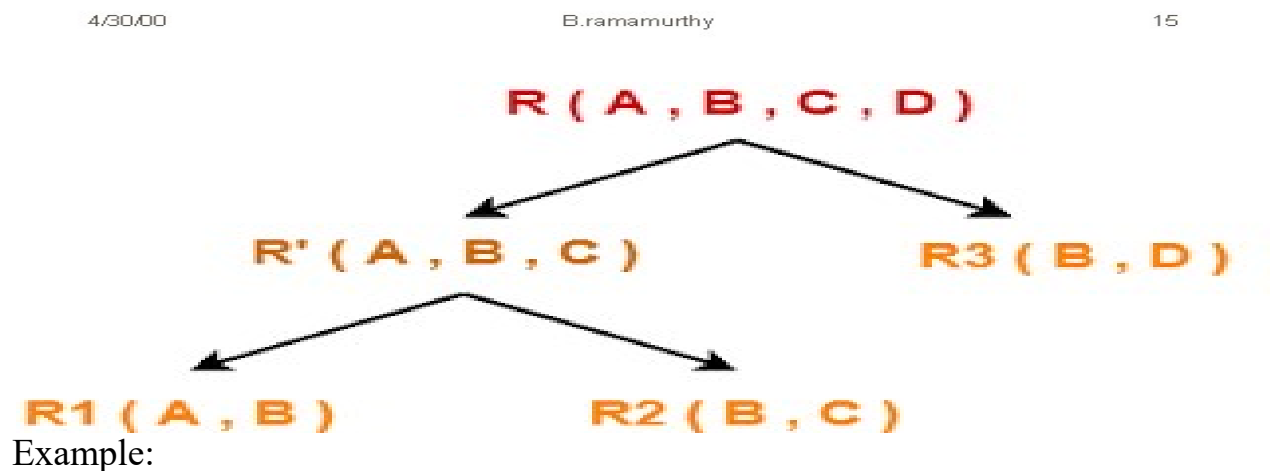
LOSSLESS JOIN DECOMPOSITION IN DBMS

Lossless Decomposition

- If R is split into R1 and R2, for the decomposition to be lossless the **at least one** of the two should hold:

$$R1 \cap R2 \rightarrow R1$$

$$R1 \cap R2 \rightarrow R2$$



Lossless Join Decomposition is a property of **relational database decomposition** that ensures **no data is lost** when a relation (table) is split into two or more smaller relations and then joined back together.

Decomposition means **breaking a relation (table)** into two or more smaller relations. This is often done to remove redundancy or anomalies in data.

A decomposition is **lossless** (or **non-lossy**) if **joining the decomposed tables gives back the original table** exactly — without any **extra** or **missing** data.

Why is Lossless Join Important?

If decomposition is **not lossless**, then when we join the decomposed tables:

- Some original data might be **lost**.
- Or we might get **incorrect data** (extra rows).

So **lossless join** ensures **data integrity** after decomposition.

Summary:

- **Lossless Join Decomposition** = No data is lost or wrongly created after decomposition and join.
- It's **essential** for maintaining correct data.
- Use the rule: **Common attributes must be a key in at least one relation.**

Lossy decomposition

After Natural Join			Original Relation		
Employee	Project	Branch	Employee	Project	Branch
Brown	Mars	L.A.	Brown	Mars	L.A.
Green	Jupiter	San Jose	Green	Jupiter	San Jose
Green	Venus	San Jose	Green	Venus	San Jose
Hoskins	Saturn	San Jose	Hoskins	Saturn	San Jose
Hoskins	Venus	San Jose	Hoskins	Venus	San Jose
Green	Saturn	San Jose			
Hoskins	Jupiter	San Jose			

After Natural Join, we get two extra tuples. Thus, there is loss of information

LOSSY DECOMPOSITION

Definition :

Lossy decomposition is when you break a big table (relation) into smaller tables, and when you try to join them back, you **do not** get the **original** table. Some **data** is **lost or incorrect**.

Simple Example of Lossy Decomposition

Suppose we have a table: **Original Table: StudentCourse**

Student	Course
Alice	Math
Alice	Physics
Bob	Math

Now let's decompose it into two tables:

Decomposed Tables (wrong way):

Table 1: StudentInfo

Student
Alice
Bob

Table 2: CourseInfo

Course
Math
Physics

Now join them back using **Cartesian Product** (or Natural Join without common attribute):

Joined Result:

Student	Course
Alice	Math
Alice	Physics
Bob	Math
Bob	Physics

- This result has **extra row**: Bob - Physics which was **not** in the original table.
- This is a **Lossy Decomposition**.

Example:

Features of Database Normalization

- **Elimination of Data Redundancy:** One of the main features of normalization is to eliminate the data redundancy that can occur in a database.
- **Ensuring Data Consistency:** Normalization helps in ensuring that the data in the database is consistent and accurate.

- **Simplification of Data Management:** Normalization simplifies the process of managing data in a database.
 - **Improved Database Design:** Normalization helps in improving the overall design of the database.
 - **Avoiding Update Anomalies:** Normalization helps in avoiding update anomalies, which can occur when updating a single record in a table affects multiple records in other tables.
 - **Standardization:** Normalization helps in standardizing the data in the database.
- Normal Forms in DBMS

Normal Forms	Description of Normal Forms
First Normal Form (1NF)	A relation is in first normal form if every attribute in that relation is single-valued attribute.
Second Normal Form (2NF)	A relation that is in First Normal Form and every non-primary-key attribute is fully functionally dependent on the primary key, then the relation is in Second Normal Form (2NF).
Third Normal Form (3NF)	A relation is in the third normal form, if there is no transitive dependency for non-prime attributes as well as it is in the second normal form. A relation is in 3NF if at least one of the following conditions holds in every non-trivial function dependency $X \rightarrow Y$. • X is a super key. • Y is a prime attribute (each element of Y is part of some candidate key).
Boyce-Codd Normal Form (BCNF)	For BCNF the relation should satisfy the below conditions • The relation should be in the 3rd Normal Form. • X should be a super-key for every functional dependency (FD) $X \rightarrow Y$ in a given relation.
Fourth Normal Form (4NF)	A relation R is in 4NF if and only if the following conditions are satisfied: • It should be in the Boyce-Codd Normal Form (BCNF). • The table should not have any Multi-valued Dependency.

Normal Forms	Description of Normal Forms
Fifth Normal Form (5NF)	A relation R is in 5NF if and only if it satisfies the following conditions: • R should be already in 4NF. • It cannot be further non loss decomposed (join dependency).

Advantages of Normalization

- Normalization eliminates data redundancy and ensures that each piece of data is stored in only one place, reducing the risk of data inconsistency and making it easier to maintain data accuracy.
- By breaking down data into smaller, more specific tables, normalization helps ensure that each table stores only relevant data, which improves the overall data integrity of the database.
- Normalization simplifies the process of updating data, as it only needs to be changed in one place rather than in multiple places throughout the database.

1NF	2NF	3NF	BCNF
Atomic	Functional Dependency not Partial Dependency	Not Transitive Dependency	$X \rightarrow Y$ X is Super Key 0% redundancy (due to Functional Dependency)

➤ 1NF – First Normal Form

Definition:

A table is in **First Normal Form (1NF)** if it contains only **atomic (indivisible)** values and each column contains values of a **single type**. There should be **no repeating groups** or arrays.

A table is in **First Normal Form** if:

- It contains only **atomic (indivisible)** values.
- There are **no repeating groups** or arrays.
- Each column contains **unique data of a single type**.

Example (Before 1NF):

Roll_No	Name	Subjects
101	Ravi	Math, English

After 1NF:

Roll_No	Name	Subject
101	Ravi	Math
101	Ravi	English

➤ **2NF – Second Normal Form**

Definition:

A table is in **Second Normal Form** if:

- It is already in **1NF**
- **No partial dependency** exists — i.e., every non-prime attribute is **fully functionally dependent** on the **whole primary key**.

Rule:

Remove **partial dependencies** (when an attribute depends on part of a composite key).

Example-1 (Before 2NF):

Roll_No	Subject	Student_Name
101	Math	Ravi

Student_Name depends only on Roll_No, not on the combination.

After 2NF: Student Table:

Roll_No	Student_Name
101	Ravi

Enrollment Table:

Roll_No	Subject
101	Math

Example-2

Students

IDSt	LastName	IDProf	Prof	Grade
1	Mueller	3	Schmid	5
2	Meier	2	Borner	4
3	Tobler	1	Bernasconi	6

Startsituation



Result after normalisation

Students

ID	LastName
1	Mueller
2	Meier
3	Tobler

Professors

IDProf	Professor
1	Bernasconi
2	Borner
3	Schmid

Grades

IDStIDProf	Grade	
1	3	5
2	2	4
3	1	6

➤ 3NF – Third Normal Form

Definition:

A table is in **Third Normal Form (3NF)** if it is in 2NF and has **no transitive dependencies**, meaning **non-key attributes** should not depend on other non-key attributes.

Example (Before 3NF):

Student_ID	Dept_ID	Dept_Name
1	D01	Computer Science

Dept_Name depends on Dept_ID, not directly on Student_ID.

After 3NF: Student Table:

Student_ID	Dept_ID
1	D01

Department Table:

Dept_ID	Dept_Name
D01	Computer Science

➤ BCNF – Boyce-Codd Normal Form

Definition:

A table is in **BCNF** if it is in **3NF** and for every **functional dependency** $X \rightarrow Y$, the left side **X must be a super key**. It handles some anomalies not handled by 3NF.

Example (Before BCNF):

Student_ID	Course	Instructor
1	Math	Mr. Singh

Course $\rightarrow$ Instructor is a functional dependency, but Course is not a super key.

After BCNF: Student Table:

Student_ID	Course
1	Math

Course Table:

Course	Instructor
Math	Mr. Singh

➤ RELATIONAL DATABASE CONCEPTS:

- Inclusion Dependence
- Lossless Join Decomposition
- Normalization using FD (Functional Dependencies)
- MVD (Multivalued Dependencies)
- JD (Join Dependencies)

1. INCLUSION DEPENDENCE (IND):

Definition:

Inclusion Dependency specifies that some attributes in one relation **must** also appear as values of attributes in another relation.

Notation:

If $R[A] \subseteq S[B]$, then every value of attribute A in relation R must also exist in attribute B of relation S.

Example:

Let:

- Employee(EmpID, Name, DeptID)
- Department(DeptID, DeptName)

Here, Employee.DeptID $\subseteq$ Department.DeptID is an inclusion dependency.

Inclusion Dependency (IND)

then $IND \vdash \text{pattern}[\text{course}] \subseteq \text{course}[\text{course}]$

pattern

student	dept	course
1001	1	COMP104
1001	1	COMP171
1002	1	COMP104
1002	1	COMP171
1003	2	ELEC102
1003	2	ELEC151
1004	2	ELEC102
1004	2	ELEC151

course

course	cdescription	dept
COMP104	C++	1
COMP171	Algorithms	1
ELEC102	Electronics	1
ELEC151	Digital Circuit	1

2. LOSSLESS JOIN DECOMPOSITION:

Definition:

A decomposition of a relation schema **R** into schemas **R₁** and **R₂** is **lossless** if we can reconstruct the original relation by joining **R₁** and **R₂**, without loss of information.

Condition (Using FD): Decomposition of R into R₁ and R₂ is lossless with respect to a set of functional dependencies F **if**: $(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2$ must hold in F⁺

Example:

Given R(A, B, C), and FD: $A \rightarrow B$ Decompose into: R₁(A, B), R₂(A, C)
Check: $A = R_1 \cap R_2$, and $A \rightarrow B$ (in FD), so it's **lossless**.

More on Lossless Join

- ❖ The decomposition of R into X and Y is **lossless-join wrt F** if and only if the closure of F contains:

- $X \cap Y \rightarrow X$, or
- $X \cap Y \rightarrow Y$

- ❖ In particular, the decomposition of R into UV and R - V is lossless-join if $U \rightarrow V$ holds over R.

A	B	C
1	2	3
4	5	6
7	2	8



A	B
1	2
4	5
7	2

B	C
2	3
5	6
2	8



A	B	C
1	2	3
4	5	6
7	2	8
1	2	8
7	2	3

3. NORMALIZATION USING FUNCTIONAL DEPENDENCIES (FD):

FD Definition: A functional dependency $X \rightarrow Y$ means that if two tuples agree on attributes X, they must also agree on Y.

Normal Forms:

1NF (First Normal Form):

- Atomic values only (no multivalued attributes).

2NF (Second Normal Form):

- In 1NF and no **partial dependency** (i.e., no FD $X \rightarrow Y$ where X is a proper subset of a candidate key).

3NF (Third Normal Form):

- In 2NF and no **transitive dependency** (i.e., $X \rightarrow Z$, $Z \rightarrow Y$ implies $X \rightarrow Y$, but Z is not a super key).

BCNF (Boyce-Codd Normal Form):

- For every non-trivial FD $X \rightarrow Y$, X must be a **super key**.

4. Normalization Using Multivalued Dependencies (MVD):

MVD Definition: A multivalued dependency $X \twoheadrightarrow Y$ means: For each value of X, the set of values of Y is **independent** of other attributes ($Z = R - X - Y$).

4NF (Fourth Normal Form):

- A relation is in 4NF if it is in BCNF and has **no non-trivial multivalued dependencies**.

Example of MVD:

R(Student, Course, Hobby)

- Student $\twoheadrightarrow$ Course
- Student $\twoheadrightarrow$ Hobby

These MVDs suggest that courses and hobbies are independent and should be in separate tables.

5. Normalization Using Join Dependencies (JD):

JD Definition: A join dependency (JD) is a generalization of multivalued dependency. A relation **R** satisfies a join dependency $JD(R_1, R_2, \dots, R_n)$ if $R = R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$.

5NF (Fifth Normal Form or PJNF):

- A relation is in 5NF if every join dependency in it is implied by the candidate keys (i.e., **decompositions preserve information and dependencies**).

Example:

Relation: R(ProjID, EmpID, Skill) Decomposition into:

- R1(ProjID, EmpID),
- R2(ProjID, Skill),
- R3(EmpID, Skill)

If we can reconstruct R by joining R1, R2, and R3 without redundancy or spurious tuples, then the relation is in 5NF.

Summary Table:

Normal Form	Based On	Purpose
1NF	Atomicity	No multivalued or composite attributes
2NF	FD (No Partial Dependency)	Remove redundancy on part of PK
3NF	FD (No Transitive Dependency)	Remove indirect dependencies
BCNF	FD (LHS is always Super Key)	Stronger version of 3NF
4NF	MVD	No non-trivial multivalued dependencies
5NF	JD	No join dependencies

➤ **Database Design in DBMS**

Before designing a database, it's crucial to understand important terms and concepts. A properly structured database guarantees efficiency, data accuracy, and usability. From understanding data storage to the principles that define data relationships, these concepts are essential for anyone involved in designing, managing, or optimizing databases.

What is database design?

Database Design can be defined as a set of procedures or collection of tasks involving various steps taken to implement a database. A good database design is important. It helps you get the right information when you need it. Following are some critical points to keep in mind to achieve a good database design:

1. Data consistency and integrity must be maintained.
2. Low Redundancy
3. Faster searching through indices
4. Security measures should be taken by enforcing various integrity constraints.
5. Data should be stored in fragmented bits of information in the most atomic format possible.

What are the Following Steps that can be taken by a Database Designer to Ensure Good Database Design?

Step 1: Determine the goal of your database, and ensure clear communication with the stakeholders (if any). Understanding the purpose of a database will help in

thinking of various use cases & where the problem may arise & how we can prevent it.

Step 2: List down all the entities that will be present in the database & what relationships exist among them.

Step 3: Organize the information into different tables such that no or very little redundancy is there.

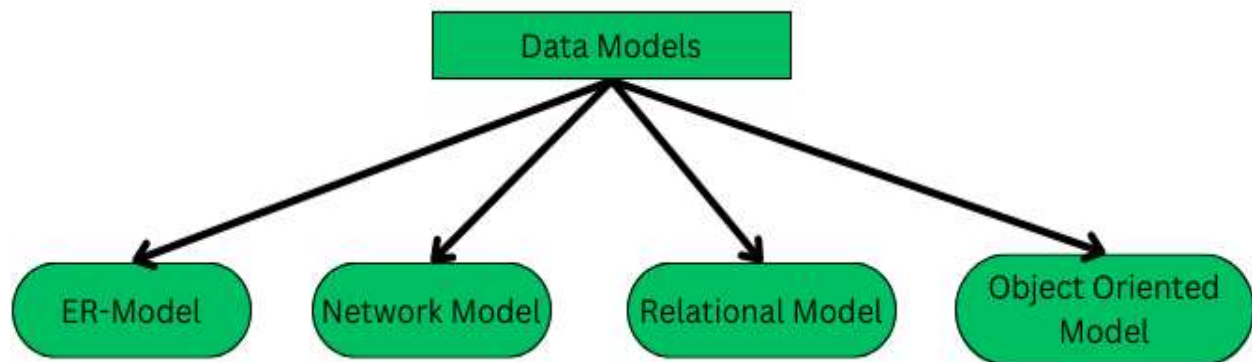
Step 4: Ensure uniqueness in every table. The uniqueness of records present in any relation is a very crucial part of database design that helps us avoid redundancy. Identify the key attributes to uniquely identify every row from columns. You can use various key constraints to ensure the uniqueness of your table, also keep in mind the uniquely identifying records must consume as little space as possible & shall not contain any NULL values.

Step 5: After all the tables are structured, and information is organized apply Normalization Forms to identify anomalies that may arise & redundancy that can cause inconsistency in the database.

LOGICAL DATA MODEL DESIGN

The logical data model design defines the structure of data and what relationship exists among those data. The following are the major components of the logical design:

1. Data Models: Data modeling is a visual modeling technique used to get a high-level overview of our database. Data models help us understand the needs and requirements of our database by defining the design of our database through diagrammatic representation. Ex: model, Network model, Relational Model, object-oriented data model.



DATA MODELS

2. Entity: Entities are objects in the real world, which can have certain properties & these properties are referred to as attributes of that particular entity. There are 2 types of entities: Strong and weak entity, weak entity do not have a key attribute to identify them, their existence solely depends on one 1-specific strong entity &

also have full participation in a relationship whereas strong entity does have a key attribute to uniquely identify them.

Weak entity example: Loan -> Loan will be given to a customer (which is optional) & the loan will be identified by the customer_id to whom the loan is granted.

3. Relationships: How data is logically related to each other defines the relationship of that data with other entities. In simple words, the association of one entity with another is defined here.

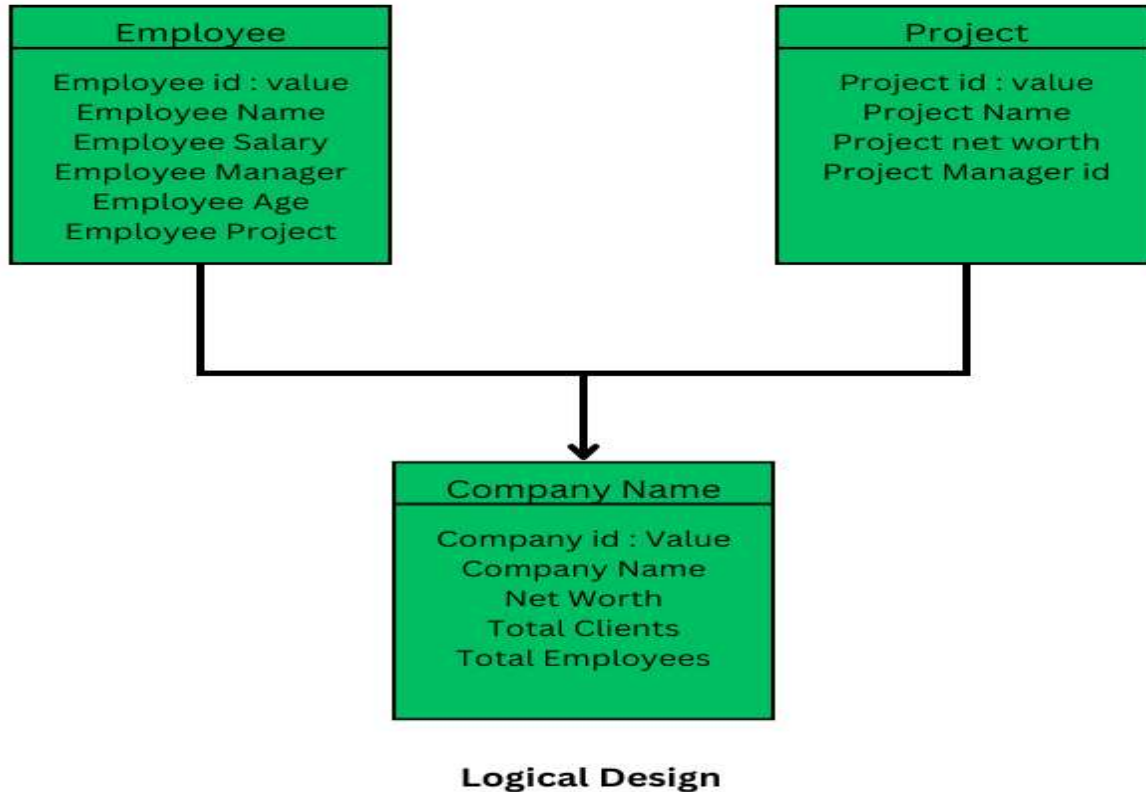
A relationship can be further categorized into - unary, binary, and ternary relationships.

4. Attributes: Attributes are nothing but properties of a specific entity that define its behavior. For example, an employee can have unique_id, name, age, date of birth (DOB), salary, department, Manager, project id, etc.

5. Normalization: After all the entities are put in place and the relationship among data is defined, we need to look for loopholes or possible ambiguities that may arise as a result of CRUD operations. To prevent various Anomalies such as INSERTION, UPDATION, and DELETION Anomalies.

Data Normalization is a basic procedure defined for databases to eliminate such anomalies & prevent redundancy.

An Example of Logical Design



Logical Design Example

Physical Design

The main purpose of the physical design is to actually implement the logical design that is, show the structure of the database along with all the columns & their data types, rows, relations, relationships among data & clearly define how relations are related to each other.

Following are the steps taken in physical design

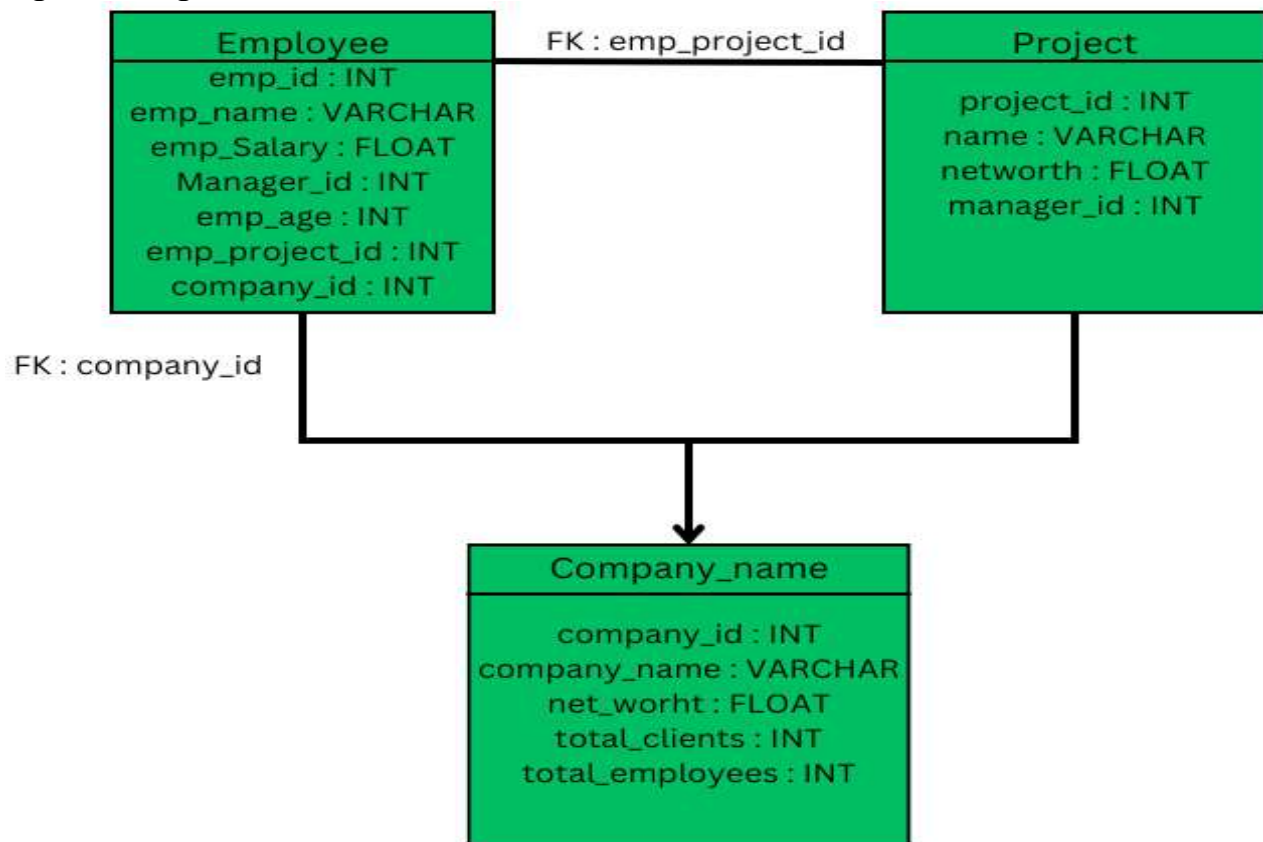
Step 1: Entities are converted into tables or relations that consist of their properties (attributes)

Step 2: Apply integrity constraints: establish foreign key, unique key, and composite key relationships among the data. And apply various constraints.

Step 3: Entity names are converted into table names, property names are translated into attribute names, and so on.

Step 4: Apply normalization & modify as per the requirements.

Step 5: Final Schemes are defined based on the entities & attributes derived in logical design.



Physical Design

UNIT-4TH (TRANSACTION PROCESSING CONCEPT)

WHAT IS A TRANSACTION?

A **transaction** is a group of steps or operations that must all happen together. If even one step fails, the whole group should be canceled.

Example:

Imagine online money transfer from **Ram** to **Shyam**:

- Step 1: Deduct ₹500 from Ram's account
- Step 2: Add ₹500 to Shyam's account

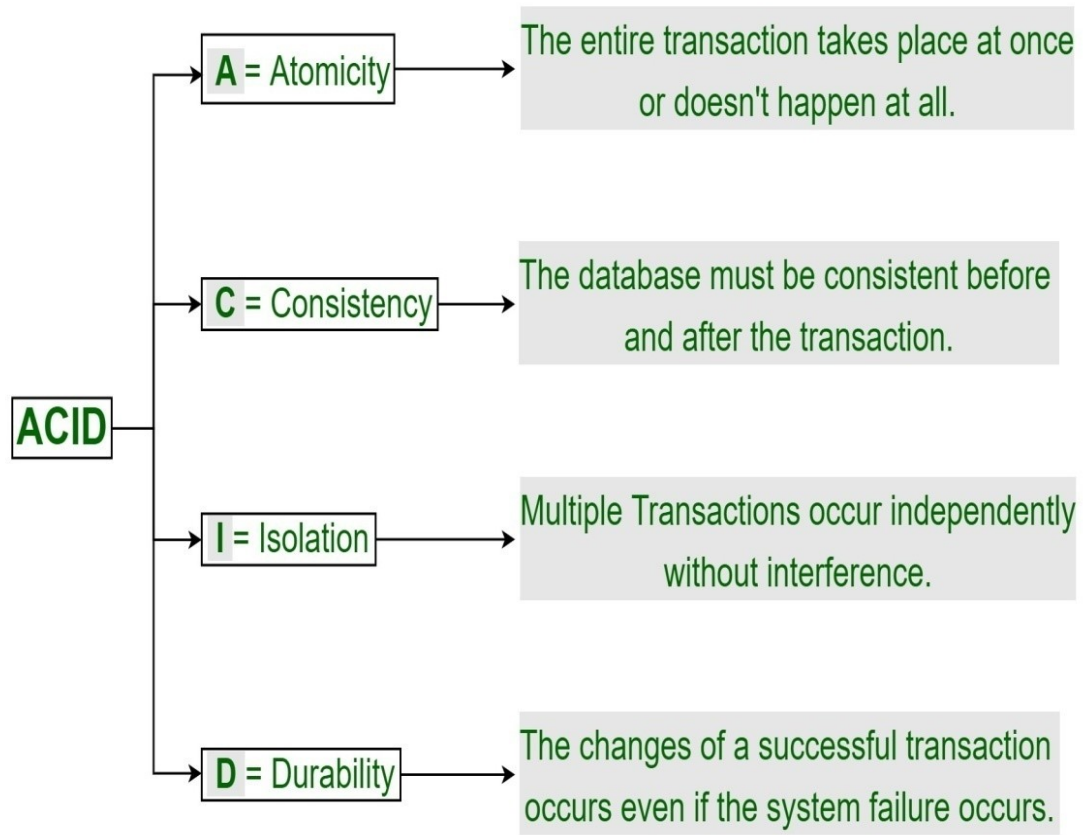
If Step 1 happens but Step 2 fails, money is lost. So both steps must happen **together** or **not at all**. This is a **transaction**.

2. Properties of a Transaction (ACID)

Property	Meaning (Simple)
Atomicity	All steps happen or none
Consistency	Data should remain correct before and after transaction
Isolation	Transactions don't disturb each other
Durability	Once done, it's saved permanently even after system failure

ACID stands for Atomicity, Consistency, Isolation, and Durability. These four key properties define how a transaction should be processed in a reliable and predictable manner, ensuring that the database remains consistent, even in cases of failures or concurrent accesses.

ACID Properties in DBMS



WHAT ARE TRANSACTIONS IN DBMS?

A **transaction** in DBMS refers to a sequence of operations performed as a single unit of work. These operations may involve reading or writing data to the database. To maintain data integrity, DBMS ensures that each transaction adheres to the ACID properties. Think of a transaction like an ATM withdrawal. When we withdraw money from our account, the transaction involves several steps:

- Checking your balance.
- Deducting the money from your account.
- Adding the money to the bank's record.

For the transaction to be successful, **all steps** must be completed. If any part of this process fails (e.g., if there's a system crash), the entire transaction should

fail, and no data should be altered. This ensures the database remains in a **consistent** state.

THE FOUR ACID PROPERTIES

1. ATOMICITY: "All or Nothing"

Atomicity ensures that a transaction is **atomic**, it means that either the entire transaction completes fully or doesn't execute at all. There is no in-between state i.e. transactions do not occur partially. If a transaction has multiple operations, and one of them fails, the whole transaction is rolled back, leaving the database unchanged. This avoids partial updates that can lead to inconsistency.

- **Commit:** If the transaction is successful, the changes are permanently applied.
- **Abort/Rollback:** If the transaction fails, any changes made during the transaction are discarded.

Example: Consider the following transaction **T** consisting of **T1** and **T2** :
Transfer of \$100 from account **X** to account **Y** .

Before: X : 500	Y : 200
Transaction T	
T1	T2
Read (X) X: = X - 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

If the transaction fails after completion of T1 but before completion of T2 , the database would be left in an inconsistent state. With Atomicity, if any part of the transaction fails, the entire process is rolled back to its original state, and no partial changes are made.

2. CONSISTENCY: Maintaining Valid Data States

Consistency ensures that a database remains in a valid state before and after a transaction. It guarantees that any transaction will take the database from one consistent state to another, maintaining the rules and constraints defined for the data. In simple terms, a transaction should only take the database from one **valid** state to another. If a transaction violates any database rules or constraints, it should be rejected, ensuring that only consistent data exists after the transaction.

Example: Suppose the sum of all balances in a bank system should always be constant. Before a transfer, the total balance is **\$700**. After the transaction, the total balance should remain \$700. If the transaction fails in the middle (like updating one account but not the other), the system should maintain its consistency by rolling back the transaction.

Total before T occurs = $500 + 200 = 700$. **Total after T occurs** = $400 + 300 = 700$.

X = 500 Rs Y = 500 Rs	
T	T''
Read (X) $X := X * 100$ Write (X) Read (Y) $Y := Y - 50$ Write (Y)	Read (X) Read (Y) $Z := X + Y$ Write (Z)

3. ISOLATION: Ensuring Concurrent Transactions Don't Interfere

This property ensures that **multiple transactions** can occur concurrently without leading to the **inconsistency** of the database state. Transactions occur independently without interference. Changes occurring in a particular transaction will not be visible to any other transaction until that particular change in that transaction is written to memory or has been committed.

This property ensures that when multiple transactions run at the same time, the result will be the same as if they were run one after another in a specific order. This property prevents issues such as **dirty reads** (reading uncommitted data), **non-repeatable reads** (data changing between two reads in a transaction), and **phantom reads** (new rows appearing in a result set after the transaction starts).

Example: Let's consider two transactions: Consider two transactions **T** and **T''**.

- **X = 500, Y = 500**

X = 500 Rs Y = 500 Rs	
T	T''
Read (X) $X := X * 100$ Write (X) Read (Y) $Y := Y - 50$ Write (Y)	Read (X) Read (Y) $Z := X + Y$ Write (Z)

Transaction T:

- T wants to transfer \$50 from **X** to **Y**.
- T reads **Y** (value: 500), deducts \$50 from **X** (new **X** = **450**), and adds \$50 to **Y** (new **Y** = **550**).

Transaction T'':

- T'' starts and reads **X** (value: 500) and **Y** (value: 500), then calculates the sum: **500 + 500 = 1000**.

But, by the time **T** finishes, **X** and **Y** have changed to **450** and **550** respectively, so the correct sum should be **450 + 550 = 1000**. **Isolation** ensures that **T''** should not see the old values of **X** and **Y** while **T** is still in progress. Both transactions should be independent, and **T''** should only see the final state after **T** commits. This prevents inconsistent data like the incorrect sum calculated by **T''**

4. DURABILITY: Persisting Changes

This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs. These updates now become permanent and are stored in **non-volatile memory**. In the event of a failure, the DBMS can recover the database to the state it was in after the last committed transaction, ensuring that no data is lost.

Example: After successfully transferring money from Account A to Account B, the changes are stored on disk. Even if there is a crash immediately after the commit, the transfer details will still be intact when the system recovers, ensuring durability.

How ACID Properties Impact DBMS Design and Operation

The ACID properties, in totality, provide a mechanism to ensure the correctness and consistency of a database in a way such that each transaction is a group of operations that acts as a single unit, produces consistent results, acts in isolation from other operations, and updates that it makes are durably stored.

1. Data Integrity and Consistency

ACID properties safeguard the **data integrity** of a DBMS by ensuring that transactions either complete successfully or leave no trace if interrupted. They prevent **partial updates** from corrupting the data and ensure that the database transitions only between valid states.

2. Concurrency Control

ACID properties provide a solid framework for **managing concurrent transactions**. Isolation ensures that transactions do not interfere with each other, preventing data anomalies such as lost updates, temporary inconsistency, and uncommitted data.

3. Recovery and Fault Tolerance

Durability ensures that even if a system crashes, the database can recover to a consistent state. Thanks to the **Atomicity** and **Durability** properties, if a transaction fails midway, the database remains in a consistent state.

Advantages of ACID Properties in DBMS

1. **Data Consistency:** ACID properties ensure that the data remains consistent and accurate after any transaction execution.
2. **Data Integrity:** It maintains the integrity of the data by ensuring that any changes to the database are permanent and cannot be lost.
3. **Concurrency Control:** ACID properties help to manage multiple transactions occurring concurrently by preventing interference between them.
4. **Recovery:** ACID properties ensure that in case of any failure or crash, the system can recover the data up to the point of failure or crash.

4. TRANSACTION SYSTEM

A transaction refers to a sequence of one or more operations (such as read, write, update, or delete) performed on the database as a single logical unit of work. A transaction ensures that either all the operations are successfully executed (committed) or none of them take effect (rolled back). Transactions are designed to maintain the integrity, consistency and reliability of the database, even in the case of system failures or concurrent access.

All types of database access operation which are held between the beginning and end transaction statements are considered as a single logical transaction.

During the transaction the database is inconsistent. Only once the database is committed the state is changed from one consistent state to another.

Example:

Let's consider an **online banking application**:

Transaction: When a user performs a **money transfer**, several operations occur, such as:

- **Reading** the account balance of the sender.
- **Writing** the deducted amount from the sender's account.
- **Writing** the added amount to the recipient's account.

In a **transaction**, all these steps should either complete successfully or, if any error occurs, the database should **rollback** to its previous state, ensuring no partial data is written to the system.

Facts about Database Transactions

- A transaction is a program unit whose execution may or may not change the contents of a database.
- The transaction is executed as a single unit.
- If the database operations do not update the database but only retrieve data, this type of transaction is called a read-only transaction.
- A successful transaction can change the database from one CONSISTENT STATE to another.
- DBMS transactions must be atomic, consistent, isolated and durable.
- If the database were in an inconsistent state before a transaction, it would remain in the inconsistent state after the transaction.

Operations of Transaction

A user can make different types of requests to access and modify the contents of a database. So, we have different types of operations relating to a transaction. They are discussed as follows:

1) Read(X)

A read operation is used to read the value of a particular database element **X** and stores it in a temporary buffer in the main memory for further actions such as displaying that value.

Example:

For a **banking system**, when a user checks their balance, a **Read** operation is performed on their **account balance**:

```
SELECT balance FROM accounts WHERE account_id = 'A123';
```

This updates the balance of the user's account after withdrawal.

2) Write(X)

A write operation is used to write the value to the database from the buffer in the main memory. For a write operation to be performed, first a read operation is performed to bring its value in buffer, and then some changes are made to it, e.g. some set of arithmetic operations are performed on it according to the user's request, then to store the modified value back in the database, a write operation is performed.

Example:

For the **banking system**, if a user withdraws money, a **Write** operation is performed after the balance is updated:

```
UPDATE accounts SET balance = balance - 100 WHERE account_id = 'A123';
```

This updates the balance of the user's account after withdrawal.

3) Commit

This operation in transactions is used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database, i.e. it might get inconsistent before and after the transaction.

Example:

After a successful money transfer in a banking system, a **Commit** operation finalizes the transaction:

```
COMMIT;
```

Once the transaction is committed, the changes to the database are permanent, and the transaction is considered **successful**.

4) Rollback

If an error occurs, the **Rollback** operation undoes all the changes made by the transaction, reverting the database to its last consistent state. In simple words, it can be said that a rollback operation does undo the operations of transactions that were performed before its interruption to achieve a safe state of the database and avoid any kind of ambiguity or inconsistency.

Example:

Suppose during the money transfer process, the system encounters an issue, like insufficient funds in the sender's account. In that case, the transaction is rolled back:

ROLLBACK;

This will undo all the operations performed so far and ensure that the database remains consistent.

A **transaction system** is a software system that:

- Manages transactions
- Makes sure data is safe and correct
- Supports many users at the same time

Main parts:

Part	Role
Transaction Manager	Starts, ends transactions
Concurrency Control	Makes sure transactions don't interfere
Recovery Manager	Fixes the database after a crash
Buffer Manager	Handles memory and data access

5. TESTING OF SERIALIZABILITY

What is a serializable schedule?

If a non-serial schedule can be transformed into its corresponding serial schedule, it is said to be serializable. Simply said, a non-serial schedule is referred to as a serializable schedule if it yields the same results as a serial timetable.

Non-serial Schedule

A schedule where the transactions are overlapping or switching places. As they are used to carry out actual database operations, multiple transactions are running at once. It's possible that these transactions are focusing on the same data set. Therefore, it is crucial that non-serial schedules can be serialized in order for our database to be consistent both before and after the transactions are executed.

Example:

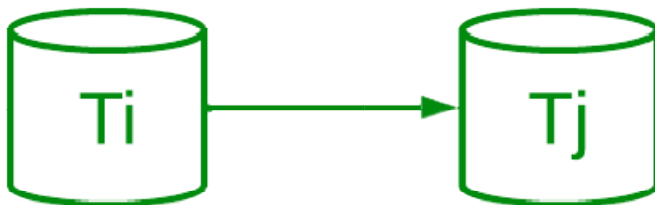
Transaction-1	Transaction-2
R(a)	
W(a)	
	R(b)
	W(b)

Transaction-1	Transaction-2
R(b)	
	R(a)
W(b)	
	W(a)

We can observe that Transaction-2 begins its execution before Transaction-1 is finished, and they are both working on the same data, i.e., "a" and "b", interchangeably. Where "R"-Read, "W"-Write

Serializability testing

We can utilize the Serialization Graph or Precedence Graph to examine a schedule's serializability. A schedule's full transactions are organized into a Directed Graph, what a serialization graph is.



Precedence Graph

It can be described as a Graph $G(V, E)$ with vertices $V = "V1, V2, V3, ..., Vn"$ and directed edges $E = "E1, E2, E3, ..., En"$. One of the two operations—READ or WRITE—performed by a certain transaction is contained in the collection of edges. Where $T_i \rightarrow T_j$, means Transaction- T_i is either performing read or write before the transaction- T_j .

TYPES OF SERIALIZABILITY

There are two ways to check whether any non-serial schedule is serializable.



1. CONFLICT SERIALIZABILITY

Conflict serializability refers to a subset of serializability that focuses on maintaining the consistency of a database while ensuring that identical data items are executed in an order. In a DBMS each transaction has a value and all the transactions, in the database rely on this uniqueness. This uniqueness ensures that no two operations with the conflict value can occur simultaneously.

For example let's consider an order table and a customer table as two instances. Each order is associated with one customer even though a single client may place orders. However there are restrictions for achieving conflict serializability in the database. Here are a few of them.

1. Different transactions should be used for the two procedures.
2. The identical data item should be present in both transactions.
3. Between the two operations, there should be at least one write operation.

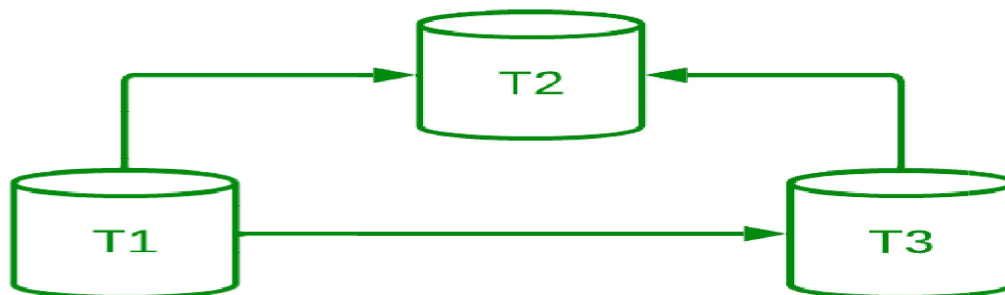
Example

Three transactions—t1, t2, and t3—are active on a schedule "S" at once. Let's create a graph of precedence.

Transaction - 1 (t1)	Transaction - 2 (t2)	Transaction - 3 (t3)
R(a)		
	R(b)	
		R(b)
	W(b)	

Transaction - 1 (t1)	Transaction - 2 (t2)	Transaction - 3 (t3)
W(a)		
		W(a)
	R(a)	
	W(a)	

It is a conflict serializable schedule as well as a serial schedule because the graph (a DAG) has no loops. We can also determine the order of transactions because it is a serial schedule.



As there is no incoming edge on Transaction 1, Transaction 1 will be executed first. T3 will run second because it only depends on T1. Due to its dependence on both T1 and T3, t2 will finally be executed.

Therefore, the serial schedule's equivalent order is: t1 --> t3 --> t2

Note: A schedule is unquestionably consistent if it is conflicting serializable. A non-conflicting serializable schedule, on the other hand, might or might not be serial. We employ the idea of View Serializability to further examine its serial behavior.

2. VIEW SERIALIZABILITY

View serializability is a kind of operation in a serializable in which each transaction should provide some results, and these outcomes are the output of properly sequentially executing the data item. The view serializability, in contrast to conflict serialized, is concerned with avoiding database inconsistency. The view serializability feature of DBMS enables users to see databases in contradictory ways.

To further understand view serializability in DBMS, we need to understand the schedules S1 and S2. The two transactions T1 and T2 should be used to establish these two schedules. Each schedule must follow the three transactions in order to retain the equivalent of the transaction. These three circumstances are listed below.

1. The first prerequisite is that the same kind of transaction appears on every schedule. This requirement means that the same kind of group of transactions cannot appear on both schedules S1 and S2. The schedules are not equal to one another if one schedule commits a transaction but it does not match the transaction of the other schedule.
2. The second requirement is that different read or write operations should not be used in either schedule. On the other hand, we say that two schedules are not similar if schedule S1 has two write operations whereas schedule S2 only has one. The number of the write operation must be the same in both schedules, however there is no issue if the number of the read operation is different.
3. The last requirement is that there should not be a conflict between execution order for a single data item. Assume, that schedule S1's transaction is T1, and schedule S2's transaction is T2. The data item A is written by both the transaction T1 and the transaction T2. The schedules are not equal in this instance. However, we referred to the schedule as equivalent to one another if it had the same number of all write operations in the data item.

What is view equivalency?

Schedules (S1 and S2) must satisfy these two requirements in order to be viewed as equivalent:

1. The same piece of data must be read for the first time. For instance, if transaction t1 is reading "A" from the database in schedule S1, then t1 must also read A in schedule S2.
2. The same piece of data must be used for the final write. As an illustration, if transaction t1 updated A last in S1, it should also conduct final write in S2.
3. The middle sequence need to follow suit. As an illustration, if in S1 t1 is reading A, and t2 updates A, then in S2 t1 should read A, and t2 should update A.

View Serializability refers to the process of determining whether a schedule's views are equivalent.

Example

We have a schedule "S" with two concurrently running transactions, "t1" and "t2."

Schedule - S:

Transaction-1 (t1)	Transaction-2 (t2)
R(a)	
W(a)	
	R(a)
	W(a)
R(b)	
W(b)	
	R(b)
	W(b)

By switching between both transactions' mid-read-write operations, let's create its view equivalent schedule (S').

Schedule - S':

Transaction-1 (t1)	Transaction-2 (t2)
R(a)	
W(a)	
R(b)	
W(b)	

Transaction-1 (t1)	Transaction-2 (t2)
	R(a)
	W(a)
	R(b)
	W(b)

It is a view serializable schedule since a view similar schedule is conceivable.

***Note:** A conflict serializable schedule is always viewed as serializable, but vice versa is not always true.*

Advantages of Serializability

1. **Execution is predictable:** In serializable, the DBMS's threads are all performed simultaneously. The DBMS doesn't include any such surprises. In DBMS, no data loss or corruption occurs and all variables are updated as intended.
2. DBMS executes each thread independently, making it much simpler to understand and troubleshoot each database thread. This can greatly simplify the debugging process. The concurrent process is therefore not a concern for us.
3. **Lower Costs:** The cost of the hardware required for the efficient operation of the database can be decreased with the aid of the serializable property. It may also lower the price of developing the software.
4. **Increased Performance:** Since serializable executions provide developers the opportunity to optimize their code for performance, they occasionally outperform non-serializable equivalents.

Why do we need it?

When multiple users run transactions at the same time, their steps may **mix**. We must check if the result is still correct – as if they were done one by one (serially).

This check is called **Serializability Testing**.

Summary in Simple Table

Concept	Simple Meaning	Example
Transaction	Group of actions done together	Money transfer
Atomicity	All or none	If error in step 2, cancel step 1
Consistency	Database stays correct	Balance before and after matches
Isolation	No mix-up	Two transfers don't affect each other
Durability	Saved forever	Data remains even after power cut
Conflict Serializability	Steps don't interfere wrongly	Graph test: No cycle = OK
Transaction System	Manages all of the above	DBMS like Oracle, MySQL, etc.

1. What is a Schedule in DBMS?

A **schedule** in DBMS is a sequence of operations (read, write, commit, etc.) from one or more transactions. It defines the order in which operations are executed in a database system.

For example, if you have:

- Transaction T1: Read(A), Write(A)
- Transaction T2: Read(B), Write(B)

A schedule could be:

S = R1(A) R2(B) W1(A) W2(B)

2. Conflicting Operations

Two operations are said to **conflict** if:

1. They belong to different transactions.
2. They access the same data item.
3. At least one of them is a **write** operation.

Examples of Conflicting Operations:

Operation 1	Operation 2	Conflict?	Reason
R1(A)	R2(A)	No	Both read
R1(A)	W2(A)	Yes	Read-Write conflict
W1(A)	R2(A)	Yes	Write-Read conflict
W1(A)	W2(A)	Yes	Write-Write conflict

3. Conflict Serializable Schedule

A **conflict-serializable** schedule is one that can be transformed into a **serial schedule** (no interleaving of transactions) by **swapping non-conflicting operations**.

Serial Schedule:

Transactions are executed one after another:

S1: R1(A) W1(A) R2(B) W2(B)

Non-serial but Conflict Serializable Example:

Consider schedule:

S = R1(A) R2(A) W1(A) W2(A)

Check for Conflicts:

- R1(A) and R2(A): No conflict (read-read)
- W1(A) and R2(A): Yes (write-read)
- W2(A) and W1(A): Yes (write-write)

We create a **precedence graph**:

- T1 → T2 (due to W1(A) before R2(A))
- T1 → T2 (due to W1(A) before W2(A))

Since the graph has no cycles, the schedule is **conflict serializable** (equivalent to T1 → T2).

4. View Serializable Schedule

A schedule is **view serializable** if it is **view equivalent** to some serial schedule. View serializability is a broader concept than conflict serializability.

Conditions for View Equivalence:

1. **Initial Reads:** Same transaction reads the initial value.
2. **Read-From:** For each read in the schedule, the transaction it reads from must be the same in both schedules.
3. **Final Writes:** Final write on each data item must be from the same transaction.

Example:

Schedule:

S = R1(A) W1(A) R2(A) W2(A)

- T1 reads and writes A.
- T2 reads A (which is written by T1), then writes A.

This schedule may not be **conflict serializable** (due to write-read conflicts), but it can be **view serializable**, because:

- T1 does the initial read and first write.
- T2 reads the value written by T1.
- T2 writes the final value.

So it's view equivalent to:

T1 → T2

How Log-Based Recovery Works ?

Transaction Log:

- The log stores all changes made by transactions, ensuring recoverability.
- Each transaction's start, changes (with old and new values), and its commit or abort state are recorded.

Recovery Process:

After a system crash, the database uses the log to determine:

- **Undo:** Transactions that started but didn't commit (incomplete transactions) are undone to reverse their changes.
- **Redo:** Transactions that committed before the crash are redone to ensure their changes are applied to the database.

1. RECOVERABILITY

What is Recoverability?

In DBMS, **recoverability** ensures that **transactions commit only if all the transactions from which they read data also commit**. This protects the database from inconsistent states when failures occur.

Types of Recoverability:

a) Recoverable Schedule

A schedule where if **T2** reads data written by **T1**, then **T2 commits only after T1 commits**.

Example (Recoverable Schedule):

```
T1:  W1(A)
T2:   R2(A)
T1:   COMMIT
T2:   COMMIT
```

Here, T2 reads from T1 and waits for T1 to commit → Recoverable

b) Non-Recoverable Schedule

Occurs when **T2 commits before T1**, even though T2 read a value written by T1.

✗Example (Non-Recoverable Schedule):

```
T1:  W1(A)
T2:   R2(A)
T2:   COMMIT
```

T1: ABORT

Here, T2 committed but later T1 aborted. This means T2 has used an invalid value
→ ✗Not Recoverable.

c) Cascading Schedule / Cascade less Schedule

- **Cascading Rollback:** If one transaction fails, others that read from it must also roll back.
- **Cascadeless Schedule:** Transactions **only read committed data** to prevent cascading rollback.

Example (Cascadeless Schedule):

T1: W1(A) COMMIT

T2: R2(A) COMMIT

T2 reads committed value of A ⇒ No rollback required

2. RECOVERY FROM TRANSACTION FAILURES (Database Recovery Techniques in DBMS)

Database Systems like any other computer system, are subject to failures but the data stored in them must be available as and when required. When a database fails it must possess the facilities for fast recovery. It must also have atomicity i.e. either transactions are completed successfully and committed (the effect is recorded permanently in the database) or the transaction should have no effect on the database.

TYPES OF RECOVERY TECHNIQUES IN DBMS

Database recovery techniques are used in database management systems (DBMS) to restore a database to a consistent state after a failure or error has occurred. The main goal of recovery techniques is to ensure data integrity and consistency and prevent data loss.

There are mainly two types of recovery techniques used in DBMS

- **Rollback/Undo Recovery Technique**
- **Commit/Redo Recovery Technique**
- **Checkpoint Recovery Technique**

Database recovery techniques ensure data integrity in case of system failures. Understanding how these techniques work is crucial for managing databases effectively.

ROLLBACK/UNDO RECOVERY TECHNIQUE

The rollback/undo recovery technique is based on the principle of backing out or undoing the effects of a transaction that has not been completed successfully due to a system failure or error. This technique is accomplished by undoing the changes made by the transaction using the log records stored in the transaction log. The transaction log contains a record of all the transactions that have been performed

on the database. The system uses the log records to undo the changes made by the failed transaction and restore the database to its previous state.

COMMIT/REDO RECOVERY TECHNIQUE

The commit/redo recovery technique is based on the principle of reapplying the changes made by a transaction that has been completed successfully to the database. This technique is accomplished by using the log records stored in the transaction log to redo the changes made by the transaction that was in progress at the time of the failure or error. The system uses the log records to reapply the changes made by the transaction and restore the database to its most recent consistent state.

CHECKPOINT RECOVERY TECHNIQUE

Checkpoint Recovery is a technique used to improve data integrity and system stability, especially in databases and distributed systems. It entails preserving the system's state at regular intervals, known as checkpoints, at which all ongoing transactions are either completed or not initiated. This saved state, which includes memory and CPU registers, is kept in stable, non-volatile storage so that it can withstand system crashes. In the event of a breakdown, the system can be restored to the most recent checkpoint, which reduces data loss and downtime. The frequency of checkpoint formation is carefully regulated to decrease system overhead while ensuring that recent data may be restored quickly.

A **transaction failure** occurs when a transaction cannot complete successfully due to:

- Power failure
- System crash
- Deadlock
- Software bugs

Transaction States

1. **Active** → transaction is executing.
2. **Partially committed** → finished execution, ready to commit.
3. **Failed** → an error has occurred.
4. **Aborted** → transaction is rolled back.
5. **Committed** → all changes are saved permanently.

Recovery Process

The system must:

- **UNDO** the changes made by **uncommitted** transactions.
- **REDO** the changes made by **committed** transactions.

Example:

Assume we have:

- Transaction T1: Update A = 200

- Original A = 100

Scenario:

1. <T1, A, 100, 200>
2. T1 COMMIT
3. System crash

During recovery:

- T1 has committed $\Rightarrow$ Apply **REDO**
- Final value of A = 200

If T1 had **not committed**:

- Apply **UNDO**
- Restore A = 100

3. LOG-BASED RECOVERY

What is a Log?

A **log** is a sequence of records maintained to track all updates made to the database. It stores:

- Transaction ID
- Data Item
- Old Value
- New Value
- Commit or Abort

Log Entry Format:

<transaction-id, data-item, old-value, new-value>

Example:

Transaction T1 updates A from 100 to 200:

1. <T1, A, 100, 200>
2. T1 COMMIT

TYPES OF LOG-BASED RECOVERY

a) Deferred Update (Deferred Write)

- All changes are logged.
- Data is updated **only at commit**.
- **Only REDO** is needed during recovery.

Example:

<T1, A, 100, 200>

T1 COMMIT

On crash after commit:

- Just **REDO** T1

b) Immediate Update

- Data may be updated **before** commit.

- Changes are logged **before actual write** (Write-Ahead Logging).
- On crash:
 - **UNDO** uncommitted transactions.
 - **REDO** committed transactions.

Example:

<T1, A, 100, 200>

System Crash (no commit)

Recovery:

- **UNDO** T1 → Restore A = 100

4. CHECKPOINTS in DBMS

What is a Checkpoint?

A **checkpoint** is a mechanism that helps in **minimizing the recovery time** by marking a safe point in the log from which recovery should start.

A checkpoint marks a point in time where the DBMS ensures that the database is in a consistent state and all prior transactions have been committed. During transaction execution, these checkpoints are recorded periodically.

After a checkpoint is reached:

- The transaction log up to that point can be safely discarded, as all corresponding updates have already been written to the database.
- A new log file is then created to record changes from subsequent transactions.
- This process continues, allowing the system to limit the amount of log data and improve recovery efficiency by avoiding scanning the entire log history.

Checkpoints in Real Time

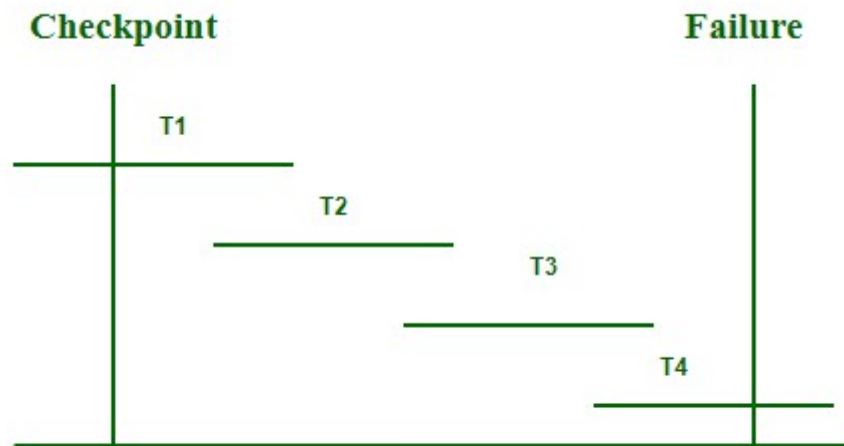
In a real-time environment, transaction logs are continuously created, which can consume a significant amount of storage space. Tracking and maintaining every update may also increase the physical storage requirements of the system. Over time, the transaction log file can grow so large that it becomes difficult to manage effectively. This issue is addressed through the use of checkpoints.

A checkpoint is a process that removes all previous transaction logs from active storage and saves the necessary information to permanent storage. This helps control the size of the transaction log and improves system efficiency.

Steps to Use Checkpoints in the Database

- Write the `begin_checkpoint` record into a log.
- Collect checkpoint data in stable storage.
- Write the `end_checkpoint` record into a log.

The behavior when the system crashes and recovers when concurrent transactions are executed is shown below:



Understanding Checkpoints in multiple Transactions

Transactions and operations of the above diagram:

Transaction 1 (T1)	Transaction 2 (T2)	Transaction 3 (T3)	Transaction 4 (T4)
START			
	START		
	COMMIT		
		START	
		COMMIT	
			START
			FAILURE

Recovery Rule Summary

- **Redo List:** Transactions that have a COMMIT log (whether or not a START is shown).
- **Undo List:** Transactions that have a START but no COMMIT or ABORT i.e., incomplete transactions at the time of failure.

Types of Checkpoints

There are basically two main types of Checkpoints:

Automatic Checkpoint

- These checkpoints occur at regular intervals, such as every hour or every day.
- The frequency is configured by the database administrator (DBA).
- They are commonly used in large or heavily used databases where frequent updates occur.
- Automatic checkpoints help ensure that data can be recovered quickly and efficiently in case of a system failure.

Manual Checkpoint

- These checkpoints are explicitly triggered by the **DBA**, rather than occurring on a set schedule.
- They are typically used in smaller or less active databases.
- Manual checkpoints are created infrequently, only when the DBA decides it is necessary (e.g., before maintenance or system upgrades).

Purpose:

- Save the current state of the database.
- Avoid scanning the entire log during crash recovery.
- Reduces the time to redo/undo.

How Checkpoints Work:

1. All modified buffers are written to disk.
2. Log is flushed to disk.
3. A special log record <CHECKPOINT> is written.

Example:

Log Sequence:

1. <T1, A, 100, 200>
2. <T2, B, 50, 70>
3. <CHECKPOINT>
4. T1 COMMIT
5. <T2, B, 70, 90>
6. System Crash

During recovery:

- Start reading logs from <CHECKPOINT>
- **REDO** T1 (committed)
- **UNDO** T2 (not committed)

Summary Table

Concept	Purpose	Actions on Recovery
Recoverability	Ensures valid commits	Delay commit if needed
Transaction Failures	Handle crashes/errors	UNDO/REDO as needed
Log-Based Recovery	Record of changes to support recovery	Use logs to undo/redo
Deferred Update	Update after commit only	REDO only
Immediate Update	Immediate update with logging	UNDO + REDO
Checkpoints	Reduce recovery time	Start from last checkpoint

Visual Example

Here's a simplified visual flow of recovery:

Transaction Execution



Write Log Entry



Checkpoint



System Crash



Recovery:

- Start from Checkpoint
- REDO committed transactions
- UNDO uncommitted transactions

WHAT IS A DEADLOCK IN DBMS?

A **deadlock** is a situation in a multi-transaction system where **two or more transactions are waiting for each other to release resources**, and none of them can proceed.

Real-life Analogy:

Two people are trying to pass through a narrow hallway from opposite ends. Both wait for the other to move, but neither moves. → Deadlock.

Conditions for Deadlock

According to **Coffman's conditions**, all **four** must be true for a deadlock to occur:

1. **Mutual Exclusion:** A resource can be held by only one transaction at a time.
2. **Hold and Wait:** A transaction holds one resource and waits for another.
3. **No Preemption:** A resource cannot be forcibly taken from a transaction.

4. **Circular Wait:** A circular chain of transactions exists, each waiting for a resource held by the next.

Example of Deadlock in DBMS

Assume two transactions:

- **T1** locks **A** and needs **B**
- **T2** locks **B** and needs **A**

Step-by-step:

T1: LOCK(A) → Success

T2: LOCK(B) → Success

T1: LOCK(B) → Waiting...

T2: LOCK(A) → Waiting...

Now, T1 waits for B (held by T2), and T2 waits for A (held by T1) → **Deadlock**

DISTRIBUTED DATA STORAGE

Distributed data storage refers to how data is stored across multiple physical locations (nodes or sites) in a distributed database system.

Key Concepts:

- **Fragmentation:**
 - **Horizontal:** Rows of a table are divided.
 - **Vertical:** Columns of a table are divided.
 - **Mixed (Hybrid):** Combination of horizontal and vertical.
- **Replication:**
 - Duplicating data across sites for availability and fault tolerance.
 - **Full:** Entire database is copied.
 - **Partial:** Only some fragments are copied.
- **Allocation:**
 - Deciding where to store each fragment or replica.
 - Can be centralized, partitioned, or fully replicated.

Goals:

- Minimize data access time
- Improve availability and fault tolerance
- Reduce communication costs

2. CONCURRENCY CONTROL

Concurrency control ensures that simultaneous operations on the database do not conflict and maintain data integrity.

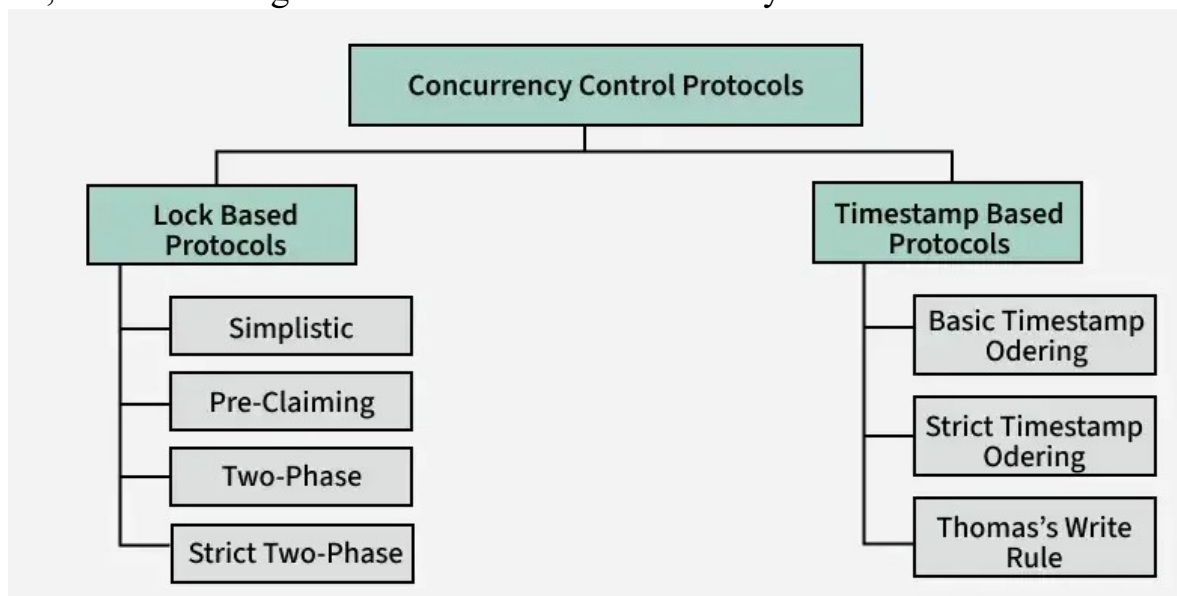
- Concurrency control is a concept in Database Management Systems (DBMS) that ensures multiple transactions can simultaneously access or modify data without causing errors or inconsistencies. It provides mechanisms to handle the concurrent execution in a way that maintains ACID properties.

- By implementing concurrency control, a DBMS allows transactions to execute concurrently while avoiding issues such as deadlocks, race conditions, and conflicts between operations.
- The main goal of concurrency control is to ensure that simultaneous transactions do not lead to data conflicts or violate the consistency of the database. The concept of serializability is often used to achieve this goal.

Concurrent Execution can lead to various challenges:

- **Dirty Reads:** One transaction reads uncommitted data from another transaction, leading to potential inconsistencies if the changes are later rolled back.
- **Lost Updates:** When two or more transactions update the same data simultaneously, one update may overwrite the other, causing data loss.
- **Inconsistent Reads:** A transaction may read the same data multiple times during its execution, and the data might change between reads due to another transaction, leading to inconsistency.
- **Without Concurrency Control:** Transactions interfere with each other, causing issues like lost updates, dirty reads or inconsistent results.
- **With Concurrency Control:** Transactions are properly managed (e.g., using locks or timestamps) to ensure they execute in a consistent, isolated manner, preserving data accuracy.

Concurrency control is critical to maintaining the accuracy and reliability of databases in multi-user environments. By preventing conflicts and inconsistencies during concurrent transactions, it ensures the database remains consistent and correct, even under high levels of simultaneous activity.



Techniques:

- **Two-Phase Locking (2PL):**
 - Ensures serializability by acquiring all locks before releasing any.
- **Timestamp Ordering:**
 - Transactions are ordered based on timestamps to avoid conflicts.
- **Optimistic Concurrency Control:**
 - Transactions execute without restrictions and validate before committing.

Challenges in Distributed Systems:

- Distributed deadlocks
- Increased latency
- Clock synchronization (important for timestamp methods)

LOCK BASED CONCURRENCY CONTROL PROTOCOL IN DBMS

In a DBMS, lock-based concurrency control is a method used to manage how multiple transactions access the same data. This protocol ensures data consistency and integrity when multiple users interact with the database simultaneously.

This method uses locks to manage access to data, ensuring transactions don't clash and everything runs smoothly when multiple transactions happen at the same time. In this article, we'll take a closer look at how the Lock-Based Protocol works.

What is a Lock?

A lock is a control mechanism used in databases to manage concurrent access to data items. It is a variable associated with a data item that indicates whether it is in use. Locks prevent multiple transactions from accessing the same data simultaneously, ensuring data integrity, consistency, and preventing conflicts during concurrent operations.

Lock Based Protocols

Lock-Based Protocols in DBMS ensure that a transaction cannot read or write data until it gets the necessary lock. Here's how they work:

- These protocols prevent concurrency issues by allowing only one transaction to access a specific data item at a time.
- Locks help multiple transactions work together smoothly by managing access to the database items.
- Locking is a common method used to maintain the serializability of transactions.
- A transaction must acquire a read lock or write lock on a data item before performing any read or write operations on it.

TYPES OF LOCK

1. **Shared Lock (S):** Shared Lock is also known as Read-only lock. As the name suggests it can be shared between transactions because while holding this lock the transaction does not have the permission to update data on the data item. S-lock is requested using lock-S instruction.
2. **Exclusive Lock (X):** Data item can be both read as well as written. This is Exclusive and cannot be held simultaneously on the same data item. X-lock is requested using lock-X instruction.

Rules of Locking

The basic rules for Locking are given below:

Read Lock (or) Shared Lock(S)

- If a Transaction has a Read lock on a data item, it can read the item but not update it.
- If a transaction has a Read lock on the data item, other transaction can obtain Read Lock on the data item but no Write Locks.
- So, the Read Lock is also called a Shared Lock.

Write Lock (or) Exclusive Lock (X)

- If a transaction has a write Lock on a data item, it can both read and update the data item.
- If a transaction has a write Lock on the data item, then other transactions cannot obtain either a Read lock or write lock on the data item.
- So, the Write Lock is also known as Exclusive Lock.

Concurrency Control Protocols

Concurrency Control Protocols are the methods used to manage multiple transactions happening at the same time. They ensure that transactions are executed safely without interfering with each other, maintaining the accuracy and consistency of the database.

These protocols prevent issues like data conflicts, lost updates or inconsistent data by controlling how transactions access and modify data.

Types of Lock-Based Protocols

1. Simplistic Lock Protocol

It is the simplest method for locking data during a transaction. Simple lock-based protocols enable all transactions to obtain a lock on the data before inserting, deleting, or updating it. It will unlock the data item once the transaction is completed.

Example:

Consider a database with a single data item $X = 10$.

Transactions:

- **T1:** Wants to read and update X.
- **T2:** Wants to read X.

Steps:

1. T1 requests an exclusive lock on X to update its value. The lock is granted.
 - T1 reads $X = 10$ and updates it to $X = 20$.
2. T2 requests a shared lock on X to read its value. Since T1 is holding an exclusive lock, T2 must wait.
3. T1 completes its operation and releases the lock.
4. T2 now gets the shared lock and reads the updated value $X = 20$.

This example shows how simplistic lock protocols handle concurrency but do not prevent problems like deadlocks or limits concurrency.

2. Pre-Claiming Lock Protocol

The Pre-Claiming Lock Protocol requires a transaction to request all needed locks in advance, before execution begins. If all locks are granted, the transaction proceeds; otherwise, it rolls back and waits. This approach prevents deadlocks by ensuring a transaction only runs when it can lock all required data items upfront.

Example:

Consider two transactions T1 and T2 and two data items, X and Y:

1. Transaction T1 declares that it needs:

- A write lock on X.
- A read lock on Y.

Since both locks are available, the system grants them. T1 starts execution:

- It updates X.
- It reads the value of Y.

2. While T1 is executing, Transaction T2 declares that it needs:

- A read lock on X.

However, since T1 already holds a write lock on X, T2's request is denied. T2 must wait until T1 completes its operations and releases the locks.

3. Once T1 finishes, it releases the locks on X and Y. The system now grants the read lock on X to T2, allowing it to proceed.

This method is simple but may lead to inefficiency in systems with a high number of transactions.

3. Two-phase locking (2PL)

A transaction is said to follow the Two-Phase Locking protocol if Locking and Unlocking can be done in two phases :

- **Growing Phase:** New locks on data items may be acquired but none can be released.

- **Shrinking Phase:** Existing locks may be released but no new locks can be acquired.

For more detail refer the article Two-phase locking (2PL).

4. Strict Two-Phase Locking Protocol

Strict Two-Phase Locking requires that in addition to the 2-PL all Exclusive(X) locks held by the transaction be released until after the Transaction Commits.

For more details refer the article Strict Two-Phase Locking Protocol.

Problem With Simple Locking

S.No	T1	T2
1	lock-X(B)	
2	read(B)	
3	B:=B-50	
4	write(B)	
5		lock-S(A)
6		read(A)
7		lock-S(B)
8	lock-X(A)	
9		

3. DIRECTORY SYSTEM

The directory system keeps track of where data is stored in a distributed database.

Functions:

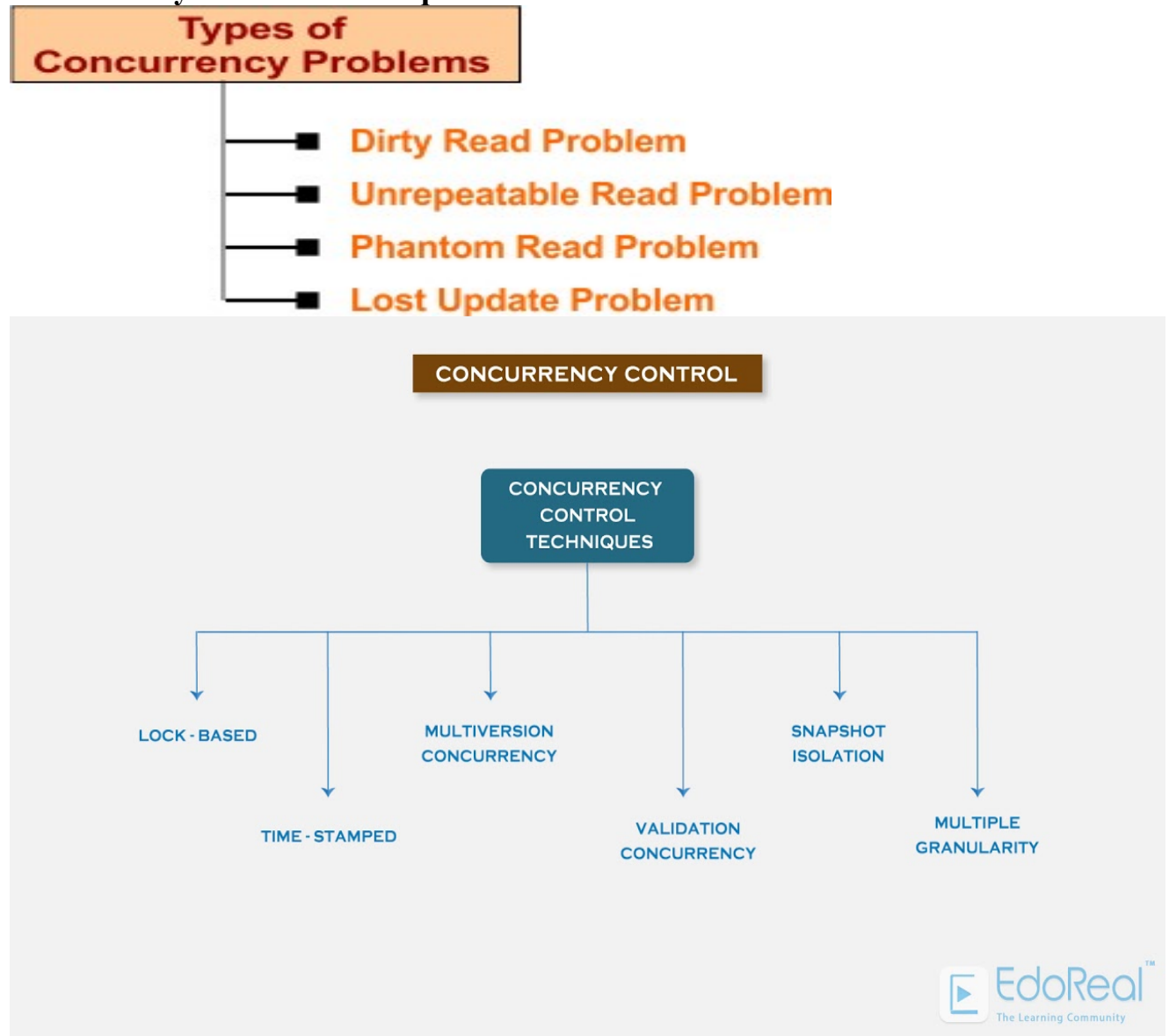
- Maps logical data names to physical storage locations.
- Keeps metadata such as:
 - Location of fragments
 - Replication status
 - Node status and health

Types:

- **Centralized Directory:** Single point of control (may be a bottleneck).
- **Distributed Directory:** Metadata is distributed; more scalable and fault-tolerant.

UNIT-5 (CONCURRENCY CONTROL TECHNIQUES)

Concurrency Control Techniques



WHAT IS CONCURRENCY CONTROL?

When many people (users) try to use or update the same database at the same time, we need to **control their actions** so that the data does not get **mixed up or wrong**. This control is called **concurrency control**.

1. Lock-Based Technique

Think like a door lock: Only one person can enter the room at a time if the door is locked.

Example:

- Two people (Amit and Neha) try to update the same bank account.
- Amit gets the lock first and updates the balance.
- Neha has to **wait** until Amit finishes.
- After Amit is done, Neha can go next.

This prevents both from changing the data at the same time.

2. Timestamp Ordering Technique

Think like a queue (first come, first serve): Everyone gets a number when they come in.

Example:

- Amit starts his work at 10:00 AM.
- Neha starts at 10:05 AM.
- If Neha tries to change data that Amit is using, the system checks the time and lets **Amit finish first**, because he came earlier.

Helps to follow the correct order.

3. Optimistic Technique (OCC)

Think like checking your homework: Do everything first, and then check for mistakes before submitting.

Example:

- Amit reads data and does his work.
- Before saving (committing), the system checks: "Did anyone else change this data?"
 - If **yes** → Amit's work is **cancelled** (rollback).
 - If **no** → Amit's work is **saved**.

Good when people rarely work on the same data.

4. Multiversion Technique (MVCC)

Think like Google Docs version history: You can see old versions and new versions.

Example:

- Amit is reading a record (old version).
- Neha updates the record (new version).
- Amit still sees the old version.
- Neha sees the new version.

Both can work **without disturbing each other**.

- Concurrency control is a fundamental concept in correct execution of simultaneous transactions without violating data integrity. Below are some characteristics of concurrency control:

- It enforces isolation among transactions.
- It preserve database consistency through consistency preserving execution of transactions.
- It resolve read-write and write-read conflicts.

Concurrency Control Techniques (In Details)

Concurrency control techniques in DBMS ensure that multiple transactions can execute simultaneously without conflicting with each other. These techniques maintain data consistency and prevent issues like lost updates, dirty reads, or deadlocks. Various concurrency control techniques are:

1. *Two-phase locking Protocol*
2. *Time stamp ordering Protocol*
3. *Multi version concurrency control*
4. *Validation concurrency control*

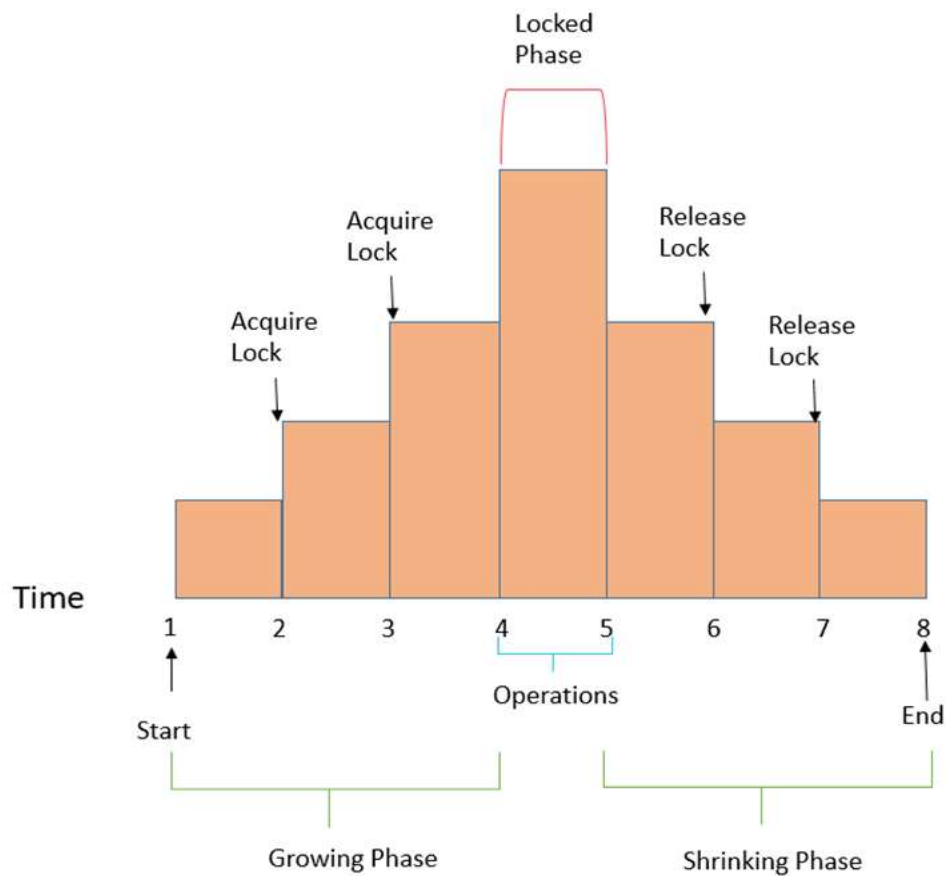
Concurrency control techniques in DBMS manage simultaneous operations without conflicts. Techniques like lock-based protocols, timestamp ordering, and optimistic concurrency control ensure that database transactions remain consistent, even when multiple transactions access the same data concurrently. These methods prevent problems such as deadlocks, dirty reads, and lost updates.

These are briefly explained below.

1. TWO-PHASE LOCKING PROTOCOL



Time	T1	T2
t_0	BEGIN TRANSACTION	BEGIN TRANSACTION
t_1	Get Write Lock for A	
t_2	Get Write Lock for B	
t_3	Update $A = A + 1$	
t_4	Update $B = B + 1$	
t_5	UNLOCK A	
t_6	UNLOCK B	
t_7		Get Write Lock for A
t_8		Get Write Lock for B
t_9		Update $A = A * 2$
t_{10}		Update $B = B * 2$
t_{11}		UNLOCK A
t_{12}		UNLOCK B
t_{13}	COMMIT TRANSACTION	COMMIT TRANSACTION



Locking is an operation which secures: permission to read, OR permission to write a data item. Two phase locking is a process used to gain ownership of shared resources without creating the possibility of deadlock. The 3 activities taking place in the two phase update algorithm are:

1. *Lock Acquisition*
2. *Modification of Data*
3. *Release Lock*

Two phase locking prevents deadlock from occurring in distributed systems by releasing all the resources it has acquired, if it is not possible to acquire all the resources required without waiting for another process to finish using a lock. This means that no process is ever in a state where it is holding some shared resources, and waiting for another process to release a shared resource which it requires. This means that deadlock cannot occur due to resource contention

A transaction in the Two Phase Locking Protocol can assume one of the 2 phases:

- **Growing Phase:** In this phase a transaction can only acquire locks but cannot release any lock. The point when a transaction acquires all the locks it needs is called the Lock Point.
- **Shrinking Phase:** In this phase a transaction can only release locks but cannot acquire any.

2. Time Stamp Ordering Protocol

A timestamp is a tag that can be attached to any transaction or any data item, which denotes a specific time on which the transaction or the data item had been used in any way. A timestamp can be implemented in 2 ways. One is to directly assign the current value of the clock to the transaction or data item. The other is to attach the value of a logical counter that keeps increment as new timestamps are required. The timestamp of a data item can be of 2 types:

- **W-timestamp(X):** This means the latest time when the data item X has been written into.
- **R-timestamp(X):** This means the latest time when the data item X has been read from. These 2 timestamps are updated each time a successful read/write operation is performed on the data item X.

3. Multi-version Concurrency Control

Multi-version schemes keep old versions of data item to increase concurrency. Multiversion 2 phase locking: Each successful write results in the creation of a new version of the data item written. Timestamps are used to label the versions. When a read(X) operation is issued, select an appropriate version of X based on the timestamp of the transaction.

4. Validation Concurrency Control

The optimistic approach is based on the assumption that the majority of the database operations do not conflict. The optimistic approach requires neither

locking nor time stamping techniques. Instead, a transaction is executed without restrictions until it is committed. In Validation Concurrency Control each transaction moves through 2 or 3 phases, referred to as read, validation and write

- During read phase, the transaction reads the database, executes the needed computations and makes the updates to a private copy of the database values. All update operations of the transactions are recorded in a temporary update file, which is not accessed by the remaining transactions.
- During the validation phase, the transaction is validated to ensure that the changes made will not affect the integrity and consistency of the database. If the validation test is positive, the transaction goes to a write phase. If the validation test is negative, the transaction is restarted and the changes are discarded.
- During the write phase, the changes are permanently applied to the database.

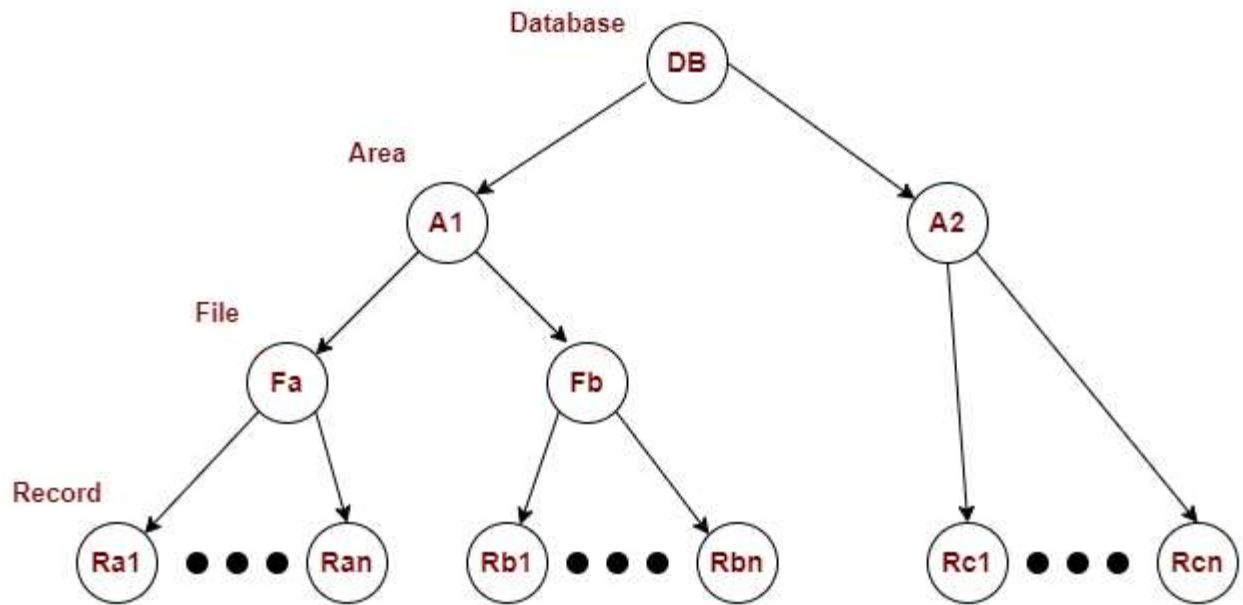
MULTIPLE GRANULARITY LOCKING

Multiple granularity locking is a locking mechanism that provides different levels of locks for different database objects. It allows for different locks at different levels of granularity. This mechanism allows multiple transactions to lock different levels of granularity, ensuring that conflicts are minimized, and concurrency is maximized.

To illustrate, let's consider a tree structure that has four levels of nodes. The top level represents the entire database, and below it are nodes of type "area", which represent specific areas of the database. Each area has child nodes called "files", and each file represents a specific subset of data within that area. Importantly, no file can span more than one area.

Finally, each file has child nodes called "records", which represent individual units of data within the file. Like files, each record is a child node of its corresponding file and cannot be present in more than one file. Therefore, the tree can be divided into the following levels, starting from the top ?

- Database
- Area
- File
- Record



Multi Granularity tree Hierarchy

Multiple granularity locking uses two types of locks

Shared Lock

It allows multiple transactions to read the same data simultaneously. It is used to prevent other transactions from modifying the data while a transaction is reading it.

Exclusive Lock

It prevents any other transaction from accessing the data. It is used to prevent other transactions from reading or modifying the data while a transaction is writing to it.

Different Types of Intention Mode Locks in Multiple Granularity

Intention mode locks are a type of lock used in multiple granularity locking that allows multiple transactions to acquire locks on the same resource, but with different levels of access.

There are three types of intention mode locks in multiple granularity locking ?

Intent Shared (IS) Locks

This lock is used when a transaction needs to read a resource but does not intend to modify it. It indicates that the transaction wants to acquire a Shared lock on a resource.

Intent Exclusive (IX) Locks

This lock is used when a transaction needs to modify a resource but does not intend to share it. It indicates that the transaction wants to acquire an Exclusive lock on a resource.

Shared with Intent Exclusive (SIX) Locks

This lock is used when a transaction intends to acquire both Shared and Exclusive locks on a resource. It indicates that the transaction wants to acquire an Exclusive lock on a resource after acquiring Shared locks on other resources.

These intention mode locks are used to optimize the locking mechanism in a database by allowing transactions to acquire locks on multiple resources in a coordinated manner. They help prevent deadlocks and improve concurrency in a database system.

WHAT IS MULTI-VERSION CONCURRENCY CONTROL (MVCC) IN DBMS?

Multi-Version Concurrency Control is a technology, utilized to enhance databases by resolving concurrency problems and also data locking by preserving older database versions. When many tasks attempt to update the same piece of data simultaneously, MVCC causes a conflict and necessitates a retry from one or more of the processes.

Types of Multi-Version Concurrency Control (MVCC) in DBMS

Below are some types of Multi-Version Concurrency Control (MVCC) in DBMS

- **Timestamp-based MVCC:** The data visibility to transactions is defined by the unique timestamp assigned to each transaction that creates a new version of a record.
- **Snapshot-based MVCC:** This utilizes the database snapshot that is created at the beginning of a transaction to supply the information that is needed for the transaction.
- **History-based MVCC:** This Keeps track of every modification made to a record, making transaction rollbacks simple.
- **Hybrid MVCC:** This coordinates data flexibility and performance by combining two or more MVCC approaches.

How Does Multi-Version Concurrency Control (MVCC) in DBMS Works?

- In the database, every tuple has a version number. The tuple with the greatest version number can have a read operation done on it simultaneously.
- Only a copy of the record may be used for writing operations.

- While the copy is being updated concurrently, the user may still view the previous version.
- The version number is increased upon successful completion of the writing process.
- The upgraded version is now used for every new record operation and every time there is an update, this cycle is repeated.

Implementation of Multi-Version Concurrency Control (MVCC) in DBMS

- MVCC operates time stamps (TS) and increases transaction IDs to assure transactional consistency. MVCC manage many copies of the object, ensuring that a transaction (T) never has to wait to read a database object (P).
- A specific transaction T_i can read the most recent version of the object, which comes before the transaction's Read Timestamp $RTS(T_i)$ since each version of object P contains both a Read Timestamp and a Write Timestamp.
- If there are other pending transactions to the same object that have an earlier Read Timestamp (RTS), then a Write cannot be completed.
- You cannot finish your checkout transaction until the person in front of you has finished theirs, much as when you are waiting in a queue at the shop.

Advantages of Multi-Version Concurrency Control (MVCC) in DBMS

Below are some advantages of Multi-Version Concurrency Control in DBMS

- **The reduced read-and-write necessity for database locks:** The database can support many read-and-write transactions without locking the entire system thanks to MVCC.
- **Increased Concurrency:** This Enables several users to use the system at once.
- **Minimize read operation delays:** By enabling concurrent read operations, MVCC helps to cut down on read times for data.
- **Accuracy and consistency:** Preserves data integrity over several concurrent transactions.

Disadvantages of Multi-Version Concurrency Control (MVCC) in DBMS

Below are some disadvantages of Multi-Version Concurrency Control in DBMS

- **Overhead:** Keeping track of many data versions might result in overhead.
- **Garbage collecting:** To get rid of outdated data versions, MVCC needs effective garbage collecting systems.
- **Increase the size of the database:** Expand the capacity of the database since MVCC generates numerous copies of the records and/or tuples.
- **Complexity:** Compared to more straightforward locking systems, MVCC usage might be more difficult.

Example: MVCC in a Library System

Imagine a **digital library** where multiple students can **read or update** the same book record at the same time.

Initial Setup:

- Book "Database Systems" has **100 copies**.
- This data has **Version 1** (timestamp: T0).

Student A (Transaction T1):

- Starts reading the book record at **time T1**.
- Sees **Version 1** (100 copies).
- T1 is a **read-only** transaction — it doesn't change anything.

Student B (Transaction T2):

- Starts after T1, at **time T2**.
- Updates the record: **donates 50 more copies**.
- Now the book has **150 copies**.
- This creates **Version 2** (timestamp: T2).

What Happens?

- **T1 (Student A)** still sees the old version with **100 copies** even though the data was updated later.
- **T2 (Student B)** sees the new version with **150 copies**.
- Both transactions work **independently and without blocking each other**.

Benefit of MVCC:

- Readers don't block writers.
- Writers don't block readers.
- Each transaction works on its own **snapshot** of the data.
- **No locking** needed for simple operations.

• RECOVERY WITH CONCURRENT TRANSACTIONS

Concurrency control means that multiple transactions can be executed at the same time and then the interleaved logs occur. But there may be changes in transaction results so maintain the order of execution of those transactions. During recovery, it would be very difficult for the recovery system to backtrack all the logs and then start recovering. Recovery with concurrent transactions can be done in the following four ways.

1. Interaction with concurrency control
2. Transaction rollback
3. Checkpoints
4. Restart recovery

Interaction with concurrency control : In this scheme, the recovery scheme depends greatly on the concurrency control scheme that is used. So, to rollback a failed transaction, we must undo the updates performed by the transaction. **Transaction rollback :**

- In this scheme, we rollback a failed transaction by using the log.
- The system scans the log backward a failed transaction, for every log record found in the log the system restores the data item.

CHECKPOINTS :

- Checkpoints is a process of saving a snapshot of the applications state so that it can restart from that point in case of failure.
- Checkpoint is a point of time at which a record is written onto the database from the buffers.
- Checkpoint shortens the recovery process.
- When it reaches the checkpoint, then the transaction will be updated into the database, and till that point, the entire log file will be removed from the file. Then the log file is updated with the new step of transaction till the next checkpoint and so on.
- The checkpoint is used to declare the point before which the DBMS was in the consistent state, and all the transactions were committed.

To ease this situation, 'Checkpoints' Concept is used by the most DBMS.

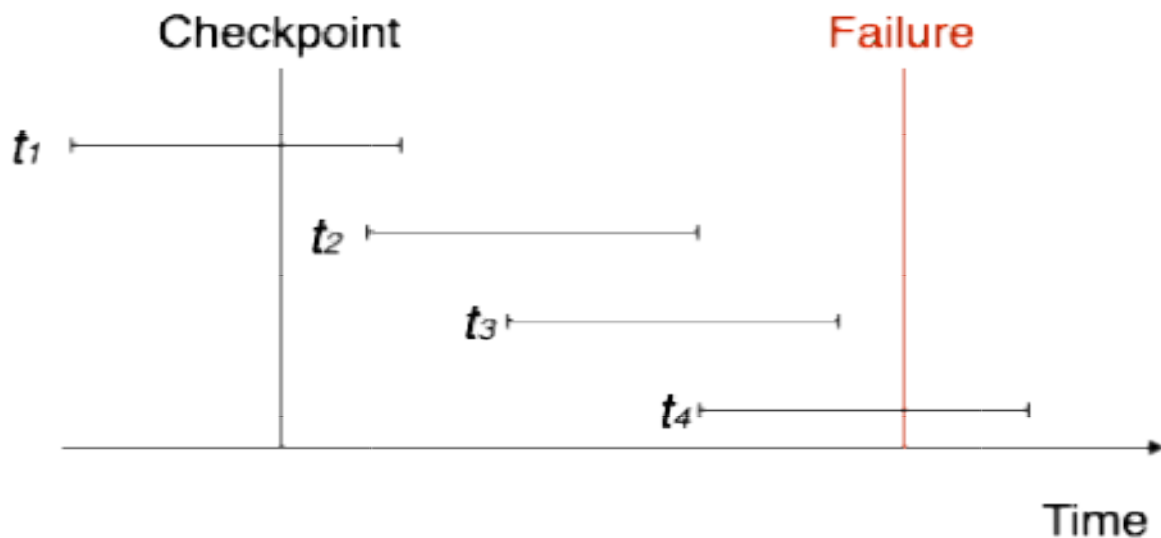
- In this scheme, we used checkpoints to reduce the number of log records that the system must scan when it recovers from a crash.

Checkpoint

- A **checkpoint** is a point of synchronization between the database and the transaction log
 - DBMS refuses new requests, finishes processing outstanding requests, and writes its buffers to disk
 - The DBMS waits until the writing is successfully completed → the log and the database are synchronized
- Checkpoints speed up database recovery process
 - Database can be recovered using after-images since the last checkpoint
 - Checkpoint can be done several times per hour
- Most DBMS products automatically checkpoint themselves

DAVID M. KROENKE'S DATABASE PROCESSING, 10th Edition
© 2006 Pearson Prentice Hall

9-20



RESTART RECOVERY :

- When the system recovers from a crash, it constructs two lists.
- The undo-list consists of transactions to be undone, and the redo-list consists of transaction to be redone.

- The system constructs the two lists as follows: Initially, they are both empty. The system scans the log backward, examining each record, until it finds the first <checkpoint> record.

- **CASE STUDY OF ORACLE**

Essay on Oracle Corporation

Oracle Corporation is one of the biggest software companies in the world. It is known for making database software, cloud services, and many tools used by large businesses. Oracle helps companies store, manage, and use data in a better way.

1. How Oracle Started

Oracle was started in **1977** by three people: **Larry Ellison, Bob Miner, and Ed Oates**. They built a software that could store and manage data using a special method called "relational database". This idea became very popular. In 1986, Oracle became a public company and was listed on the stock market.

2. What Oracle Does

Oracle makes many types of software and services that help companies run their work smoothly. Its main products are:

- **Oracle Database** – A powerful tool to store and manage a lot of data.
- **Oracle Cloud** – A service that lets companies store their data and run applications on the internet (cloud).
- **Fusion Cloud Applications** – These are programs for accounting, human resources, supply chain, and customer service.
- **Java and MySQL** – Two popular programming tools used by software developers. Oracle got them by buying another company called Sun Microsystems.

3. Oracle's Business Style

Oracle mostly works with other businesses (not directly with normal people). It sells software, cloud services, and some computer hardware to big companies, banks, hospitals, and government offices. These customers use Oracle products to handle important business operations.

4. Important Acquisitions (Companies Oracle Bought)

Oracle has grown bigger by buying other companies. Some important ones are:

- **PeopleSoft** – For HR and finance software.
- **Siebel** – For customer management software.
- **Sun Microsystems** – Gave Oracle Java and MySQL.
- **NetSuite** – A cloud company for small businesses.
- **Cerner** – A health tech company, helping Oracle enter the healthcare field.

5. Moving to Cloud Computing

At first, Oracle sold software that customers installed on their own computers. But now, businesses want cloud services so they can access everything online. Oracle was slow to move to the cloud, but now it offers **Oracle Cloud Infrastructure (OCI)** and cloud applications.

Oracle also launched the **Autonomous Database**, which uses Artificial Intelligence to fix itself and work automatically. This saves time and effort.

6. Competitors and Problems

Oracle has many strong competitors like:

- **Amazon Web Services (AWS)** – Leading in cloud services
- **Microsoft Azure** – Offers both cloud and software tools
- **Salesforce and SAP** – Experts in business software

Oracle also faced problems like:

- Being slow to change to cloud technology
- Legal cases (like the one with Google over Java)
- Tough competition in the software market

7. Oracle Today

Today, Oracle makes over \$50 billion every year. Its cloud business is growing fast. By buying Cerner, Oracle is now also helping hospitals manage health records using technology. Oracle is also using AI and machine learning to improve its software.

Oracle is a strong and important company in the world of technology. From building databases to cloud services and healthcare software, Oracle helps many businesses around the world. Even though it faced some challenges, it is still growing and improving. Oracle's smart planning, new ideas, and strong leadership have helped it stay successful for over 45 years.

- **ESSAY ON ORACLE DBMS (DATABASE MANAGEMENT SYSTEM)**

Introduction

Oracle DBMS is one of the most popular and powerful database management systems in the world. It is used by large companies, banks, hospitals, schools, and even governments to store, manage, and protect huge amounts of data. Oracle DBMS is made by Oracle Corporation.

What is Oracle DBMS?

Oracle DBMS is a software system that helps users to **create, store, update, and retrieve** data easily and securely. It uses a type of database called a **Relational Database**, which means data is stored in **tables** made up of rows and columns.

For example, a student database may have a table like this:

Student_ID	Name	Age	Class
101	Ravi	15	10
102	Sita	16	11

With Oracle DBMS, we can add more students, search for a student, update their details, or delete a record.

Features of Oracle DBMS

1. **Relational Model** Data is stored in structured tables with rows and columns.
2. **SQL Support** It uses **SQL (Structured Query Language)** to perform all database tasks.
3. **High Security** Oracle DBMS provides strong security features to protect data from unauthorized access.

4. **Multi-user Access** Many users can access the database at the same time without conflicts.
5. **Backup and Recovery** It has tools to back up the data and recover it in case of a failure.
6. **Transaction Control** It supports **ACID** properties (Atomicity, Consistency, Isolation, Durability) to keep data correct and safe.
7. **Scalability**
It can handle small databases as well as huge databases with millions of records.

Uses of Oracle DBMS

Oracle DBMS is used in many areas:

- **Banking** – to manage customer accounts and transactions.
- **Healthcare** – to store patient records.
- **Education** – for student and staff databases.
- **Retail** – to keep track of products, sales, and customers.
- **Government** – to manage citizen records and documents.

Benefits of Oracle DBMS

- Saves time and effort in managing data.
- Makes data handling fast and efficient.
- Keeps data safe and secure.
- Can handle very large amounts of data.
- **Works on many platforms (Windows, Linux, UNIX).**

Oracle DBMS is a powerful tool that helps businesses and organizations manage their data in a safe, organized, and efficient way. With features like high security, fast processing, and easy data handling, Oracle DBMS has become one of the most trusted database systems in the world. It is an essential part of many modern applications and industries today.